

**PROGRAM STUDI SARJANA SISTEM INFORMASI
PENELITIAN TUGAS AKHIR**



***Performance Analysis of Random Forest and XGBoost for Phishing Website
Detection with Integrated Rule-Based Filtering***

OLEH:

12S22005	Nikita Simanjuntak
12S22022	Grace Arintya Siahaan
12S22026	Ruben Martogi Jeremia Sianipar

**PROGRAM STUDI SARJANA SISTEM INFORMASI
FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO
INSTITUT TEKNOLOGI DEL
LAGUBOTI
2025**

ABSTRAK

Nama : 1. Nikita Simanjuntak
2. Grace Arintya Siahaan
3. Ruben Martogi Jeremia Sianipar
Program Studi : Sarjana Sistem Informasi
Judul : Analisis Kinerja Random Forest dan XGBoost untuk
Deteksi Situs Web Phishing dengan Integrasi Rule-Based
Filtering

Kata kunci: *phishing, Machine learning, Random Forest, XGBoost, rule-based filtering*

ABSTRACT

Names : 1. Nikita Simanjuntak
2. Grace Arintya Siahaan
3. Ruben Martogi Jeremia Sianipar
Study Program : *Bachelor of Information Systems*
Title : *Performance Analysis of Random Forest and XGBoost for
Phishing Website Detection with Integrated Rule-Based
Filtering*

Key word: *phishing, Machine learning, Random Forest, XGBoost, rule-based
filtering*

DAFTAR ISI

ABSTRAK	I
ABSTRACT	II
DAFTAR ISI	III
DAFTAR GAMBAR.....	VIII
DAFTAR TABEL.....	IX
DAFTAR POTONGAN KODE.....	X
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Pertanyaan Penelitian	7
1.3 Tujuan Penelitian	7
1.4 Ruang Lingkup.....	7
1.5 Sistematika penyajian	8
BAB II LANDASAN TEORI	11
2.1 <i>Phishing</i>	11
2.2 Deteksi <i>Phishing</i> pada <i>Website</i>	13
2.3 <i>Feature Extraction</i>	14
2.4 <i>Machine learning</i>	16
2.5 <i>Random Forest</i>	17
2.5.1 <i>Description</i>	17
2.5.2 <i>Hyperparameters of Random Forests</i>	18
2.5.2.1 RF <i>Hyperparameter</i> num.trees	18
2.5.2.2 RF <i>Hyperparameter</i> mtry.....	19
2.5.2.3 RF <i>Hyperparameter</i> Sample.fraction.....	20
2.5.2.4 RF <i>Hyperparameter</i> Replace (<i>bootstrap</i>)	21
2.5.2.5 RF <i>Hyperparameter</i> Respect.unordered.factors	22

2.6 <i>XGBoost</i>	24
2.6.1 <i>Description</i>	24
2.6.2 <i>Hyperparameter of XGBoost</i>	25
2.6.2.1 <i>XGBoost Hyperparameter nrounds</i>	25
2.6.2.2 <i>XGBoost Hyperparameter eta</i>	26
2.6.2.3 <i>XGBoost Hyperparameter lambda</i>	28
2.6.2.4 <i>XGBoost Hyperparameter alpha</i>	28
2.6.2.5 <i>XGBoost Hyperparameter subsample</i>	29
2.6.2.6 <i>XGBoost Hyperparameter colsample_bytree</i>	30
2.6.2.7 <i>XGBoost Hyperparameter gamma</i>	31
2.6.2.8 <i>XGBoost Hyperparameter maxdepth</i>	32
2.6.2.9 <i>XGBoost Hyperparameter min_child_weight</i>	32
2.7 <i>Rule-Based Filtering</i>	33
2.7.1 <i>Struktur URL</i>	37
2.8 <i>Evaluasi</i>	39
2.9 <i>Penelitian Terdahulu</i>	42
BAB III ANALISIS	45
3.1 <i>Analisis Dataset</i>	45
3.1.1 <i>Data Preparation</i>	45
3.1.1.1 <i>Pengumpulan Data</i>	45
3.1.1.2 <i>Atribut Dataset</i>	46
3.1.2 <i>Data Preprocessing</i>	51
3.1.2.1 <i>Data Selection</i>	52
3.1.2.2 <i>Feature Engineering</i>	61
3.1.2.3 <i>Data Transformation</i>	62
3.1.2.4 <i>Data Cleaning</i>	62

3.2 Analisis <i>Machine Learning</i>	63
3.2.1 Analisis <i>Random Forest</i>	63
3.2.2 Analisis <i>XGBoost</i>	66
3.3 Analisis <i>Stacking</i> menggunakan <i>Logistic Regression</i>	69
3.4 Analisis <i>Rule based Filtering</i>	71
3.4.1 <i>Rule 1: High-Confidence Rule</i>	76
3.4.2 <i>Rule 2 : Threshold-Based Rule</i>	77
3.5 Analisis Eksperimen	79
3.5.1 Eksperimen <i>Random Forest</i>	79
3.5.2 Eksperimen <i>XGBoost</i>	79
3.5.3 Eksperimen <i>Stacking</i>	80
3.5.4 Eksperimen <i>Rule based Filtering</i> dengan <i>Stacking</i>	81
BAB IV DESAIN	85
4.1 Metode Penelitian	85
4.2 Rancangan Sistem	87
4.2.1 Desain Umum Sistem	87
4.3 Desain Analisis Dataset	90
4.3.1 Desain <i>Data Preparation</i>	92
4.3.2 Desain <i>Data Preprocessing</i>	93
4.3.2.1 Desain <i>Data Selection</i>	95
4.3.2.2 Desain <i>Feature Engineering</i>	96
4.3.2.3 Desain <i>Transformation Data</i>	98
4.3.2.4 Desain <i>Data Cleaning</i>	99
4.4 Rancangan Arsitektur <i>Machine Learning</i>	99
4.4.1 Rancangan Arsitektur <i>Random Forest</i>	100
4.4.2 Rancangan Arsitektur <i>XGBoost</i>	101

4.5 Rancangan Arsitektur <i>Rule based Filtering</i>	103
4.6 Rancangan Arsitektur Eksperimen	104
4.7 <i>Mockup</i> Desain Sistem	106
BAB V IMPLEMENTASI	108
5.1 Lingkungan Implementasi	108
5.1.1 Perangkat Keras (<i>Hardware</i>)	108
5.1.2 Perangkat Lunak (<i>Software</i>)	108
5.2 Batasan Implementasi	109
5.3 Implementasi Analisis Dataset	109
5.3.1 Implementasi <i>Data Preparation</i>	109
5.3.1.1 Pembacaan Dataset	109
5.3.1.2 Jumlah baris dan kolom	110
5.3.1.3 Pengecekan <i>Missing Values</i>	110
5.3.1.4 Pengecekan Duplikat	111
5.3.1.5 Distribusi Label Status	111
5.3.1.6 Korelasi Antar Fitur	112
5.3.2 Implementasi <i>Data Preprocessing</i>	112
5.3.2.1 <i>Data Selection</i>	112
5.3.2.2 <i>Feature Engineering</i>	113
5.3.2.3 <i>Data Transformation</i>	118
5.3.2.4 <i>Data Cleaning</i>	119
5.4 Implementasi Model	119
5.5 Implementasi Eksperimen	109
5.4.1 Eksperimen <i>Random Forest</i>	119
5.4.2 Eksperimen <i>XGBoost</i>	120
5.4.3 Eksperimen <i>Stacking</i>	120

5.4.4 Eksperimen <i>Rule based Filtering</i> dengan <i>Stacking</i>	120
5.6 Implementasi Evaluasi	120
DAFTAR PUSTAKA	121

DAFTAR GAMBAR

Gambar 2. 1 Jenis <i>Machine Learning</i>	16
Gambar 2. 2 Struktur URL	38
Gambar 4. 1 Tahapan Penelitian	85
Gambar 4. 2 Desain Umum Sistem.....	89
Gambar 4. 3 Desain Analisis Dataset.....	91
Gambar 4. 4 Desain <i>Data Preparation</i>	93
Gambar 4. 5 Desain <i>Data Preprocessing</i>	95
Gambar 4. 6 Desain <i>Data Selection</i>	96
Gambar 4. 7 Desain <i>Feature Engineering</i>	97
Gambar 4. 8 Desain <i>Transformation Data</i>	98
Gambar 4. 9 Desain Data Cleaning.....	99
Gambar 4. 10 Rancangan Arsitektur <i>Random Forest</i>	100
Gambar 4. 11 Rancangan Arsitektur <i>XGBoost</i>	102
Gambar 4. 12 Rancangan Arsitektur <i>Rule based Filtering</i>	104
Gambar 4. 13 Desain Eksperimen.....	105
Gambar 4. 14 <i>Mockup</i> Desain Sistem	107

DAFTAR TABEL

Tabel 2. 1 Tren dan Jenis Serangan <i>Phishing</i>	12
Tabel 2. 2 Contoh Proses Ekstraksi Fitur dari URL <i>Phishing</i>	15
Tabel 2. 3 <i>Characteristics of Rule-based Filtering</i>	33
Tabel 2. 4 Contoh <i>Rule-Based</i> dalam Deteksi <i>Phishing</i>	36
Tabel 2. 5 Komponen URL dan Data Fitur.....	38
Tabel 2. 6 <i>Confusion Matrix</i>	39
Tabel 2. 7 Penelitian Terdahulu	42
Tabel 3. 1 Karakteristik Dataset <i>Phishing</i>	45
Tabel 3. Atribut pada Dataset dataset_phishing.....	46
Tabel 3. 3 <i>Feature Selection</i>	52
Tabel 3. 4 Analisis Pemilihan <i>Hyperparameter Random Forest</i>	63
Tabel 3. 5 Analisis Pemilihan <i>Hyperparameter XGBoost</i>	65
Tabel 3. 6 Metode Umum <i>Ensamble Learning</i>	69
Tabel 3. 7 <i>Rule base Filtering</i>	72
Tabel 3. 8 Kategori Fitur.....	77
Tabel 3. 9 Nilai Skor Risiko pada <i>Rule-based</i>	78

DAFTAR POTONGAN KODE

Kode 5. 1 Membaca dataset	109
Kode 5. 2 Menampilkan jumlah baris dan kolom	110
Kode 5. 3 Implementasi Pengecekan Missing Values	110
Kode 5. 4 Implementasi Pengecekan Data Duplikat.....	111
Kode 5. 5 Implementasi Analisis Distribusi Kelas Target.....	111
Kode 5. 6 Implementasi korelasi antar fitur.....	112
Kode 5. 7 Implementasi data selection	113

BAB I

PENDAHULUAN

Bab pendahuluan berisi penjelasan terkait latar belakang pemilihan topik tugas akhir, tujuan yang ingin dicapai dalam tugas akhir, ruang lingkup penelitian, tahapan yang dilakukan dalam penelitian, dan sistematika penyajian laporan tugas akhir.

1.1 Latar Belakang

Dalam perkembangan dunia digital saat ini, aktivitas pengguna internet semakin meningkat [1]. Namun, peningkatan ini juga diikuti oleh munculnya berbagai ancaman terhadap keamanan informasi. Salah satu ancaman yang paling sering terjadi adalah serangan *phishing* [2].

Phishing merupakan bentuk kejahatan siber yang memanfaatkan teknik manipulasi untuk menipu korban agar mengakses situs web palsu dan menyerahkan informasi penting, seperti kredensial akun, data keuangan, atau informasi pribadi lainnya. [3] Serangan ini dapat menimbulkan kerugian yang signifikan dan mencakup berbagai aspek, antara lain pencurian dana dari rekening atau kartu kredit, penyalahgunaan data pribadi korban untuk tindak penipuan, serta kerusakan reputasi individu maupun perusahaan. [4] Selain itu phishing yang dilakukan dengan menyamar sebagai entitas terpercaya dalam komunikasi elektronik mengganggu stabilitas sistem keuangan dan menurunkan kepercayaan publik terhadap layanan digital. Besarnya dampak yang ditimbulkan menggambarkan bahwa aktivitas *phishing* terus berkembang dan mengalami peningkatan dari waktu ke waktu.

Menurut laporan *Anti Phishing Working Group (APWG)* yang mencatat aktivitas serangan selama Januari hingga Maret 2025, tercatat sebanyak 1.003.924 serangan phishing yang terjadi di seluruh dunia. Jumlah ini merupakan yang tertinggi sejak akhir tahun 2023 dan menunjukkan tren peningkatan yang bertahap dari 877.536 awal 2024 naik menjadi 932.923 pada pertengahan tahun, dan 989.123 akhir 2024 [5]. Peningkatan ini menggambarkan bahwa phishing masih menjadi ancaman

serius di ranah keamanan siber. APWG juga melaporkan bahwa sektor layanan keuangan, pembayaran daring, dan perbankan menjadi target utama dengan total 30,9% dari seluruh serangan yang terjadi [5].

Seiring dengan meningkatnya ancaman *phishing* menunjukkan perlunya solusi yang efektif dan adaptif untuk mendeteksi serta mengidentifikasi situs berbahaya secara akurat. Oleh karena itu, penting untuk mengembangkan algoritma komputasi yang mampu mendeteksi dan mengidentifikasi situs web *phishing* dengan tingkat akurasi dan efisiensi yang tinggi [6][7]. Salah satu pendekatan yang banyak digunakan dalam pengembangan sistem tersebut adalah penerapan *Machine learning*, yaitu teknik yang memungkinkan sistem untuk belajar secara otomatis dari data atau contoh yang diberikan [7]. Melalui pendekatan ini, *Machine learning* dapat membantu mendeteksi adanya kejanggalan atau aktivitas mencurigakan secara lebih efektif dan adaptif terhadap pola serangan yang terus berkembang [8].

Beberapa penelitian telah membahas efektivitas *Machine learning* dalam mendeteksi situs web *phishing*. Penelitian yang membandingkan beberapa algoritma termasuk *Gradient Boosting Classifier*, *Random Forest*, *Decision Tree*, SVC, dan *K-Neighbors Classifier* menunjukkan bahwa dua algoritma pertama secara konsisten memberikan hasil akurasi tertinggi [9]. Namun, penelitian tersebut masih sebatas perbandingan tanpa mencoba menggabungkan model untuk hasil yang lebih stabil. Penelitian lain yang menggunakan algoritma *Decision Tree*, *Random Forest*, *Multilayer Perceptron*, *XGBoost*, dan *Support Vector Machines* (SVM) juga menemukan bahwa *Random Forest* dan *XGBoost* memiliki kinerja paling baik dalam mengenali situs *phishing* [10]. Penelitian ini belum mengatasi masalah *overfitting* dan belum melakukan evaluasi *meta-learning* untuk mengoptimalkan hasil prediksi.

Selanjutnya, penelitian terdahulu yang membandingkan algoritma *Random Forest*, *K-Nearest Neighbour* (KNN), dan *Artificial Neural Network* (ANN) di mana *Random Forest* menunjukkan akurasi tertinggi dalam mendeteksi situs *phishing* [11]. Penelitian ini modelnya cenderung menghasilkan keputusan yang kurang

adaptif terhadap pola serangan baru. Hasil lain diperoleh dari penelitian yang membandingkan *XGBoost*, *Random Forest*, dan *Decision Tree*, dengan hasil bahwa *XGBoost* yang telah dioptimalkan menggunakan *hyperparameter tuning* memberikan akurasi tertinggi [7]. Meski akurat, pendekatan tersebut memerlukan waktu pelatihan yang lebih lama dan penggabungan modelnya masih menggunakan metode sederhana seperti rata-rata, sehingga belum optimal dalam menangani variasi data yang kompleks.

Namun, seiring dengan evolusi teknik serangan *phishing*, tantangan baru yang menuntut sistem deteksi dengan stabilitas dan adaptabilitas yang lebih tinggi agar mampu menghadapi berbagai pola serangan yang terus berubah. Berbagai metode baru diusulkan untuk menangani keragaman jenis serangan yang semakin kompleks [12]. Hasil positif dari penelitian sebelumnya menunjukkan bahwa dibutuhkan pendekatan yang lebih kuat dan adaptif untuk mengatasi ancaman *phishing* yang terus berkembang. Oleh karena itu, penelitian ini mengusulkan penggunaan kombinasi dua algoritma *Machine learning* yang terbukti efektif [13]. *Random Forest* dan *XGBoost*, keduanya memiliki keunggulan dalam kemampuan beradaptasi terhadap data yang bervariasi, ketahanan terhadap *overfitting*, serta efisiensi dalam pemrosesan data berukuran besar. Penggunaan gabungan *Random Forest* dan *XGBoost* bertujuan untuk memberikan akurasi yang lebih tinggi, lebih stabil, dan lebih adaptif terhadap perubahan pola serangan *phishing* yang terus berkembang [14].

Dalam kombinasi *Random Forest* dan *XGBoost* terbukti efektif dalam mendeteksi situs *phishing* karena keduanya merepresentasikan dua pendekatan *ensemble* yang saling melengkapi [14]. *XGBoost* adalah algoritma *boosting* yang efisien karena mampu menjalankan proses secara paralelisasi serta menggunakan *Decision Tree* mempercepat perhitungan yang kompleks. yang membantu mencegah *overfitting*, terutama ketika digunakan pada dataset. Keunggulan lain dari *XGBoost* adalah kemampuannya dalam regularisasi, besar dengan banyak atribut yang tidak relevan atau mengandung *noise* [13][15]. Sementara itu, *Random Forest* merupakan algoritma *bagging* yang membangun beberapa *Decision Tree* untuk menghasilkan

prediksi gabungan yang lebih stabil. Keunggulan *Random Forest* ada pada kemampuannya mengurangi *varians*, terhadap *overfitting*, dan mampu melakukan generalisasi dengan baik, bahkan pada dataset besar dengan banyak fitur. Selain itu, *Random Forest* juga mudah dipahami karena menggunakan indeks *gini* atau *entropy*, serta fleksibel dalam menangani data yang tidak lengkap tanpa perlu dilakukan imputasi [16]. Kombinasi kedua algoritma ini dipilih untuk mengoptimalkan efektivitas pendekatan *bagging* dan *boosting* dalam konteks yang lebih adaptif. Kedua algoritma tersebut sama-sama berbasis *decision tree*, sehingga memiliki fondasi pemodelan yang serupa namun dengan karakteristik pembelajaran yang saling melengkapi. Keduanya memiliki dasar yang sama sebagai model berbasis *decision tree*, sehingga dapat dikombinasikan dengan baik dan saling memperkuat kemampuan satu sama lain. Penelitian ini bertujuan untuk mengembangkan pendekatan sebelumnya dengan menambahkan *rule-based filtering*. Komponen ini diharapkan mampu meningkatkan kemampuan sistem dalam mengenali pola serangan *phishing* yang terus berubah, sekaligus mempertahankan tingkat akurasi dan stabilitas model.

Pendekatan *ensemble stacking* diterapkan untuk menggabungkan dua algoritma *Machine learning* yaitu *Random Forest* dan *XGBoost*. *Stacking* merupakan salah satu teknik *ensemble learning* yang menggabungkan dua atau lebih model *Machine learning* [17] untuk meningkatkan performa prediksi. Proses *stacking* dilakukan dengan melatih beberapa model *base learners* secara paralel [17]. Metode ini meningkatkan performa prediksi dengan cara menggabungkan hasil dari beberapa *base learner* yang memiliki karakteristik berbeda, kemudian mempelajari hubungan antarhasil prediksi tersebut melalui *meta-learner* [18]. Kedua algoritma digunakan sebagai bagian dari model *ensemble stacking* yang dioptimasi menggunakan *Genetic Algorithm* untuk menyesuaikan parameter terbaik dari masing-masing model [19]. Pendekatan *ensemble* seperti *stacking* mampu mengatasi keterbatasan model tunggal dengan menggabungkan keunggulan berbagai algoritma.

Secara umum, proses *stacking* diawali dengan melatih kedua model, yaitu *Random Forest* dan *XGBoost*, sebagai *base learners* menggunakan dataset yang memuat fitur-fitur seperti URL, domain, dan konten situs. Setiap model menghasilkan probabilitas yang menggambarkan kemungkinan situs termasuk dalam kategori *phishing* atau *benign*, dengan nilai antara 0 hingga 1. Hasil prediksi dari kedua algoritma tersebut kemudian dikombinasikan dan dijadikan masukan bagi *meta learner* [20], yang bertugas mempelajari pola dari hasil prediksi tersebut untuk menghasilkan keputusan akhir yang lebih akurat dan stabil.

Dalam penelitian ini, *Logistic Regression* digunakan sebagai *meta learner* karena model ini memiliki sifat yang ringan, stabil, dan interpretatif. *Logistic Regression* mampu menggabungkan output probabilistik dari kedua *base learner* secara efisien tanpa menimbulkan *overfitting* [18], serta menghasilkan bobot yang dapat menunjukkan seberapa besar kontribusi masing-masing model terhadap prediksi akhir. Selain itu, *Logistic Regression* juga merupakan pilihan yang umum dan efektif dalam *stacking ensemble* karena kemampuannya membentuk kombinasi linear dari hasil prediksi model dasar, sehingga dapat mempelajari pola kesalahan keduanya dan menghasilkan keputusan yang lebih seimbang.

Kontribusi utama penelitian ini terletak pada penerapan *rule based filtering* dengan model gabungan *Random Forest* dan *XGBoost*. *Rule-based filtering berfungsi* sebagai lapisan awal yang menyaring hasil deteksi menggunakan aturan logis sederhana (misalnya panjang URL, simbol mencurigakan, atau umur domain), tetapi juga sebagai analisis pola serangan baru [21]. Fungsi utamanya adalah sebagai filter awal untuk mengidentifikasi pola *phishing* klasik yang jelas, sehingga hanya kasus ambigu yang diproses lebih lanjut oleh algoritma *Machine learning*.

Namun dalam konteks penelitian ini, aturan pada *Rule-based filtering* tidak bersifat statis, melainkan dikembangkan secara dinamis berdasarkan observasi dan pelatihan *Machine Learning*. Pendekatan ini memungkinkan sistem untuk menyaring dan menambahkan pola serangan phishing yang sebelumnya belum terdeteksi dengan meninjau kembali data yang tidak sesuai dengan aturan lama

(*outlier patterns*) dan menambahkannya sebagai kandidat aturan baru. Proses ini memungkinkan sistem untuk mengenali serangan *phishing* yang lebih canggih dengan cara menggunakan hasil analisis dari machine learning, yang kemudian diperkenalkan sebagai aturan baru dalam *rule-based filtering*. Sehingga, sistem tidak hanya mendeteksi *phishing* klasik tetapi juga mampu beradaptasi dengan evolusi teknik *phishing* modern saat ini. Dengan demikian, *rule-based filtering* tidak hanya meningkatkan efisiensi komputasi, tetapi juga membantu mengurangi kesalahan deteksi, yaitu situs aman ditandai *phishing* (*False Positive*) maupun situs *phishing* ditandai sebagai aman (*False Negative*) [22]. Selain itu, *rule-based filtering* berperan penting dalam menyiapkan data yang lebih bersih dan terfokus bagi model *Machine learning* yang digunakan.

Dengan menggabungkan *rule-based filtering* yang dinamis sebagai lapisan penyaring awal, *XGBoost* yang berfungsi mengurangi bias, dan *Random Forest* yang mengurangi varians, sistem deteksi *phishing* yang dihasilkan mampu memanfaatkan kelebihan masing-masing pendekatan [22]. Kombinasi ini memungkinkan sistem lebih cerdas untuk bekerja secara efisien, responsif, menjaga kestabilan kinerja, serta lebih adaptif terhadap perubahan pola serangan *phishing* yang terus berkembang. Data yang tidak terklasifikasi dengan baik oleh aturan awal akan dikaji kembali melalui hasil inferensi model *Random Forest* dan *XGBoost* untuk menemukan pola baru yang berpotensi menjadi fitur atau aturan tambahan. Pendekatan dua arah ini memastikan bahwa tidak ada data *phishing* yang hilang dari proses pelatihan, karena setiap data yang tidak lolos filter akan tetap dianalisis ulang melalui tahap pembelajaran mesin

Dengan demikian, model *ensemble Random Forest–XGBoost* yang diperkaya dengan *rule-based filtering* mampu memberikan solusi yang lebih komprehensif dan andal dibandingkan penggunaan satu model saja [12]. Penelitian ini diharapkan dapat berkontribusi dalam pengembangan sistem deteksi *phishing* yang cerdas, adaptif, dan berkelanjutan, sehingga mampu memberikan perlindungan yang lebih efektif terhadap ancaman keamanan siber di masa mendatang.

1.2 Pertanyaan Penelitian

Pertanyaan penelitian yang akan dilakukan dalam pengerjaan tugas akhir ini adalah sebagai berikut.

1. Bagaimana kinerja algoritma *Random Forest* dan *XGBoost* sebagai model tunggal serta sebagai model gabungan (*ensemble*) dalam mendeteksi website *phishing* berdasarkan evaluasi performa model?
2. Bagaimana pengaruh penerapan *rule-based filtering* sebagai lapisan awal terhadap efisiensi pemrosesan dan konsistensi kinerja prediksi sistem deteksi *phishing* berbasis model gabungan *Random Forest* dan *XGBoost*?

1.3 Tujuan Penelitian

Tujuan penelitian dalam tugas akhir ini adalah sebagai berikut.

1. Mengevaluasi dan membandingkan kinerja algoritma *Random Forest* dan *XGBoost* sebagai model tunggal serta sebagai model gabungan (*ensemble*) dalam mendeteksi website *phishing* baru yang lebih kompleks berdasarkan metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *AUC*.
2. Menganalisis dan mengevaluasi pengaruh penerapan *rule-based filtering* sebagai lapisan awal terhadap efisiensi pemrosesan dan konsistensi kinerja prediksi sistem deteksi *phishing* berbasis *stacking* antara model *Random Forest* dan *XGBoost*.

1.4 Ruang Lingkup

1. Dataset yang digunakan merupakan kumpulan data *phishing* dan *benign website* yang diperoleh dari repositori publik *Kaggle*. Dataset ini berisi URL beserta fitur-fitur relevan yang dapat digunakan dalam proses klasifikasi.
2. Implementasi sistem ini mencakup pembuatan *dashboard* interaktif yang memungkinkan pengguna untuk melakukan input URL dan mendapatkan hasil klasifikasi *phishing* atau *benign*. *Dashboard* ini tidak hanya menampilkan hasil klasifikasi dalam bentuk probabilitas untuk masing-masing kategori (*phishing* atau *benign*), tetapi juga memberikan informasi

mengenai fitur-fitur utama yang paling berpengaruh dalam klasifikasi tersebut.

1.5 Sistematika penyajian

Dokumen tugas akhir ini disusun dengan sistematika sebagai berikut untuk memberikan gambaran yang jelas dan terstruktur mengenai penelitian yang dilakukan:

1. Bab 1 Pendahuluan

Bab ini berisi uraian mengenai dasar dilakukannya penelitian. Bagian ini meliputi:

1. Latar Belakang, yang menjelaskan fenomena meningkatnya serangan *phishing*, kelemahan deteksi menggunakan satu algoritma, serta urgensi penggunaan kombinasi metode *Machine learning*.
2. Rumusan Masalah, yang memuat pertanyaan penelitian yang ingin dijawab.
3. Tujuan Penelitian, yang menjelaskan target yang ingin dicapai melalui penelitian ini.
4. Ruang Lingkup Penelitian, yang menjelaskan batas cakupan penelitian, meliputi penggunaan dataset publik, penerapan algoritma *Random Forest* dan *XGBoost*, serta integrasi dengan *rule-based filtering*
5. Batasan Penelitian, yang memperjelas keterbatasan penelitian, baik dari sisi algoritma, data, maupun implementasi sistem.
6. Sistematika Penyajian, yaitu gambaran umum isi dokumen tugas akhir.

2. Bab 2 Landasan Teori

Bab ini membahas teori-teori dan konsep yang menjadi dasar penelitian, serta penelitian terdahulu yang relevan. Bagian ini meliputi:

1. *Phishing*, yang menjelaskan definisi, karakteristik, teknik serangan, dan dampak yang ditimbulkan.

2. *Machine learning* untuk Deteksi *Phishing*, yang membahas konsep *supervised learning*, klasifikasi biner, dan relevansinya dalam mendeteksi *phishing*.
 3. Algoritma *Random Forest*, yang membahas prinsip kerja, konsep *bagging*, keunggulan, dan keterbatasannya.
 4. Algoritma *XGBoost*, yang membahas prinsip kerja, konsep *boosting*, keunggulan, dan keterbatasannya.
 5. *Rule-Based Filtering*, yang membahas definisi, contoh aturan sederhana (misalnya panjang URL, simbol mencurigakan, umur domain), serta peranannya dalam menyaring hasil deteksi agar lebih efisien.
 6. Evaluasi, yang akan membahas teori dan metrik yang digunakan untuk menilai kinerja model deteksi *phishing*. Evaluasi dilakukan untuk mengukur seberapa baik model mampu membedakan antara situs *phishing* dan situs yang *benign*.
 7. Penelitian Terdahulu, yang menyajikan ringkasan penelitian-penelitian relevan dalam bentuk tabel, mencakup judul, tahun, hasil, kesimpulan, serta kekurangan penelitian terdahulu sebagai dasar penelitian ini.
3. Bab 3 Metode Penelitian

Bab ini menjelaskan pendekatan dan tahapan penelitian yang dilakukan. Bagian ini meliputi:

1. Tahapan Penelitian, yang dipaparkan dalam bentuk *flowchart* serta penjelasan setiap tahap, mulai dari studi literatur, pengumpulan data, pra proses data, penerapan *rule-based filtering*, pembangunan model dengan *Random Forest* dan *XGBoost*, evaluasi model, hingga implementasi sistem pengecekan sederhana.
2. Teknik Pengumpulan Data, yang menjelaskan sumber dataset (misalnya *Kaggle*, *UCI*, *PhishTank*, atau *OpenPhish*) serta cara validasi dan persiapan data.
3. Proses Pengolahan Data, yang mencakup pembersihan data, ekstraksi fitur URL dan domai,.

4. Metode Analisis, yang menjelaskan penggunaan algoritma *Random Forest*, *XGBoost*, kombinasi keduanya, serta integrasi *rule-based filtering*, disertai metrik evaluasi seperti *accuracy*, *precision*, *recall*, *F1-score*, dan AUC.

BAB II

LANDASAN TEORI

Bab ini berisi pemaparan konsep dasar dan teori yang mendukung penelitian, serta uraian mengenai hasil penelitian terdahulu yang relevan. Landasan teori ini menjadi dasar konseptual bagi pengembangan sistem deteksi *phishing website* menggunakan kombinasi algoritma *Random Forest*, *XGBoost*, dan *rule-based filtering*.

2.1 *Phishing*

Phishing adalah salah satu bentuk kejahatan siber yang dilakukan dengan cara memanipulasi korban agar memberikan informasi pribadi, seperti *username*, *password*, data kartu kredit, atau kredensial akun lainnya [23]. Serangan *phishing* biasanya dilakukan melalui email, pesan instan, media sosial, maupun *website* tiruan yang menyerupai situs resmi [1].

Beberapa jenis *phishing* yang paling umum antara lain email *phishing*, *website phishing*, *spear phishing*, *whaling* dan *smishing* [4]. Email *phishing* melibatkan pengiriman email palsu yang mengelabui korban untuk memberikan informasi pribadi atau mengunduh *malware* [23]. *Website phishing* melibatkan pembuatan situs web palsu yang mirip dengan situs web asli untuk menipu pengguna agar memasukkan data sensitif mereka seperti kata sandi, nomor kartu kredit, atau informasi pribadi lainnya [23]. *Spear phishing* adalah serangan yang ditargetkan pada individu atau organisasi tertentu menggunakan informasi yang dipersonalisasi untuk meningkatkan efektivitas serangan [4]. *Whaling* adalah jenis *spear phishing* yang menargetkan individu berprofil tinggi seperti eksekutif perusahaan atau pejabat pemerintah. *Smishing*, atau SMS *phishing*, adalah jenis *phishing* yang menggunakan pesan teks untuk menipu korban [4].

Website phishing sering kali memiliki karakteristik yang mencurigakan, seperti penggunaan domain yang sangat mirip dengan domain asli, dengan mengganti atau menambahkan satu karakter dalam alamat URL (misalnya, mengganti "g" dengan

"q") [23][1]. Selain itu, tampilan *website phishing* biasanya disalin dengan sangat detail, termasuk logo, warna, dan tata letak, untuk menipu korban agar percaya bahwa mereka berada di situs resmi [4][23]. Meskipun menggunakan *HTTPS*, *website phishing* sering memanfaatkan sertifikat SSL yang tidak sah atau palsu, yang dapat mengecoh pengguna dengan memberi kesan bahwa situs tersebut aman padahal sebenarnya tidak [4][1].

Kerugian yang ditimbulkan akibat *phishing* dapat berupa kerugian finansial, pencurian identitas, hingga kerusakan reputasi perusahaan maupun individu. Korban *phishing* bisa kehilangan uang secara langsung, terutama jika mereka memberikan informasi kartu kredit atau melakukan transfer uang ke penyerang [23]. Selain itu, data yang dicuri, seperti NIK atau informasi bank, dapat digunakan untuk mencuri identitas korban dan melakukan penipuan lebih lanjut [4]. Serangan *phishing* terhadap perusahaan atau lembaga juga dapat merusak reputasi mereka, mengurangi kepercayaan pelanggan, dan menciptakan keraguan tentang keamanan data [1]. Menurut penelitian [1], serangan *phishing* semakin marak terjadi di platform media sosial karena tingginya interaksi pengguna, sehingga memperbesar peluang penyerang untuk menyebarkan tautan berbahaya [1]. Hal ini menunjukkan betapa pentingnya kewaspadaan pengguna dalam berinteraksi di media sosial untuk mencegah serangan *phishing*. Pada Tabel 2. 1 menunjukkan tren dan jenis serangan *phishing* terbaru.

Tabel 2. 1 Tren dan Jenis Serangan *Phishing*

No	Jenis	Target	Penjelasan
1.	<i>Spear Phishing</i>	Pejabat, perusahaan tertentu, atau individu.	Serangan sangat personal atau kelompok tertentu, menyesuaikan komunikasi dengan target, sering menggunakan data media sosial untuk membobol basis data rahasia.
2.	<i>Whaling</i>	Direktur perusahaan atau eksekutif tingkat tinggi.	Menyamar sebagai staf atau mengirim pengumuman internal palsu via email/medsos untuk mengganggu perusahaan.

No	Jenis	Target	Penjelasan
3.	<i>Vishing</i>	Panggilan suara (<i>Voice Call</i>).	Memanfaatkan rasa mendesak atau takut (misalnya, berita kecelakaan/masalah hukum) agar korban mentransfer uang. Menggunakan nomor tidak dikenal.
4.	<i>Smishing</i>	Pesan teks (SMS)	Menggunakan manipulasi, seringkali dengan modus memenangkan undian 5atau hadiah besar, menyamar sebagai pihak resmi.
5.	<i>Social Media-Based Phishing</i>	<i>Platform</i> media sosial (Facebook, Instagram, dll.).	Menyebarkan tautan <i>phishing</i> dari akun terpercaya atau membuat profil palsu untuk membangun hubungan dan kepercayaan.
6.	<i>Deepfake AI</i>	Teknologi AI untuk membuat video atau audio palsu yang sangat meyakinkan.	Membuat rekaman palsu mirip atasan/rekan kerja, meminta korban mentransfer uang atau memberikan informasi pribadi.
7.	<i>Scam Phishing</i>	Mencuri informasi pribadi sensitif (nomor rekening, kata sandi, dll.).	Mengirim tautan/berkas berisi malware untuk mencuri informasi, media umum: SMS, email, telepon, atau medsos.
8.	<i>Cloning and Blind Phishing</i>	<i>Cloning</i> meniru situs web asli. <i>Blind Phishing</i> adalah serangan yang tidak menargetkan individu spesifik.	<i>Cloning</i> : Membuat situs web palsu untuk mencuri data input korban. <i>Blind</i> : Menyebarkan pesan <i>phishing</i> secara massal tanpa target spesifik (misalnya via email/SMS).

Sumber: [4]

2.2 Deteksi *Phishing* pada *Website*

Deteksi *phishing* di *website* bertujuan untuk mengidentifikasi dan memblokir situs *website* yang berbahaya sebelum dapat merugikan penggunanya. Dalam melakukan deteksi *phishing*, ada beberapa pendekatan yang digunakan, antara lain *blacklist-*

based, *heuristic-based*, dan *Machine learning-based*. *Blacklist-based* sangat efisien dalam mendeteksi situs web *phishing* yang sudah dikenal, namun kurang efektif dalam menangani situs baru [24]. *Heuristic-based* dapat lebih fleksibel dan mendeteksi situs web dengan karakteristik mencurigakan, tetapi sering kali terhambat oleh keterbatasan dalam aturan yang ditetapkan [23][25].

Machine learning-based lebih kuat dalam mendeteksi situs *phishing* yang tidak dikenal, tetapi membutuhkan data pelatihan yang banyak dan sumber daya komputasi yang besar [26][23]. Dalam penelitian ini, penggabungan *Machine learning* dengan *rule-based filtering* dipilih karena kombinasi kedua pendekatan ini dinilai memberikan hasil yang optimal [25]. *Machine learning* memungkinkan deteksi situs *phishing* yang belum diketahui sebelumnya, sedangkan *rule-based filtering* dapat memberikan lapisan awal untuk menyaring situs web dengan ciri-ciri umum yang dapat dikenali lebih cepat [24][25]. Melalui integrasi kedua pendekatan tersebut, keunggulan masing-masing dapat dimanfaatkan untuk meningkatkan akurasi dan efisiensi sistem deteksi *website phishing* [26][25].

2.3 Feature Extraction

Feature extraction adalah proses penting dalam sistem deteksi *phishing*, di mana data mentah dari *website* diubah menjadi fitur numerik atau representasi lain yang dapat diproses oleh algoritma *Machine learning* [27]. Dalam konteks deteksi *phishing website* menggunakan *Random Forest*, *XGBoost*, dan *rule-based filtering*, *feature extraction* memegang peranan kunci karena memungkinkan sistem untuk mengubah data yang bersifat raw (seperti URL, konten HTML, informasi DNS, dan atribut domain) menjadi format yang lebih terstruktur dan dapat dianalisis oleh algoritma *Machine learning* [28][29]. Dengan mengaplikasikan teknik *feature extraction* yang efektif, model *Machine learning* dapat mempelajari pola-pola yang membedakan antara *website* yang sah dan yang berpotensi berbahaya, yang akan meningkatkan akurasi sistem dalam mendeteksi *website phishing* [27]. Proses ekstraksi fitur dari URL ini mengubah elemen-elemen URL yang berbeda menjadi fitur numerik yang dapat dianalisis lebih lanjut oleh algoritma deteksi, memungkinkan sistem untuk mendeteksi URL yang mencurigakan dengan lebih

efisien [30]. URL merupakan elemen utama yang sering dianalisis dalam deteksi *phishing* karena banyak serangan *phishing* memanfaatkan URL yang tampak mirip dengan situs web sah namun telah dimodifikasi untuk menipu pengguna [27]. Beberapa fitur yang dapat diekstraksi dari URL antara lain panjang URL, di mana URL *phishing* cenderung lebih panjang dan kompleks untuk menyamarkan tujuan aslinya jumlah subdomain yang berlebihan seperti pada *login.bank.fakewebsite.com* yang sering menjadi indikator aktivitas *phishing* [28], keberadaan simbol mencurigakan seperti tanda “@”, “/” ganda, atau banyak karakter pemisah “-” yang digunakan untuk menutupi tujuan sebenarnya [29].

Tabel 2. 2 menampilkan contoh URL sebelum dan sesudah melalui proses ekstraksi fitur, yang diadaptasi dari penelitian Aung & Yamana (2024) dalam jurnal berjudul “*PhiSN: Phishing URL Detection Using Segmentation and NLP Features*” Proses ekstraksi ini bertujuan untuk memisahkan komponen penting dalam URL yang dapat digunakan untuk mendeteksi indikasi *phishing*.

Tabel 2. 2 Contoh Proses Ekstraksi Fitur dari URL *Phishing*

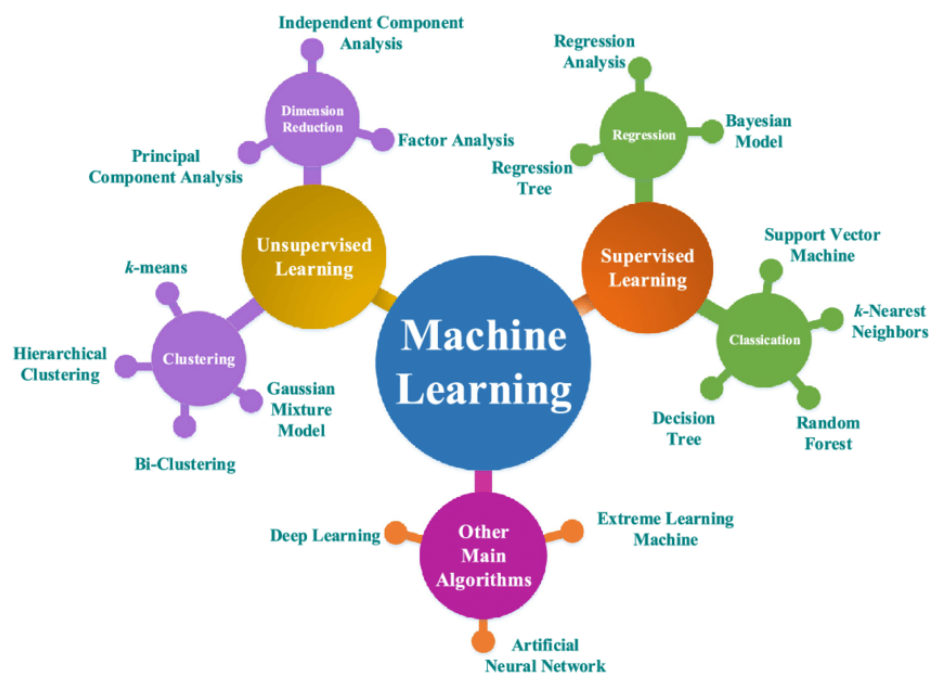
No	URL Asli	Ekstraksi Fitur	Keterangan
1.	http://www.amaz0necure.me/login	["amazon", "secure", "login"]	Terdapat penggantian karakter “0” → “o”; panjang URL > 75 karakter; mengandung kata “login”.
2.	https://paypal-update-info.com/verify	["paypal", "update", "info", "verify"]	Mengandung kata “update” dan “verify”; domain tidak resmi dari brand.
3.	http://31.192.106.250/bank/login	["31.192.106.250", "bank", "login"]	Domain menggunakan IP address, bukan <i>hostname</i> .
4.	https://secure-google-account.co/reset	["secure", "google", "account", "reset"]	Mengandung kata “secure” dan “reset”; domain menyerupai domain sah “google.com”.
5.	http://update-facebook-security.info/login	["update", "facebook", "security", "login"]	Panjang URL berlebih; mengandung kata “update” dan “login”.

Sumber: [11]

2.4 Machine learning

Machine learning merupakan cabang dari *artificial intelligence* yang memungkinkan komputer untuk belajar dari data guna melakukan prediksi atau klasifikasi [8]. Dalam konteks deteksi *phishing*, *Machine learning* digunakan untuk membedakan antara *website phishing* dan *benign* berdasarkan sejumlah fitur, seperti panjang URL, jumlah subdomain, penggunaan simbol mencurigakan, umur domain, dan validitas sertifikat SSL [31][32].

Penelitian [33] menunjukkan bahwa algoritma *Machine learning* seperti *Logistic Regression*, *K-Nearest Neighbor* (KNN), *Decision Tree*, *Random Forest*, dan *XGBoost* dapat digunakan untuk klasifikasi *phishing* dengan tingkat akurasi yang cukup tinggi [33]. Dalam hal ini, teknik *supervised learning* digunakan, yang di mana model dilatih menggunakan dataset berlabel, yang biasanya terdiri dari label *phishing* (*malicious*) dan *benign* (*non-phishing*) [32]. Dengan memanfaatkan data berlabel, model dapat mempelajari pola yang membedakan antara *website* berbahaya dan yang aman [33]. Pada Gambar 2. 1 adalah gambaran jenis *meachine learning*



Gambar 2. 1 Jenis *Machine Learning*

Sumber: [11]

2.5 Random Forest

2.5.1 Description

Random Forest adalah algoritma *ensemble learning* yang menggunakan pendekatan *bagging (Bootstrap Aggregating)* untuk membangun banyak *Decision Trees* secara acak dan menghasilkan prediksi berdasarkan *voting* mayoritas dari setiap *decision tree* [34]. Algoritma ini membangun banyak *decision tree*, di mana setiap *decision tree* berdasarkan bagian-bagian acak dari data pelatihan [35]. Pendekatan ini bertujuan untuk meningkatkan akurasi dan stabilitas model, dengan cara mengurangi kesalahan dan meningkatkan ketahanan terhadap *overfitting* (terlalu menyesuaikan model dengan data latihan), sehingga model dapat bekerja dengan lebih baik meskipun menggunakan data yang besar dan beragam fitur [36].

Keunggulan *Random Forest* adalah sebagai berikut:

- Mengurangi kesalahan dan meningkatkan stabilitas: Karena menghasilkan banyak *decision tree*, setiap *tree* membuat prediksi secara terpisah, dan hasil akhirnya ditentukan berdasarkan suara terbanyak dari setiap *tree*, yang mengurangi kesalahan model secara keseluruhan [37].
- Tahan terhadap *overfitting*: *Random Forest* dapat menangani *noise* (gangguan) dan kerumitan data tanpa kehilangan kemampuan untuk membuat prediksi yang akurat, menjadikannya efektif untuk data yang besar dan bervariasi [38].
- Performa baik meskipun dataset berukuran besar dengan banyak fitur: *Random Forest* tetap dapat mengelola berbagai fitur tanpa mengurangi performa, bahkan ketika dataset yang digunakan sangat besar dan memiliki banyak variabel [39].
- Relatif mudah dipahami: Algoritma ini menggunakan indeks *gini* atau *entropy* untuk memilih fitur terbaik dalam pemisahan data, sehingga lebih mudah dipahami dibandingkan dengan banyak algoritma lainnya [34].
- Dapat menangani data yang hilang tanpa perlu perbaikan tambahan: *Random Forest* dapat bekerja dengan data yang memiliki nilai kosong tanpa perlu melakukan langkah khusus untuk mengisi data yang hilang [37].

Penelitian dalam [37] dan [39] menunjukkan bahwa *Random Forest* sangat andal dalam mengklasifikasikan situs web *phishing* karena kemampuan algoritma ini untuk mengatasi gangguan dan ketahanan terhadap *noise* pada data [36][38].

2.5.2 Hyperparameters of Random Forests

Pada bagian ini merupakan beberapa *Hyperparameter* yang dalam algoritma *random forest* yaitu :

2.5.2.1 RF Hyperparameter num.trees

Number of trees adalah *hyperparameter* yang menentukan jumlah *tree* yang digabungkan dalam model *ansemble* secara keseluruhan. Pada umumnya semakin banyak *tree*, maka semakin baik kualitas performa pemodelan serta waktu yang dibutuhkan untuk melatih dan membuat prediksi dengan model semakin banyak.

Type:	Integer, Skalar
Default:	Log (500,2)
Sensitivity:	Menurut Breiman (2001), kesalahan generalisasi dari model akan konvergen dengan [24] meningkatnya jumlah <i>tree</i> ke suatu nilai batas yang lebih rendah. Hal ini berarti bahwa model akan menjadi kurang sensitif terhadap perubahan num.trees dengan nilai num.trees yang semakin tinggi. Hal ini juga terlihat dalam benchmark Louppe (2015). Hanya dengan nilai kecil (num.trees <50), model sangat sensitif terhadap perubahan parameter ini. Hasil empiris dari Probst et al. (2019) juga menunjukkan bahwa tingkat kemampuan menyesuaikan (<i>tunability</i>) num.trees diperkirakan rendah.
Heuristik:	Ada hasil teoritis tentang konvergensi model terkait num.trees (Breiman 2001; Scornet 2017). Namun, ini tidak menghasilkan pendekatan heuristik yang jelas untuk menentukan parameter ini. Rekomendasi umum adalah untuk memilih num.trees yang cukup tinggi (Probst et al. 2019) (karena lebih banyak <i>tree</i> biasanya lebih baik), sambil

memastikan bahwa waktu eksekusi model tidak menjadi terlalu lama.

Rentang:	$num. trees \in [1, \infty)$
Transformasi:	<i>trans_2pow_round</i>
Batas:	<i>Lower = 0; upper = 11.</i>
Keterbatasan:	Tidak ada.
Interaksi:	Tidak diketahui

2.5.2.2 RF *Hyperparameter mtry*

Hyperparameter mtry menentukan berapa banyak fitur yang dipilih secara acak untuk dipertimbangkan pada setiap pemisahan. Oleh karena itu, *mtry* mengontrol aspek penting, yaitu randomisasi *tree* individu. Nilai *mtry* $\ll n$ mengimplikasikan bahwa perbedaan antara *tree* akan lebih besar (lebih banyak randomisasi). Ini meningkatkan potensi kesalahan *tree* individu, tetapi keseluruhan *ensemble* menjadi lebih baik (Breiman 2001; Louppe 2015). Sementara itu, *mtry* $\ll n$ juga dapat mengurangi waktu *runtime* secara signifikan (Louppe 2015). Namun demikian, temuan tentang parameter ini sangat tergantung pada heuristik dan hasil empiris. Menurut Scornet (2017), tidak ada hasil teoritis tentang randomisasi fitur pemisahan yang tersedia.

Type:	Integer , Skalar
Default:	Floor (sqrt(nFeatures))
Sensivitas:	Menurut Breiman (2001), <i>Random Forest</i> relatif tidak sensitif terhadap perubahan <i>mtry</i> : "Namun prosedur ini tidak terlalu sensitif terhadap nilai F. Perbedaan absolut rata-rata antara tingkat kesalahan menggunakan F = 1 dan nilai yang lebih tinggi dari F kurang dari 1." (Breiman 2001) (di sini: F sesuai dengan <i>mtry</i>). Hal ini nampaknya bertentangan dengan hasil benchmark oleh Louppe (2015), yang menentukan bahwa <i>mtry</i> mungkin memang memiliki dampak yang cukup besar, terutama untuk nilai <i>mtry</i> yang rendah. Penelitian tentang "tunability" oleh Probst et al. (2019) juga mengidentifikasi

mtry sebagai parameter penting (yaitu, dapat diatur). Ini tidak selalu menjadi kontradiksi dengan pengamatan Breiman, karena Probst et al. (2019) menentukan *Random Forest* sebagai model yang paling tidak dapat diatur dalam investigasi eksperimental mereka. Jadi, meskipun mtry mungkin memiliki beberapa dampak (diban dingkan dengan parameter lain), mungkin kurang sensitif dibandingkan dengan *hyperparameter* dari model lain.

Heuristik: Breiman (2001) mengusulkan heuristik berikut: $mtry = \text{floor}(\log_2(n) + 1)$. Jika fitur kategorikal ada, Breiman menyarankan untuk menggandakan atau mengtripel nilai tersebut. Tidak ada motivasi teoretis yang diberikan. Suggestion yang sering digunakan adalah $mtry = \sqrt{n}$ (atau $mtry = \text{floor}(\sqrt{n})$). Meskipun ini digunakan dalam berbagai implementasi *Random Forest*, tidak ada motivasi teoretis yang jelas. Untuk $n < 20$ kedua heuristik memberikan nilai yang sangat mirip. Beberapa implementasi membedakan antara klasifikasi ($mtry = \sqrt{n}$) dan regresi ($mtry = n/3$). Hasil empiris dengan heuristik ini dijelaskan oleh Probst et al. (2019c).

Rentang: $mtry \in [1, n]$.
 Transformasi: `trans_id`
 Batas: `lower = 1; upper = nFeatures`
 Keterbatasan: Tidak ada
 Interaksi: Tidak diketahui

2.5.2.3 RF *Hyperparameter* `Sample.fraction`

Parameter `sample.fraction` menentukan jumlah pengamatan yang secara acak digunakan untuk melatih satu *tree* dalam model. Menurut Probst et al. (2019c), `sample.fraction` memiliki pengaruh yang sama seperti `mtry`, yaitu mempengaruhi sifat *tree* dalam model. Dengan menggunakan nilai `sample.fraction` yang kecil (sama dengan nilai `mtry` yang kecil), kualitas prediksi dari *tree* individu menjadi

lebih rendah, tetapi keberagaman *tree* meningkat. Hal ini berdampak positif pada kualitas model *ensemble*. Selain itu, nilai *sample.fraction* yang lebih kecil juga dapat mengurangi waktu *runtime* dari model.

Type:	Double, Skalar
Default:	1
Sensivitas:	<i>sample.fraction</i> dapat berdampak signifikan pada kualitas model. Scornet melaporkan: Namun, berdasarkan hasil empiris, tidak ada justifikasi untuk nilai default dalam <i>random forest</i> untuk sub sampling atau kedalaman <i>tree</i> , karena mengoptimalkan salah satunya menghasilkan kinerja yang lebih baik.
Heuristik:	Tidak diketahui
Rentang:	$sample.fraction \in (0, 1]$.
Transformasi:	<i>trans_id</i>
Batas:	$Lower = 0.1; upper = 1$
Keterbatasan:	Tidak ada.
Interaksi:	Potensialnya, <i>sample.fraction</i> berinteraksi dengan parameter yang mempengaruhi pelatihan <i>tree</i> individu DT (misalnya <i>maxdepth</i> , <i>minsplit</i> , <i>cp</i>). Scornet: "Menurut analisis teoritis dari median forests, bahwa tidak perlu mengoptimalkan ukuran sub-sampling dan kedalaman <i>tree</i> : mengoptimalkan hanya salah satu dari dua parameter ini menghasilkan kinerja yang sama dengan mengoptimalkan keduanya" (Scornet 2017). Namun, pengamatan teoritis ini hanya berlaku untuk median <i>trees</i> masing-masing dan tidak selalu untuk model <i>Random Forest</i> klasik yang menjadi pertimbangan.

2.5.2.4 RF Hyperparameter Replace (bootstrap)

Parameter *replace* menentukan apakah sampel yang diambil secara acak diganti, yaitu apakah sampel individu dapat diambil beberapa kali untuk pelatihan sebuah *tree* (*replace* = TRUE) atau tidak (*replace* = FALSE). Jika *replace* = TRUE,

kemungkinan dua *tree* menerima sampel data yang sama akan berkurang. Hal ini dapat lebih lanjut mengurangi korelasi antar *tree* dan meningkatkan kualitas.

Type:	Logikal
Default:	2 (TRUE)
Sensivitas:	Sensitivitas dari <i>replace</i> seringkali cukup kecil. Namun, survei Probst et al. (2019) mencatat adanya bias yang berpotensi merugikan untuk <i>replace</i> = TRUE, jika variabel kategorikal dengan jumlah level variabel hadir.
Heuristik:	Karena bias yang disebutkan di atas, pilihan bisa bergantung pada varian kardinalitas dalam fitur data. Namun, rekomendasi yang dapat diukur tidak tersedia.
Rentang:	$\text{replace} \in \{\text{TRUE}, \text{FALSE}\}$.
Transformasi:	Trans_id
Batas:	lower = 1 (FALSE); upper = 2 (TRUE).
Keterbatasan:	Tidak ada.
Interaksi:	Satu interaksi yang jelas terjadi dengan <i>sample.fraction</i> . Kedua parameter mengontrol pilihan acak data pelatihan untuk setiap <i>tree</i> . Pengaturan (<i>replace</i> = TRUE $\wedge$ <i>sample.fraction</i> = 1) serta pengaturan (<i>replace</i> = FALSE $\wedge$ <i>sample.fraction</i> < 1) menyiratkan bahwa <i>tree</i> individu tidak akan melihat seluruh set data.

2.5.2.5 RF *Hyperparameter* **Respect.unordered.factors**

Parameter ini menentukan bagaimana pembagian variabel kategori ditangani. Ada tiga opsi: *ignore*, *order*, atau *partition*, yang akan dijelaskan singkat di bawah ini. Diskusi terperinci dapat ditemukan pada Wright dan König (2019). Standar yang juga digunakan oleh Breiman (2001) adalah *respect.unordered.factors* = *partition*. Dalam kasus ini, semua pembagian potensial dari variabel nominal, kategorikal dipertimbangkan. Ini menghasilkan model yang baik, tetapi jumlah pembagian yang dipertimbangkan dapat menyebabkan waktu yang tidak efektif. Alternatif yang lebih sederhana adalah *respect.unordered.factors* = *ignore*. Di sini, sifat kategorikal

variabel akan diabaikan. Sebaliknya, diasumsikan bahwa variabel tersebut berurutan, dan pembagian dipilih seperti halnya dengan variabel numerik. Ini mengurangi waktu komputasi tetapi dapat menurunkan kualitas model. Pilihan yang lebih baik seharusnya adalah `respect.unordered.factors = order`. Di sini, setiap variabel kategori diurutkan terlebih dahulu, tergantung pada frekuensi setiap level dalam kelas pertama dari dua kelas (dalam kasus klasifikasi) atau tergantung pada rata-rata nilai variabel dependen (regresi). Setelah pengurutan ini, variabel dianggap sebagai numerik. Ini memungkinkan waktu komputasi yang sama dengan `respect.unordered.factors = ignore` tetapi dengan kualitas model yang lebih baik. Ini mungkin tidak memungkinkan untuk klasifikasi dengan lebih dari dua kelas, karena kurangnya kriteria pengurutan yang jelas (Wright dan König 2019; Wright 2020).

Dalam kasus tertentu, `respect.unordered.factors = ignore` mungkin berhasil dalam praktiknya. Ini dapat terjadi ketika variabel sebenarnya bersifat nominal (tidak diketahui oleh analis).

Type:	Karakter, Skalar
Default:	1 (<i>ignore</i>)
Sensivitas:	Tidak diketahui
Heuristik:	Tidak ada
Rentang:	$respect.unordered.factors \in \{ignore, order, partition\}$. Parameter <code>respect.unordered.factors</code> juga dapat dipahami sebagai nilai biner. Maka, <i>TRUE</i> sesuai dengan <i>order</i> dan <i>FALSE</i> dengan <i>ignore</i> (Wright 2020).
Transformasi:	<code>trans_id</code>
Batas:	<i>lower = 1 (ignore); upper = 2 (order)</i> .
Keterbatasan:	Tidak ada
Interaksi:	Tidak diketahui

2.6 XGBoost

2.6.1 Description

Extreme Gradient Boosting (XGBoost) adalah algoritma berbasis *boosting* yang dirancang untuk efisiensi dan performa tinggi, yang sangat cocok digunakan dalam tugas klasifikasi dan regresi [40]. *XGBoost* bekerja dengan membangun model secara bertahap, di mana setiap *decision tree* baru bertujuan untuk memperbaiki kesalahan yang dibuat oleh *decision tree* sebelumnya, yang memungkinkan model untuk meningkatkan kualitas prediksi secara bertahap [41].

Keunggulan *XGBoost*:

- Efisiensi Komputasi Tinggi dengan Dukungan Paralelisasi

XGBoost dirancang untuk paralelisasi, yang memungkinkan pemrosesan data dalam jumlah besar dengan cepat. Dukungan paralelisasi ini menjadikannya sangat efisien dalam menangani data dalam skala besar dan sangat sesuai untuk aplikasi *real-time* yang membutuhkan kecepatan pengolahan tinggi [41].

- Dapat Menangani Dataset Besar dengan Cepat

XGBoost sangat efisien dalam menangani dataset besar yang memiliki banyak fitur. Algoritma ini mengoptimalkan proses pembelajaran model dengan memanfaatkan teknik *gradient boosting* yang dapat memproses dataset dalam waktu singkat tanpa mengorbankan akurasi [22].

- Mekanisme Regularisasi untuk Mengurangi Risiko *Overfitting*

XGBoost dilengkapi dengan mekanisme regularisasi untuk mengurangi risiko *overfitting*, yang sering kali menjadi masalah pada model yang kompleks. Regularisasi ini membantu algoritma untuk menjaga keseimbangan antara kesalahan pelatihan dan kesalahan pengujian, sehingga menghasilkan model yang lebih stabil dan dapat dipercaya [40].

- Efektif untuk Data dengan Banyak Atribut, Termasuk Atribut yang Tidak Relevan atau Berisik

XGBoost juga efektif dalam menangani dataset dengan banyak atribut, bahkan jika atribut tersebut tidak relevan atau mengandung *noise*. Hal ini memungkinkan *XGBoost* untuk tetap memberikan performa yang tinggi

meskipun data yang digunakan memiliki kualitas yang bervariasi dan banyaknya atribut yang tidak penting [22].

- Peningkatan Performa dengan Teknik Seleksi Fitur

Penelitian dalam [22] menunjukkan bahwa *XGBoost* memberikan performa yang sangat tinggi dalam mendeteksi *phishing*, terutama ketika dipadukan dengan teknik seleksi fitur yang memungkinkan untuk memilih atribut yang paling relevan dan mengurangi dimensi data, sehingga meningkatkan efisiensi dan akurasi model [22].

Secara keseluruhan, *XGBoost* adalah algoritma yang sangat efisien dan kuat, ideal untuk aplikasi yang melibatkan dataset besar dan kompleks. Dengan kemampuan paralelisasi, regularisasi, dan kemampuannya dalam menangani atribut berisik, serta kinerjanya yang semakin baik dengan teknik seleksi fitur, *XGBoost* telah menjadi algoritma unggulan dalam deteksi *phishing* dan aplikasi pembelajaran mesin lainnya [40][41][22].

2.6.2 Hyperparameter of XGBoost

Pada bagian ini merupakan beberapa *Hyperparameter* yang dalam algoritma *XGBoost* yaitu

2.6.2.1 XGBoost Hyperparameter nrounds

XGBoost Hyperparameter nrounds Parameter *nrounds* menentukan jumlah langkah boosting. Karena sebuah *tree* dibuat di setiap langkah boosting individual, *nrounds* juga mengendalikan jumlah *tree* yang digabungkan ke dalam *ensemble* secara keseluruhan. Makna praktisnya dapat dijelaskan sebagai berikut: nilai *nrounds* yang lebih besar berarti model yang lebih kompleks dan mungkin lebih akurat, tetapi juga menyebabkan waktu eksekusi yang lebih lama. Makna praktisnya sangat mirip dengan *num.trees* dalam *random forests*. Berbeda dengan *num.trees*, *overfitting* menjadi risiko pada nilai yang sangat besar, tergantung pada parameter lain seperti *eta*, *lambda*, *alpha*. Sebagai contoh, hasil empiris Friedman (2001) menunjukkan bahwa dengan *eta* yang rendah, bahkan nilai *nrounds* yang tinggi tidak menyebabkan *overfitting*.

<i>Type:</i>	<i>Integer, scalar</i>
<i>Default:</i>	<i>0</i>
<i>Sensitivity/robustness:</i>	Serupa dengan parameter <code>num.trees</code> pada random forests, <code>nrounds</code> juga memiliki sensitivitas yang lebih tinggi, terutama pada nilai yang rendah.
<i>Heuristics:</i>	<i>Heuristics cannot be derived from the literature. Often values of several hundred to several thousand trees are set as the upper limit</i> (Brownlee 2018).
<i>Range:</i>	$\in [1, \infty[$. Hanya nilai bilangan bulat yang valid.
<i>Transformation:</i>	<i>trans_2pow_round.</i>
<i>Bounds:</i>	<i>lower = 0; upper = 11</i>
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	<i>There is a connection between the hyperparameters beta, rounds, and subsample.</i>

2.6.2.2 XGBoost Hyperparameter eta

Parameter eta adalah laju pembelajaran dan juga disebut parameter “penyusutan”. Parameter ini mengontrol penurunan bobot pada setiap langkah boosting. Ini memiliki makna praktis menurunkan bobot membantu untuk mengurangi pengaruh *tree* tunggal pada ansambel. Ini juga dapat menghindari overfitting.

<i>Type:</i>	<i>double, Scalar</i>
<i>Default:</i>	<i>Log 2 {0.3}</i>
<i>Sensitivity:</i>	Hasil empiris menunjukkan bahwa XGBoost lebih sensitif terhadap eta ketika eta besar (Friedman 2001). Secara umum, nilai yang lebih kecil lebih baik. Dalam sebuah studi empiris, Probst et al. (2018) menggambarkan eta sebagai parameter dengan kemampuan penyesuaian yang relatif tinggi.
<i>Heuristics:</i>	Sebuah heuristik sulit untuk diformulasikan karena ketergantungannya pada parameter lain dan situasi data,

tetapi Hastie et al. (2017) merekomendasikan strategi terbaik tampaknya adalah menetapkan η sangat kecil ($\eta < 0,1$) dan kemudian memilih n rounds melalui penghentian dini. Hal ini dapat menyebabkan waktu berjalan yang lebih lama karena n rounds yang besar. Brownlee (2018) menyebutkan sebuah heuristik, yang menjelaskan rentang pencarian tergantung pada n rounds. $\eta \in [0,1]$. Menggunakan skala logaritmik tampaknya masuk akal, misalnya, $2^{-10}, \dots, 2^{-20}$), seperti yang digunakan dalam studi oleh Probst et al. (2018) atau Sigrist (2020), karena nilai yang mendekati nol sering menunjukkan hasil yang baik.

Range:

Transformation:

Bounds:

Constraints:

Interactions:

Trans_2pow

Lower = -10; upper = 0

none

Ada hubungan antara η dan n rounds: Jika salah satu dari kedua parameter meningkat, yang lain sebaiknya dikurangi jika kesalahan tetap sama (Friedman 2001; Probst et al. 2019a). Hal ini juga ditunjukkan oleh Hastie et al. (2017)

Nilai η yang lebih kecil mengarah ke nilai n rounds yang lebih besar untuk risiko pelatihan yang sama, sehingga terdapat trade-off antara keduanya.

Selain itu, Hastie et al. (2017) juga menunjukkan korelasi dengan parameter subsample: Dalam sebuah studi empiris, subsample=1 dan $\eta=1$ menunjukkan hasil yang secara signifikan lebih buruk dibandingkan subsample=0,5 dan $\eta=0,1$. Jika subsample=0,5 dan $\eta=1$, hasilnya bahkan lebih buruk daripada $\eta=1$ dan subsample=1. Dalam kasus terbaik (subsample=0,5 dan

eta=0,1), bagaimanapun, nilai nrounds yang lebih besar diperlukan untuk mencapai hasil optimal.

2.6.2.3 XGBoost Hyperparameter lambda

Parameter lambda digunakan untuk regularisasi model. Parameter ini memengaruhi kompleksitas model (mirip dengan parameter dengan nama yang sama dalam EN). Signifikansi praktisnya dapat dijelaskan sebagai berikut: sebagai parameter regularisasi, lambda membantu mencegah *overfitting* (Chen dan Guestrin 2016). Dengan nilai yang lebih besar, model yang lebih halus atau lebih sederhana diharapkan.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	0
<i>Sensitivity:</i>	<i>Not known</i>
<i>Heuristics:</i>	<i>None known</i>
<i>Range:</i>	lambda $\in [0, \infty]$. Skala logaritmik tampaknya berguna, misalnya, 2–10,...,210, seperti yang digunakan dalam studi oleh Probst et al. (2019a) untuk mencakup berbagai nilai yang sangat kecil dan sangat besar.
<i>Transformation:</i>	Trans_2pow
<i>Bounds:</i>	lower = -10; upper = 10.
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	Karena baik lambda maupun alpha mengontrol regularisasi model, interaksi kemungkinan besar terjadi.

2.6.2.4 XGBoost Hyperparameter alpha

Para penulis paket R xgboost, Chen dan Guestrin (2016), tidak menyebutkan parameter ini. Dokumentasi dari implementasi referensi juga tidak memberikan informasi rinci tentang alpha. Karena deskripsinya sebagai parameter untuk regularisasi dari bobot (Chen et al. 2020), penggunaan yang sangat mirip dengan parameter yang sama namanya pada elastic net dapat diasumsikan. Makna

praktisnya dapat dijelaskan sebagai berikut: mirip dengan λ , α juga berfungsi sebagai parameter regularisasi.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	-10
<i>Sensitivity:</i>	<i>unknown</i>
<i>Heuristics:</i>	<i>No heuristics are known.</i>
<i>Range:</i>	$\alpha \in [0, \infty[$. Skala logaritmik tampaknya berguna, misalnya, , seperti yang digunakan dalam studi oleh Probst et al. (2019) untuk mencakup rentang nilai yang sangat kecil hingga sangat besar.
<i>Transformation:</i>	Trans_2pow
<i>Bounds:</i>	<i>lower</i> = -10; <i>upper</i> = 10.
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	Karena baik λ maupun α mengontrol regularisasi model, interaksi kemungkinan besar terjadi.

2.6.2.5 XGBoost Hyperparameter subsample

Dalam setiap langkah *boosting*, *tree* baru yang akan dibuat biasanya hanya dilatih pada sebagian dari seluruh set data, mirip dengan *random forest*. Parameter *subsample* menentukan bagian dari data yang dipilih secara acak pada setiap iterasi. Signifikansi praktisnya dapat dijelaskan sebagai berikut: efek yang jelas dari nilai *subsample* yang kecil adalah waktu pelatihan yang lebih singkat untuk *tree* individu, yang sebanding dengan *subsample*.

<i>Type:</i>	<i>Double, scalar</i>
<i>Default:</i>	1
<i>Sensitivity:</i>	Studi oleh Friedman (2002) menunjukkan sensitivitas yang tinggi terhadap nilai yang sangat kecil atau besar dari <i>subsample</i> . Namun, dalam rentang nilai <i>subsample</i> yang relatif besar (sekitar 0,3 hingga 0,6), hampir tidak ada perbedaan yang diamati dalam kualitas model.

<i>Heuristics:</i>	Hastie et al. (2017) menyarankan subsample = 0,5 sebagai nilai awal yang baik, tetapi menunjukkan bahwa nilai ini dapat dikurangi jika nrounds meningkat. Dengan banyak <i>tree</i> (nround besar) sudah cukup jika setiap <i>tree</i> individu melihat bagian data yang lebih kecil, karena data yang tidak terlihat lebih mungkin diperhitungkan di <i>tree</i> lain.
<i>Range:</i>	subsample $\in]0,1]$. Berdasarkan hasil empiris Friedman (2002); Hastie et al. (2017), skala logaritmik tidak disarankan.
<i>Transformation:</i>	Trans_id
<i>Bounds:</i>	lower = 0.1; upper = 1.
<i>Constraints:</i>	none
<i>Interactions:</i>	Ada hubungan antara eta, nrounds, dan subsample.

2.6.2.4 XGBoost Hyperparameter colsample_bytree

Parameter colsample_bytree memiliki kemiripan dengan parameter mtry pada hutan acak. Di sini juga, sejumlah fitur acak dipilih untuk pemisahan sebuah *tree*. Namun, di *XGBoost*, pilihan ini dibuat hanya sekali untuk setiap *tree* yang dibuat, bukan untuk setiap pemisahan (pengembang xgboost 2020). Di sini colsample_bytree adalah faktor relatif. Jumlah fitur yang dipilih adalah colsample_bytree $\times$ n. Makna praktisnya mirip dengan mtry: colsample_bytree memungkinkan *tree* dalam ansambel memiliki keberagaman yang lebih besar. Waktu eksekusi juga berkurang, karena Jumlah pemisahan yang lebih kecil harus diperiksa setiap kali (jika colsample_bytree <1).

<i>Type:</i>	Double, scalar
<i>Default:</i>	1
<i>Sensitivity:</i>	Studi empiris oleh Probst et al. (2019a) menunjukkan bahwa model ini sangat sensitif terhadap perubahan nilai colsample_bytree yang mendekati 1. Namun, sensitivitas ini menurun di sekitar nilai yang lebih sesuai.

<i>Heuristics:</i>	<i>None known</i>
<i>Range:</i>	<i>colsample_bytree $\in [0,1]$. Brownlee (2018) mentions search ranges such as colsample_bytree = 0.4, 0.6, 0.8, 1, but mostly works with colsample_bytree = 0.1, 0.2, ..., 1.</i>
<i>Transformation:</i>	<i>Trans_id</i>
<i>Bounds:</i>	<i>lower = 1/nFeatures; upper = 1.</i>
<i>Constraints:</i>	<i>none</i>
<i>Interactions:</i>	<i>None known.</i>

2.6.2.7 XGBoost Hyperparameter gamma

Parameter dari satu *tree* keputusan ini sangat mirip dengan parameter *cp*: Seperti *cp*, *gamma* mengontrol jumlah pemisahan pada *tree* dengan mengasumsikan peningkatan minimal untuk setiap pemisahan. Menurut dokumentasinya. Pengurangan kerugian minimum yang diperlukan untuk membuat partisi lebih lanjut pada simpul daun dari *tree*. Semakin besar nilainya, semakin konservatif algoritma tersebut. Perbedaan utama antara *cp* dan *gamma* adalah definisi *cp* sebagai faktor relatif, sedangkan *gamma* didefinisikan sebagai nilai absolut. Ini juga berarti bahwa rentangnya berbeda.

<i>Default:</i>	<i>-10</i>
<i>Range:</i>	<i>gamma $\in [0, \infty]$. Alogarithmic scale seems to make sense, e.g., 2⁻¹⁰, ..., 2¹⁰, as, e.g., in the study by Thomas et al. (2018) to cover a wide range of very small and very large values</i>
<i>Transformation:</i>	<i>Trans_2pow</i>
<i>Bounds:</i>	<i>lower = -10; upper = 10.</i>

2.6.2.8 XGBoost Hyperparameter maxdepth

Parameter dari satu *tree* keputusan ini sudah dikenal sebagai *maxdepth*.

<i>Default:</i>	6
<i>Heuristics/ Sensitivity:</i>	Meskipun dalam banyak aplikasi $J = 2$ akan kurang memadai, tidak mungkin $J > 10$ akan diperlukan. Pengalaman sejauh ini menunjukkan bahwa $4 \leq J \leq 8$ bekerja dengan baik dalam konteks boosting, dengan hasil yang cukup tidak sensitif terhadap pilihan tertentu di rentang ini.
<i>Transformation:</i>	Trans_id
<i>Bounds:</i>	$Lower = 1$; $upper = 15$.

2.6.2.9 XGBoost Hyperparameter min_child_weight

Seperti γ dan *max_depth*, *min_child_weight* membatasi jumlah pembelahan setiap *tree*. Dalam hal *min_child_weight*, batasan ini ditentukan menggunakan matriks Hessian dari fungsi kerugian (jumlahnya di semua pengamatan di setiap node terminal baru) (Chen et al., 2020; Sigrist, 2020). Dalam eksperimen oleh Sigrist (2020), parameter ini ternyata relatif sulit untuk disesuaikan: hasilnya menunjukkan bahwa penyetelan dengan *min_child_weight* memberikan hasil yang lebih buruk dibandingkan penyetelan dengan parameter serupa (batasan jumlah sampel per lempeng).

<i>Type:</i>	Double, scalar
<i>Default:</i>	0
<i>Sensitivity:</i>	unknown
<i>Heuristics:</i>	None known
<i>Range:</i>	$\text{min_child_weight} \in [0, \infty[$. Skala logaritmik tampaknya masuk akal, misalnya, 2–10,...,210, seperti yang digunakan dalam studi oleh Probst et al. (2019a) untuk mencakup rentang nilai yang sangat kecil hingga sangat besar.

Transformation: Trans_2pow

Bounds: lower = 0; upper = 7

Constraints: none

Interactions: Interaksi dengan parameter seperti gamma dan maxdepth kemungkinan terjadi, karena ketiga parameter tersebut memengaruhi kompleksitas *tree* individual dalam *ensemble*.

2.7 Rule-Based Filtering

Rule-based filtering adalah pendekatan berbasis aturan logis yang digunakan untuk menyaring data sebelum diproses oleh algoritma *Machine learning* [42]. Aturan ini biasanya mencakup panjang URL, keberadaan simbol mencurigakan (misalnya “@” atau “//”), jumlah subdomain, serta umur domain [42]. Karakteristik atau aturan yang sering digunakan dalam memfilter menggunakan *Rule Based Filtering* dalam penelitian terdahulu dapat dilihat dalam Tabel 2. 3 berikut:

Tabel 2. 3 Characteristics of Rule-based Filtering

No	Kategori	Rule	Deskripsi Rule	Contoh
1.	<i>Search Engine-based</i>	URL Tidak Terindeks	URL tidak muncul di hasil pencarian Google, Bing, atau Yahoo, maka kemungkinan besar situs baru dibuat untuk <i>phishing</i> .	URL seperti http://secure-login-update.com
2.	<i>Search Engine-based</i>	Domain Tidak Terindeks	Domain utama tidak muncul di indeks mesin pencari, maka berpotensi <i>phishing</i> .	Domain login-verifikasi-bankid.net
3.	<i>Keyword - based</i>	Keyword Mencurigakan	URL mengandung kata seperti “ <i>verify</i> ”, “ <i>update</i> ”, “ <i>account</i> ”, “ <i>secure</i> ”, dll.	http://paypal-secure-update.com

No	Kategori	Rule	Deskripsi Rule	Contoh
4.	<i>Obfuscation-based</i>	URL IP-based	Menggunakan IP address langsung sebagai URL.	http://192.168.0.10/login.php phisher sering memakai IP agar tidak mudah dilacak.
5.	<i>Obfuscation-based</i>	Karakter Mencurigakan	URL mengandung karakter seperti @, -, _ , atau port tidak standar.	http://login-paypal.com@fake.ru:8080
6.	<i>Obfuscation-based</i>	Panjang URL/Host	URL terlalu panjang atau banyak titik (dot).	http://secure-login.bank.update.account.verify.user.info.com mengandung banyak titik dan panjang.
7.	<i>Blacklist-based</i>	URL Masuk Daftar Hitam	URL tercantum dalam Google Safe Browsing atau PhishTank blacklist.	URL http://freegift-cardlogin.com ditemukan di PhishTank sebagai <i>phishing</i> .
8.	<i>Reputation-based</i>	Domain/IP Reputasi Buruk	Domain atau IP pernah dilaporkan hosting <i>phishing</i> .	Domain malwarehub.ru muncul dalam daftar reputasi buruk dari StopBadware.
9.	<i>Content-based</i>	Form Password Tidak Aman	Halaman memiliki form password yang tidak menggunakan HTTPS atau metode GET.	Situs http://bank-login.com/login memiliki field password namun tanpa SSL (HTTP).
10.	<i>Content-based</i>	Form Password ke Domain Eksternal	Form login mengirim data ke domain berbeda.	Situs http://bank-login.com mengirim form ke http://data-collector.net , indikasi <i>phishing</i> .
11.	<i>Content-based</i>	Meta Refresh ke	META tag melakukan redirect ke domain luar.	Tag <code><meta http-equiv="refresh"</code>

No	Kategori	Rule	Deskripsi Rule	Contoh
		Domain Eksternal		content="2;url=http://phishingsite.com">.
12.	<i>Content-based</i>	Redirect Server + Password Field	Server melakukan redirect dan ada password field di halaman.	http://secure-update.com mengalihkan pengguna ke http://verify-login.net dengan form login.
13.	<i>Content-based</i>	IFrame ke Domain Blacklist	Halaman memiliki iframe yang memuat konten dari domain jahat/blacklist.	<iframe src="http://phishform.com/login.html"> memuat form <i>phishing</i> dari domain luar.
14.	<i>Content-based</i>	Eksternal Link Lebih Banyak	Halaman memiliki lebih banyak link eksternal daripada internal, terutama dengan password field.	Halaman login palsu dengan banyak tautan eksternal menuju “bank verification”.
15.	<i>Content-based</i>	HTML Buruk + Password Field	Markup HTML rusak atau tidak sesuai standar dan memiliki field password.	Situs <i>phishing</i> hasil “copy-paste” dari halaman asli, tampilan rusak dengan form <i>login</i> palsu.

Sumber: [43]

Tabel 2. 4 menampilkan contoh penerapan metode *rule-based* dalam mendeteksi *phishing*. Setiap aturan (*rule*) dibuat berdasarkan fitur-fitur tertentu, seperti sertifikat SSL, otoritas penerbit sertifikat, serta keberadaan kata kunci mencurigakan, yang dapat menunjukkan indikasi *phishing*.

Tabel 2. 4 Contoh Rule-Based dalam Deteksi Phishing

No	Fitur	Deskripsi	Contoh
1.	SSL Certificate (HTTPS)	Memeriksa apakah link dalam email menggunakan HTTPS dengan SSL aktif. Email dengan link HTTP (tanpa SSL) dianggap mencurigakan.	Link: http://secure-login.com (tidak aman, tanpa SSL) terdeteksi <i>phishing</i> .
2.	Certificate Authority (CA)	Mengecek penerbit sertifikat SSL. Jika diterbitkan oleh otoritas tidak terpercaya (<i>self-signed</i> / palsu), maka email mencurigakan.	Sertifikat dari <i>UnknownCA</i> bukan dari <i>GoDaddy</i> atau <i>Comodo</i> .
3.	Blacklist Keywords	Mendeteksi kata kunci mencurigakan di isi email seperti “Click Now”, “Verify Now”, “Update Account”, dll.	Isi email: <i>Your account will expire in 24 hours! Verify now!</i> terdeteksi <i>phishing</i> .
4.	Redirection URL	Mengecek apakah link melakukan redirect tersembunyi ke domain lain (<i>hidden proxy server</i>).	Klik link http://bank-login.com ternyata diarahkan ke http://phishsite.net .
5.	Hiding Links / Shortened URLs	Mendeteksi tautan yang disamarkan dengan layanan seperti goo.gl , bit.ly , atau hyperlink teks HTML palsu.	Link teks 'Klik di sini untuk masuk ke Gmail' tapi mengarah ke http://login-update.com .
6.	Clear IP Address in URL	Memeriksa apakah link berisi alamat IP langsung (bukan nama domain). Ini umum pada situs <i>phishing</i> sementara.	Link: https://50.10.125.26/login.php IP-based URL, mencurigakan.
7.	Website Traffic (Alexa Rank)	Mengevaluasi popularitas situs berdasarkan trafik (Alexa rank ≤ 150.000 biasanya aman).	Situs bankofamerica.com memiliki trafik tinggi, sementara bofa-login-secure.info tidak terdaftar.

No	Fitur	Deskripsi	Contoh
8.	<i>Webpage Age</i> (<i>Domain Age</i>)	Memeriksa umur domain. Domain baru (<1 tahun) biasanya digunakan untuk <i>phishing</i> sementara.	secure-account-verif.com baru dibuat 2 minggu terdeteksi mencurigakan.
9.	<i>Sender Email Address</i>	Mengecek konsistensi antara domain pengirim dan isi email.	Email dari support@dropbox.com.fak e.co mengklaim dari Dropbox terdeteksi <i>phishing</i> .
10.	<i>Attached File Extension</i>	Mengecek ekstensi lampiran seperti .exe, .dll, atau .bat yang sering membawa malware.	Email dengan lampiran <i>invoice.exe</i> atau <i>update.dll</i> terdeteksi <i>phishing</i> .

Sumber: [44]

Fungsi utama *rule-based filtering* adalah:

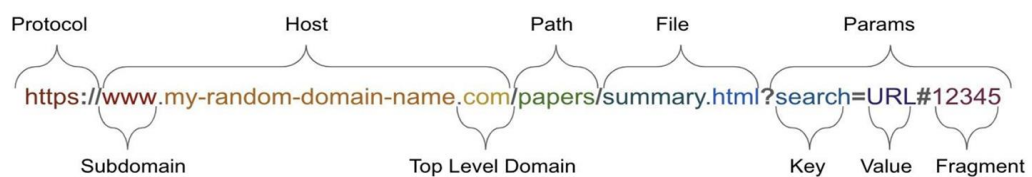
- Sebagai filter awal, untuk mendeteksi pola *phishing* klasik sehingga hanya kasus yang lebih kompleks diproses oleh algoritma *Machine learning*.
- Meningkatkan efisiensi komputasi, karena model *machine learning* tidak perlu memproses data yang sudah jelas *phishing*.
- Mengurangi kesalahan klasifikasi (*false positive* dan *false negative*), sehingga hasil prediksi lebih akurat.

Menurut [21], *rule-based filtering* terbukti mampu mengurangi *noise* data dan membantu meningkatkan performa sistem deteksi *phishing*, terutama ketika dikombinasikan dengan metode *Machine learning*.

2.7.1 Struktur URL

URL (*Uniform Resource Locator*) merupakan alamat unik yang digunakan untuk mengidentifikasi sumber daya di internet. Dalam konteks deteksi *phishing*, analisis struktur URL sangat penting karena sebagian besar serangan *phishing* dapat dikenali dari pola atau karakteristik tertentu pada URL [45].

Struktur URL terdiri dari beberapa bagian penting, yaitu *domain*, *path*, dan protokol. Domain ini seperti alamat yang memberi tahu tujuan penggunaan URL, apakah asli, dan dari negara mana asalnya. Selain itu, URL juga memiliki komponen lain seperti kata-kata, kategori, pengenalan, tanggal, atau singkatan khusus yang menunjukkan bagaimana informasi diatur atau bagaimana aplikasi menampilkan konten tersebut [45]. Karena fungsinya yang luas ini, URL sangat penting dalam berbagai aplikasi analisis data internet, terutama yang memerlukan pemeriksaan cepat seperti deteksi situs *phishing* atau malware.



Gambar 2. 2 Struktur URL

Sumber: [45]

Berikut ini adalah Tabel 2. 5 komponen dan data Fitur dari URL.

Tabel 2. 5 Komponen URL dan Data Fitur

No	Komponen	Subkomponen	Jenis Data Fitur
1.	<i>Protocol</i>		Menunjukkan jenis konten (seperti http atau https) dan tingkat keamanan.
2.	<i>Domain</i>	<i>Subdomain Top-Level Domain</i>	- Nama Utama situs yang menunjukkan Tujuan Keseluruhan - Menjelaskan Fungsi Khusus atau Pengkategorian tertentu dari konten di situs (misalnya, blog. atau support.). Menggambarkan Tujuan Bisnis/Layanan (.com, .org, .edu), Asal Geografis (.id, .uk), atau Kredibilitas situs.

3.	<i>Path</i>		Menjelaskan Bagaimana Konten Disusun dan diorganisasikan di situs, mirip Struktur Folder di komputer.
4.	<i>File</i>		Menunjukkan Aturan Penamaan File, Maksud Pembuat File tersebut, dan Jenis File (misalnya: dokumen, gambar, atau program yang bisa dijalankan).
5.	<i>Parameter</i>	<i>Keys Values Fragment</i>	Berisi detail untuk Personalisasi pengguna (misalnya, bahasa), Elemen Keamanan, dan Informasi Pelacakan aktivitas pengguna.

Sumber: [45]

2.8 Evaluasi

Tahap evaluasi model bertujuan untuk menilai kinerja sistem deteksi *website phishing* yang telah dibangun berdasarkan hasil pelatihan dan pengujian. Evaluasi dilakukan guna mengukur sejauh mana model mampu membedakan antara *website phishing* dan *website benign* secara akurat [46]. Proses evaluasi ini dilakukan dengan menerapkan sejumlah metrik evaluasi yang relevan dalam konteks klasifikasi biner. Tahapan ini juga mencakup perbandingan antara performa model individual (*Random Forest* dan *XGBoost*) dengan model gabungan (*Rule-Based Filtering + Stacking Ensemble*). Dengan demikian, hasil evaluasi dapat menunjukkan efektivitas pendekatan kombinasi. Berikut merupakan Tabel 2. 6 *confusion matrix*

Tabel 2. 6 *Confusion Matrix*

		Predicted		
		Yes	No	Total
Actual	Yes	TP	FN	P
	No	FP	TN	N
	Total	P	N	P+N

P (*positive*) adalah jumlah tupel positif, N (*negative*) adalah jumlah tupel negatif, TP (*True Positive*) adalah jumlah tupel positif yang diberi label dengan benar oleh

pengklasifikasi, TN (*True Negative*) adalah jumlah tupel negatif yang diberi label dengan benar oleh pengklasifikasi, FP (*False Positive*) adalah jumlah tupel negatif yang dilabelkan sebagai positif, dan FN (*False Negative*) adalah jumlah tupel positif yang dilabelkan sebagai negatif. Berikut adalah rumus yang digunakan untuk menghitung nilai-nilainya [46].

1. Akurasi (*Accuracy*)

Akurasi mengukur seberapa banyak prediksi model yang benar dibandingkan dengan seluruh data uji. Metrik ini digunakan untuk mengetahui tingkat ketepatan model secara keseluruhan [47].

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

[46]

2. Presisi (*Precision*)

Presisi menunjukkan sejauh mana prediksi positif yang diberikan model benar-benar akurat [47]. Nilai presisi yang tinggi menunjukkan model jarang salah mendeteksi *website* aman sebagai *phishing*.

$$Precision = \frac{TP}{TP + FP}$$

[46]

3. Recall (*Sensitivity*)

Recall menunjukkan kemampuan model dalam menemukan seluruh data positif yang sebenarnya. Nilai recall tinggi berarti model mampu mendeteksi sebagian besar situs *phishing* yang ada.

$$Recall = \frac{TP}{TP + FN}$$

[46]

4. *F1-Score*

F1-score merupakan rata-rata harmonik antara presisi dan recall, digunakan untuk menilai keseimbangan antara keduanya, terutama ketika data tidak seimbang.

$$F1\ Score = \frac{2 \times Recall \times Precision}{Recall + Precision}$$

[46]

5. *Area Under Curve (AUC)*

AUC diukur berdasarkan *Receiver Operating Characteristic (ROC) curve*, yang menggambarkan hubungan antara *True Positive Rate* dan *False Positive Rate*. Nilai AUC yang mendekati 1 menunjukkan model memiliki performa yang baik dalam membedakan dua kelas.

2.9 Penelitian Terdahulu

Tabel 2. 7 Penelitian Terdahulu

No	Research Title		Author	Year	Method	Results	Disadvantages
1	PhishGuard: A Multi-Layered Ensemble Model for Optimal <i>Phishing Website</i> Detection		Mourtaji et al.	2021	Ensemble (<i>Random Forest</i> + <i>XGBoost</i>)	Kombinasi dua algoritma meningkatkan akurasi dibandingkan model tunggal	Belum mengintegrasikan rule-based filtering sebagai lapisan awal.
2	Hybrid Rule-Based Solution for <i>Phishing</i> URL Detection Using CNN		Patgiri et al.	2021	Rule-based + CNN	Rule-based filtering efektif menyaring URL <i>phishing</i> klasik sehingga efisiensi meningkat	Terbatas pada fitur URL sederhana, tidak menguji algoritma ensemble
3	Enhancing <i>Phishing</i> Detection: Integrating <i>XGBoost</i> with Feature Selection Technique		Zhang et al.	2022	<i>XGBoost</i> + Feature Selection	Seleksi fitur meningkatkan akurasi <i>XGBoost</i> dan mengurangi overfitting	Tidak menguji kombinasi dengan algoritma lain (misalnya <i>Random Forest</i>)
4	A High-Accuracy <i>Phishing Website</i> Detection Method Based on <i>Machine learning</i>		Jain & Gupta	2023	Perbandingan beberapa ML (RF, XGB, LR, KNN)	<i>XGBoost</i> dan <i>Random Forest</i> terbukti paling unggul dalam akurasi	Dataset hanya berasal dari satu sumber, keterbatasan generalisasi
5	<i>Phishing</i> Detection Using <i>Machine learning</i> Algorithm		Vishesh Bharuka, et al.	2024	Membandingkan algoritma ML (Decision Tree, RF, Multilayer Perceptron, <i>XGBoost</i> , ANN, SVM)	<i>Random Forest</i> dan <i>XGBoost</i> mengungguli algoritma lain dalam metrik kinerja.	Efektivitas bergantung pada pilihan fitur, dataset, dan algoritma.

No	Research Title		Author	Year	Method	Results	Disadvantages
6	URL Feature Analysis for Effective <i>Phishing</i> Detection using <i>Machine learning</i>		Subashini, Dr. S. et al.	2025	Membandingkan algoritma ML (Decision Tree, RF, SVM, <i>XGBoost</i>) menggunakan 17 fitur URL (domain, HTML/JS).	Klasifikasi menggunakan <i>XGBoost</i> mencapai akurasi tertinggi.	Akurasi bergantung pada dataset hanya menganalisis fitur URL (bukan konten halaman).
7	Deteksi <i>Website Phishing</i> Menggunakan Teknik <i>Machine learning</i>		Lukito, W. et al.	2025	Membandingkan algoritma ML (<i>XGBoost</i> , <i>Decision Tree</i> , dan RF) dengan <i>hyperparameter tuning</i>	<i>XGBoost</i> dengan Grid Search tuning memberikan akurasi terbaik 96.14%.	Masih ada ruang untuk peningkatan akurasi. Perlu meningkatkan jumlah dan keragaman dataset.

BAB III

ANALISIS

Bab ini membahas analisis pengumpulan data, pembersihan data, eksplorasi data, pra-pemrosesan data, analisis model klasifikasi, dan analisis evaluasi penelitian. Analisis ini dilakukan agar mengetahui struktur data serta metode yang menjadi acuan dalam tahap perancangan.

3.1 Analisis Dataset

Subbab ini membahas dataset yang digunakan dalam penelitian. Dataset berbentuk tabular ini berasal dari beberapa sumber pada platform Kaggle, dan telah dilengkapi dengan berbagai atribut serta label yang membedakan antara URL *phishing* dan *non-phishing*.

3.1.1 Data Preparation

Subbab ini membahas tahapan persiapan data sebelum digunakan dalam proses analisis dan pemodelan. Kegiatan utama pada tahap ini meliputi pengumpulan data URL *phishing* dan *non-phishing*, identifikasi serta pemilihan atribut yang relevan, proses ekstraksi fitur dari URL, serta analisis awal terhadap hubungan antar atribut. Seluruh tahap ini dilakukan untuk memastikan bahwa data berada dalam kondisi yang siap digunakan pada tahap pemodelan selanjutnya.

3.1.1.1 Pengumpulan Data

Pada tahap ini dijelaskan dataset yang digunakan dalam penelitian memiliki karakteristik pada Tabel 3. 1 sebagai berikut:

Tabel 3. 1 Karakteristik Dataset Phishing

No	Karakteristik	Deskripsi
1.	Nama Dataset	dataset_phishing
2.	Sumber	https://www.kaggle.com/datasets/hafisafrizal/phising-dataset-url?select=dataset_phishing.csv
3.	Jumlah Sampel	11.430 baris data
4.	Jumlah Fitur	89 fitur (88 fitur independen + 1 label target)

No	Karakteristik	Deskripsi
5.	Label Target	<i>status</i> (0 = Non- <i>phishing</i> , 1 = <i>Phishing</i>)

3.1.1.2 Atribut Dataset

Pada tahap ini dilakukan analisis mendalam terhadap atribut–atribut yang terdapat pada dataset yang digunakan dalam penelitian. Analisis ini bertujuan untuk memahami karakteristik setiap fitur secara menyeluruh. Pemahaman yang tepat mengenai deskripsi dan makna dari setiap atribut menjadi landasan penting sebelum memasuki tahapan *preprocessing data*, pemilihan fitur, maupun pembangunan model *meachine learning*. Tabel 3. 2 ini adalah penjelasan dari atribut dataset pertama “dataset_phishing” yang berisi 11.431 URL, 89 *features*, serta non-*phishing* 5715 dan *phishing* 5715.

Tabel 3. 2 Atribut pada Dataset dataset_phishing

No	Feature	Deskripsi
1.	url	<i>String</i> teks mentah yang berisi alamat lengkap suatu situs web (URL) yang dianalisis. URL ini menjadi data dasar dari mana seluruh fitur lain diturunkan, karena setiap karakter, struktur, dan elemennya diproses untuk menghasilkan nilai pada fitur-fitur berikutnya.
2.	length_url	Menghitung total jumlah karakter (huruf, angka, simbol) dalam <i>string</i> URL.
3.	length_hostname	Menghitung panjang karakter hanya pada bagian nama host (domain utama dan <i>subdomain</i>), tidak termasuk protokol (<i>http://</i>) atau <i>path</i> (<i>/folder/file</i>).
4.	ip	Mendeteksi apakah URL menggunakan alamat IP (<i>numerik</i>) sebagai pengganti nama domain teks.
5.	nb_dots	Jumlah karakter '.'
6.	nb_hyphens	Jumlah karakter '-'
7.	nb_at	Jumlah karakter '@'
8.	nb_qm	Jumlah karakter '?'
9.	nb_and	Jumlah karakter '&'
10.	nb_or	Jumlah karakter ' '

No	Feature	Deskripsi
11.	nb_eq	Jumlah karakter '='
12.	nb_underscore	Jumlah karakter '_'
13.	nb_tilde	Jumlah karakter '~'
14.	nb_percent	Jumlah karakter '%'
15.	nb_slash	Jumlah karakter '/'
16.	nb_star	Jumlah karakter '*'
17.	nb_colon	Jumlah karakter ':'
18.	nb_comma	Jumlah karakter ','
19.	nb_semicolumn	Jumlah karakter ';'.
20.	nb_dollar	Jumlah karakter '\$'
21.	nb_space	Jumlah spasi atau %20 dalam URL
22.	nb_www	Menghitung kemunculan kata "www"
23.	nb_com	Menghitung kemunculan kata "com".
24.	nb_dslash	Menghitung garis miring ganda (//) pada URL, selain pada protokol http://
25.	http_in_path	Mencari kata "http" yang muncul di bagian belakang URL (<i>path</i>), bukan di depan sebagai protokol.
26.	https_token	Memeriksa apakah ada token "https" di dalam domain atau <i>path</i> .
27.	ratio_digits_url	Persentase karakter angka dalam URL. $ratio_digits_url = \frac{\text{Jumlah digit di seluruh URL}}{\text{Total panjang URL}}$
28.	ratio_digits_host	Persentase angka dalam hostname saja. $ratio_digits_host = \frac{\text{Jumlah digit pada hostname}}{\text{Total panjang hostname}}$
29.	punycode	Deteksi format Punycode (biasanya diawali xn--). Ini cara menulis domain berkarakter asing menggunakan huruf latin.
30.	port	Mendeteksi keberadaan nomor <i>port</i> eksplisit (angka setelah titik dua di <i>host</i>). Angka ini menunjukkan <i>port</i> eksplisit yang ditulis langsung di URL: 80, 443, 21, 8080, 3000, 5000.
31.	tld_in_path	Mendeteksi apakah ekstensi domain (seperti .com, .net, .org) muncul di bagian <i>path</i> (file/folder).

No	Feature	Deskripsi
32.	tld_in_subdomain	Mendeteksi apakah TLD muncul di bagian <i>subdomain</i> (di depan domain utama).
33.	abnormal_subdomain	Fitur biner yang menandai jika pola subdomain tidak sesuai standar umum (misal www bukan di depan).
34.	nb_subdomains	Menghitung jumlah <i>level subdomain</i> yang dipisahkan titik.
35.	prefix_suffix	Menandai keberadaan tanda strip (-) di nama domain utama (bukan <i>path</i>).
36.	random_domain	Apakah domain terlihat acak berdasarkan perhitungan karakter.
37.	shortening_service	Mencocokkan domain dengan daftar layanan penyingkat URL (seperti bit.ly, goo.gl).
38.	path_extension	Mendeteksi apakah URL diakhiri dengan ekstensi file umum (.html, .php, .txt).
39.	nb_redirection	Jumlah pengalihan halaman yang terjadi saat mengakses URL tersebut.
40.	nb_external_redirection	Jumlah <i>redirect</i> menuju domain lain.
41.	length_words_raw	Jumlah total kata yang ditemukan dalam URL mentah (biasanya dipisahkan oleh tanda baca).
42.	char_repeat	Menghitung jumlah karakter yang diulang secara berurutan (misal aaa).
43.	shortest_words_raw	Panjang (jumlah huruf) dari kata terpendek yang ditemukan dalam <i>string</i> URL.
44.	shortest_word_host	Panjang dari kata terpendek yang hanya berada di bagian <i>hostname</i> (domain) saja.
45.	shortest_word_path	Kata terpendek setelah domain
46.	longest_words_raw	Panjang (jumlah karakter) dari kata terpanjang yang ditemukan pada seluruh URL (<i>host + path + query + fragment</i>).
47.	longest_word_host	Jumlah karakter dari kata terpanjang di dalam domain
48.	longest_word_path	Jumlah karakter dari kata terpanjang di dalam path

No	Feature	Deskripsi
49.	avg_words_raw	Rata-rata dari jumlah karakter yang diekstrak dari seluruh URL (<i>subdomain</i> , <i>domain</i> , dan <i>path</i>) dibagi dengan jumlah katanya. Kata dipisahkan berdasarkan delimiter: /, ., ?, &, =, _, - dan TLD tidak ikut dihitung.
50.	avg_word_host	Rata-rata dari panjang karakter dibagi dengan kata yang terdapat di dalam <i>host</i> /domain saja (misalnya: <i>www</i> , <i>subdomain</i> , nama domain, TLD).
51.	avg_word_path	Rata-rata dari panjang karakter dibagi dengan kata dalam bagian <i>path</i> URL (setelah domain).
52.	phish_hints	Fitur yang menghitung jumlah kemunculan kata kunci curiga (seperti <i>login</i> , <i>secure</i> , <i>verify</i> , <i>account</i> , <i>bank</i> , <i>update</i> , <i>password</i> , dll.) di seluruh bagian URL (<i>host</i> , <i>subdomain</i> , <i>path</i> , dan <i>query</i>).
53.	domain_in_brand	Di dalam domain mengandung nama <i>brand</i> populer seperti <i>google</i> , <i>paypal</i> , <i>apple</i> , <i>amazon</i> .
54.	brand_in_subdomain	<i>Brand</i> muncul pada <i>subdomain</i> .
55.	brand_in_path	<i>Brand</i> muncul pada <i>path</i> URL.
56.	suspicious_tld	fitur URL menggunakan Top-Level Domain (TLD) yang murah, mudah didaftarkan, atau kurang memiliki pengawasan TLD seperti <i>.tk</i> , <i>.ml</i> , <i>.ga</i> , <i>.cf</i> , <i>.gq</i> , <i>.xyz</i> , <i>.top</i> , dan <i>.pw</i> .
57.	statistical_report	Domain atau URL telah dilaporkan sebagai berbahaya pada database reputasi keamanan, seperti <i>PhishTank</i> , <i>Spamhaus</i> , <i>Google SafeBrowsing</i> , atau layanan <i>blacklist</i> lainnya.
58.	nb_hyperlinks	Jumlah elemen <i>hyperlink</i> <code></code> dalam halaman web.
59.	ratio_intHyperlinks	Jumlah <i>hyperlink</i> ke domain yang sama didalam web seperti <i>login</i> ke web tersebut <i>href="/login"</i> .
60.	ratio_extHyperlinks	Jumlah <i>hyperlink</i> ke luar domain dari web seperti <i>https://google.com</i> .
61.	ratio_nullHyperlinks	Jumlah <i>hyperlink</i> yang tidak valid (<i>#</i> , <i>javascript</i> , <i>void(0)</i>).

No	Feature	Deskripsi
62.	nb_extCSS	Jumlah <i>file</i> CSS dari domain luar (< <i>link href="external.css"></i>).
63.	ratio_intRedirection	Jumlah perpindahan otomatis ke halaman dalam domain yang sama
64.	ratio_extRedirection	Jumlah perpindahan otomatis ke domain berbeda
65.	ratio_intErrors	Jumlah <i>error</i> HTTP dari domain internal (404, 403).
66.	ratio_extErrors	Jumlah permintaan <i>error</i> yang tidak berhasil dimuat dari domain lain dibagikan dengan jumlah semua permintaan
67.	login_form	Halaman yang bukan harusnya <i>login</i> tapi memiliki form dengan <i>password input</i>
68.	external_favicon	<i>Favicon</i> diambil dari domain lain.
69.	links_in_tags	Jumlah URL yang muncul di dalam <i>tag</i> HTML seperti < <i>meta</i> >, < <i>script</i> >, dan < <i>link</i> >, yaitu link yang dipakai oleh <i>website</i> di belakang layar untuk memuat <i>file</i> tambahan, bukan <i>link</i> yang bisa diklik oleh pengguna.
70.	submit_email	<i>Form</i> meminta mengirimkan data lewat email (<i>action=mailto:</i>).
71.	ratio_intMedia	Jumlah perbandingan gambar/video/audio dari domain itu sendiri dengan total semua gambar/video/audio.
72.	ratio_extMedia	Jumlah perbandingan gambar/video/audio dari luar domain dengan total semua gambar/video/audio.
73.	Sfh(Server Form Handler)	<i>Form</i> yang mengirim data ke domain luar atau kosong
74.	iframe	Halaman yang menampilkan tampilan web lain yang seharusnya bukan web itu.
75.	popup_window	Halaman yang menampilkan atau meminta data dari pengguna memakai <i>popup</i> JS.
76.	safe_anchor	Total <i>link</i> aman yang mengarah ke domain itu sendiri dibagi dengan total semua <i>link</i> yang menuju domain itu sendiri maupun di luar domain

No	Feature	Deskripsi
77.	onmouseover	Teks yang dibuat terlihat aman tapi sebenarnya itu adalah link <i>phishing</i> apabila di klik.
78.	right_click	Halaman menonaktifkan klik kanan. Digunakan untuk mencegah inspeksi elemen.
79.	empty_title	Tag <title> kosong atau tidak ada.
80.	domain_in_title	Domain muncul di title halaman.
81.	domain_with_copyright	Halaman menampilkan <i>copyright</i> (©) + domain yang biasanya aman.
82.	whois_registered_domain	Domain valid & teregister di WHOIS (<i>Who is the owner of this domain?</i>)
83.	domain_registration_length	Lama pendaftaran domain (tahun). Domain <i>phishing</i> sering umur pendek (<1 tahun).
84.	domain_age	Usia domain sejak dibuat. Domain baru (<6 bulan) cenderung <i>phishing</i> .
85.	web_traffic	<i>Ranking trafik</i> domain dari Alexa/SimilarWeb. Peringkat kunjungan <i>website</i> , situs dengan peringkat tinggi (angka besar) cenderung <i>phishing</i> .
86.	dns_record	Domain memiliki DNS <i>records</i> yang valid (A, MX, NS).
87.	google_index	Google mengindeks URL tersebut.
88.	page_rank	Seberapa banyak atau sering web itu dikunjungi maka akan diberi rank atau nilai yang tinggi(terpercaya)
89.	status	Kolom label atau target variabel dalam dataset yang menunjukkan klasifikasi keamanan suatu URL. Kolom ini memiliki dua kategori <i>output</i> : • <i>Phishing</i> = Tidak aman • <i>Legitimate</i> = Sah atau Aman

3.1.2 Data Preprocessing

Tahapan data *preprocessing* yang dilakukan pada penelitian ini, terdiri dari beberapa tahapan yang bertujuan untuk memperoleh data yang relevan dalam

mendeteksi *phishing* dan *non-phishing*. Tahapan data *preprocessing* yang dilakukan yaitu *data selection*, *data transformation*, *data cleaning*.

3.1.2.1 Data Selection

Data selection dilakukan untuk mengurangi atribut yang tidak terlalu berkontribusi dalam model, sehingga akan menyederhanakan *tree* pada model yang mempengaruhi kecepatan waktu pengaplikasian model terhadap tupel baru serta meningkatkan akurasi prediksi. Dilakukan langkah *drop column* untuk memilih beberapa kolom yang relevan dengan struktur URL dan label yang akan digunakan. Hasil *feature selection* ditunjukkan pada Tabel 3. 3 berikut ini.

Tabel 3. 3 Feature Selection

No	Feature	Digunakan		Alasan
		Ya	Tidak	
1.	url	✓		Dipilih karena menjadi sumber utama seluruh informasi, tempat semua pola <i>phishing</i> (struktur, simbol, kata kunci) berasal.
2.	length_url	✓		Dipilih karena URL <i>phishing</i> umumnya dibuat sangat panjang untuk menyembunyikan domain asli dan menipu pengguna agar tidak mudah mengenali tujuan sebenarnya.
3.	length_hostname	✓		Digunakan karena <i>hostname phishing</i> cenderung panjang dan kompleks akibat penggunaan <i>subdomain</i> berlapis untuk menyamarkan domain utama.
4.	ip	✓		Penggunaan alamat IP sebagai pengganti nama domain jarang ditemukan pada website sah dan sering digunakan oleh <i>phishing</i> .
5.	nb_dots	✓		Digunakan karena banyaknya titik mencerminkan jumlah <i>subdomain</i> yang sering dimanfaatkan <i>phishing</i> untuk menyamarkan domain utama.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
6.	nb_hyphens	✓		Digunakan karena tanda strip sering dipakai untuk meniru nama <i>brand</i> resmi.
7.	nb_at	✓		Digunakan sebagai indikator karena karakter '@' dapat menyebabkan browser mengabaikan bagian awal URL.
8.	nb_qm	✓		Dipilih karena penggunaan tanda tanya berlebih mengindikasikan manipulasi parameter URL.
9.	nb_and	✓		Dipertimbangkan karena karakter '&' yang berlebihan jarang ditemukan pada URL normal.
10.	nb_or		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
11.	nb_eq	✓		dipertimbangkan karena tanda '=' berlebih mengindikasikan manipulasi parameter URL
12.	nb_underscore	✓		dipilih karena garis bawah jarang digunakan pada domain resmi
13.	nb_tilde	✓		dipertimbangkan karena karakter '~' sering muncul pada struktur URL tidak standar.
14.	nb_percent	✓		dipilih karena simbol '%' sering digunakan untuk menyembunyikan karakter berbahaya melalui <i>encoding</i> .
15.	nb_slash	✓		digunakan untuk mengukur kompleksitas <i>path</i> yang sering diperpanjang pada URL <i>phishing</i> .
16.	nb_star	✓		dipilih karena karakter '*' tidak umum digunakan pada URL <i>website</i> resmi.
17.	nb_colon	✓		digunakan karena tanda ':' di luar protokol standar menunjukkan struktur URL tidak umum.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
18.	nb_comma	✓		dipilih karena koma jarang digunakan dalam URL resmi dan dapat menandakan URL buatan.
19.	nb_semicolumn	✓		digunakan karena titik koma sering dimanfaatkan untuk menyisipkan parameter tersembunyi.
20.	nb_dollar	✓		dipertimbangkan karena simbol '\$' sering dikaitkan dengan manipulasi atau injeksi data.
21.	nb_space	✓		digunakan sebagai indikator karena spasi atau '%20' menunjukkan URL tidak rapi dan mencurigakan.
22.	nb_www	✓		digunakan karena pengulangan kata 'www' sering menjadi upaya peniruan domain resmi
23.	nb_com	✓		Digunakan karena pengulangan kata “com” sering digunakan untuk meniru domain resmi .
24.	nb_dslash	✓		Digunakan karena penggunaan // berlebih di luar protokol dapat mengindikasikan manipulasi <i>path</i> URL.
25.	http_in_path	✓		Digunakan karena keberadaan kata “http” di dalam <i>path</i> sering digunakan untuk mengecoh pengguna agar terlihat sah.
26.	https_token	✓		dipertimbangkan karena kemunculan kata 'https' pada domain atau <i>path</i> sering digunakan untuk memberikan kesan aman palsu
27.	ratio_digits_url	✓		dipilih untuk mengukur proporsi angka yang tinggi pada URL <i>phishing</i>

No	Feature	Digunakan		Alasan
		Ya	Tidak	
28.	ratio_digits_host	✓		Digunakan karena <i>hostname phishing</i> sering mengandung banyak angka yang tidak umum pada domain resmi.
29.	punycode	✓		Digunakan karena <i>Punycode</i> sering dimanfaatkan untuk serangan <i>homograph attack</i> dengan karakter mirip huruf.
30.	port	✓		Digunakan karena penggunaan port tidak standar (selain 80/443) jarang ditemukan pada <i>website benign</i> .
31.	tld_in_path	✓		Digunakan karena TLD yang muncul di <i>path</i> merupakan teknik peniruan domain resmi.
32.	tld_in_subdomain	✓		Digunakan karena <i>phishing</i> sering meletakkan TLD palsu pada <i>subdomain</i> untuk mengelabui pengguna.
33.	abnormal_subdomain	✓		Digunakan karena struktur <i>subdomain</i> tidak wajar menunjukkan domain buatan atau otomatis.
34.	nb_subdomains	✓		dipilih karena semakin banyak <i>subdomain</i> , semakin sulit pengguna mengenali domain utama.
35.	prefix_suffix	✓		Digunakan karena tanda ‘-’ pada domain sering digunakan untuk meniru nama <i>brand</i> resmi.
36.	random_domain	✓		Digunakan karena domain acak umumnya dibuat secara otomatis dan jarang digunakan <i>website</i> sah
37.	shortening_service	✓		Digunakan karena URL <i>shortener</i> sering dimanfaatkan untuk menyembunyikan domain berbahaya.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
38.	path_extension	✓		Digunakan karena ekstensi file mencurigakan pada <i>path</i> (mis. <i>.exe</i> , <i>.php</i>) sering ditemukan pada <i>phishing</i> .
39.	nb_redirection	✓		Digunakan karena <i>phishing</i> sering menggunakan banyak <i>redirection</i> untuk menyamarkan tujuan akhir
40.	nb_external_redirect ion	✓		Digunakan karena pengalihan ke domain eksternal sering ditemukan pada <i>website phishing</i> .
41.	length_words_raw	✓		Digunakan untuk mengukur kompleksitas teks URL secara keseluruhan.
42.	char_repeat	✓		digunakan karena pengulangan karakter berlebih menunjukkan URL tidak natural
43.	shortest_words_raw	✓		Digunakan karena panjang kata paling pendek pada keseluruhan URL dapat menunjukkan penggunaan token tidak bermakna atau hasil pembangkitan otomatis yang umum pada URL <i>phishing</i> .
44.	shortest_word_host	✓		Digunakan karena <i>hostname phishing</i> sering mengandung potongan kata sangat pendek atau tidak natural yang jarang ditemukan pada domain resmi.
45.	shortest_word_path	✓		Digunakan karena <i>path</i> pada URL <i>phishing</i> sering mengandung kata sangat pendek sebagai bagian dari struktur buatan atau parameter tersembunyi.
46.	longest_words_raw	✓		Digunakan karena kata terpanjang yang ekstrem pada URL mengindikasikan upaya penyamaran melalui <i>string</i> panjang yang tidak mudah dibaca pengguna.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
47.	longest_word_host	✓		Digunakan karena <i>hostname phishing</i> sering memuat kata sangat panjang akibat penggabungan nama <i>brand</i> , kata kunci, dan karakter tambahan.
48.	longest_word_path	✓		Digunakan karena <i>path URL phishing</i> sering berisi kata sangat panjang yang digunakan untuk mengelabui atau menyembunyikan tujuan sebenarnya.
49.	avg_words_raw	✓		Digunakan karena rata-rata panjang kata pada URL mencerminkan tingkat kealamian struktur URL, di mana URL <i>phishing</i> cenderung memiliki pola kata yang tidak seimbang.
50.	avg_word_host	✓		Digunakan karena rata-rata panjang kata pada <i>hostname phishing</i> umumnya lebih tidak stabil dibandingkan <i>website</i> resmi yang terstruktur dengan baik.
51.	avg_word_path	✓		Digunakan karena rata-rata panjang kata pada <i>path</i> membantu mengidentifikasi URL <i>phishing</i> yang memiliki struktur path kompleks dan tidak natural.
52.	phish_hints	✓		Digunakan karena keberadaan kata kunci seperti <i>login</i> , <i>verify</i> , <i>secure</i> merupakan indikator <i>phishing</i> kuat.
53.	domain_in_brand	✓		Digunakan karena pencantuman domain pada <i>brand</i> sering digunakan untuk peniruan
54.	brand_in_subdomain	✓		Digunakan karena <i>phishing</i> sering meletakkan nama <i>brand</i> pada <i>subdomain</i> .
55.	brand_in_path	✓		Digunakan karena <i>path</i> sering dimanfaatkan untuk menyisipkan nama <i>brand</i> palsu.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
56.	suspicious_tld	✓		dipilih karena TLD murah atau minim pengawasan sering digunakan oleh <i>website phishing</i> .
57.	statistical_report	✓		Digunakan karena mencerminkan reputasi domain berdasarkan laporan keamanan.
58.	nb_hyperlinks	✓		Digunakan karena <i>phishing</i> sering memiliki jumlah <i>link</i> tidak wajar
59.	ratio_intHyperlinks	✓		digunakan untuk mengukur proporsi link internal yang umumnya tinggi pada <i>website</i> sah
60.	ratio_extHyperlinks	✓		dipertimbangkan karena <i>phishing</i> sering memiliki rasio <i>link</i> eksternal yang tinggi
61.	ratio_nullHyperlinks		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
62.	nb_extCSS	✓		dipilih karena ketergantungan pada <i>file</i> CSS eksternal dapat mengindikasikan <i>website</i> tidak terpercaya
63.	ratio_intRedirection		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
64.	ratio_extRedirection	✓		dipilih karena <i>redirect</i> ke domain luar sering digunakan untuk menyembunyikan tujuan sebenarnya
65.	ratio_intErrors		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
66.	ratio_extErrors	✓		Digunakan karena banyak <i>error</i> eksternal mengindikasikan <i>website</i> tidak stabil atau palsu.
67.	login_form	✓		Digunakan karena <i>form login</i> pada halaman mencurigakan merupakan ciri utama <i>phishing</i> .

No	Feature	Digunakan		Alasan
		Ya	Tidak	
68.	external_favicon	✓		digunakan karena <i>favicon</i> yang diambil dari domain lain menunjukkan <i>website</i> tidak mandiri
69.	links_in_tags	✓		Digunakan karena manipulasi <i>link</i> pada <i>tag</i> HTML sering ditemukan pada <i>phishing</i> .
70.	submit_email		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
71.	ratio_intMedia	✓		Digunakan karena <i>website</i> sah umumnya menggunakan media internal secara konsisten.
72.	ratio_extMedia	✓		Digunakan karena <i>phishing</i> sering memuat media dari domain eksternal
73.	Sfh(Server Form Handler)		✓	dimana yang seluruh baris datanya hanya berisi nilai 0
74.	iframe	✓		Digunakan karena <i>iframe</i> sering dipakai untuk menampilkan konten palsu dari domain lain.
75.	popup_window	✓		Digunakan karena <i>popup</i> sering digunakan untuk meminta data sensitif.
76.	safe_anchor	✓		Digunakan karena <i>anchor</i> tidak aman sering mengarah ke URL berbahaya.
77.	onmouseover	✓		Digunakan karena manipulasi <i>JavaScript</i> pada <i>hover</i> sering digunakan untuk menyembunyikan <i>link</i> asli.
78.	right_clic	✓		Digunakan karena pembatasan klik kanan sering ditemukan pada <i>website phishing</i>
79.	empty_title	✓		Digunakan karena halaman <i>phishing</i> sering memiliki judul kosong atau tidak relevan.

No	Feature	Digunakan		Alasan
		Ya	Tidak	
80.	domain_in_title	✓		Digunakan karena <i>website</i> resmi umumnya mencantumkan domain pada <i>title</i> .
81.	domain_with_copyri ght	✓		Digunakan karena <i>website</i> sah biasanya mencantumkan domain pada <i>copyright</i> .
82.	whois_registered_do main	✓		Digunakan karena domain <i>phishing</i> sering tidak terdaftar secara resmi.
83.	domain_registration _length	✓		Digunakan karena domain <i>phishing</i> biasanya berumur pendek.
84.	domain_age	✓		Digunakan karena usia domain merupakan indikator kuat legitimasi <i>website</i> .
85.	web_traffic	✓		Digunakan karena <i>website phishing</i> umumnya memiliki <i>traffic</i> rendah.
86.	dns_record	✓		Digunakan karena domain <i>phishing</i> sering tidak memiliki <i>DNS record valid</i> .
87.	google_index	✓		Digunakan karena banyak <i>website phishing</i> belum terindeks Google.
88.	page_rank	✓		Digunakan karena <i>website phishing</i> umumnya memiliki PageRank rendah.
89.	status	✓		digunakan sebagai variabel target untuk menentukan kelas URL, yaitu <i>phishing</i> atau <i>benign</i>

Berdasarkan hasil analisis dataset pada Tabel 3. 3, terdapat 6 fitur yang tidak digunakan dalam proses pemodelan, yaitu *nb_or*, *ratio_nullHyperlinks*, *ratio_intRedirection*, *ratio_extErrors*, *submit_email*, dan *sfh*, karena seluruh baris datanya hanya berisi nilai 0. Kolom-kolom dengan varians nol (*zero variance*) ini tidak digunakan karena tidak memiliki nilai pembeda yang berguna bagi model dalam mengenali pola antara kelas *phishing* dan *non-phishing*.

Selanjutnya, penelitian ini menggunakan pendekatan *URL-based feature extraction*, yaitu identifikasi karakteristik yang dapat diperoleh secara langsung dari struktur URL tanpa perlu mengakses isi halaman web. Pendekatan ini dipilih karena mampu menangkap pola-pola teknis yang sering muncul pada URL berbahaya, seperti penggunaan karakter khusus berlebihan, domain yang mencurigakan, hingga indikasi manipulasi struktur tautan. Berdasarkan proses seleksi, diperoleh 37 fitur *URL-based* yang dinilai relevan dan digunakan dalam penelitian ini, sebagaimana disajikan pada Tabel 3.4.

Tabel 3. 4 URL Based

"length_url", "length_hostname", "ip", "nb_dots", "nb_hyphens", "nb_at", "nb_qm", "nb_and", "nb_eq", "nb_underscore", "nb_tilde", "nb_percent", "nb_slash", "nb_star", "nb_colon", "nb_comma", "nb_semicolumn", "nb_dollar", "nb_space", "nb_www", "nb_com", "nb_dslash", "http_in_path", "https_token", "ratio_digits_url", "ratio_digits_host", "punycode", "port", "tld_in_path", "tld_in_subdomain", "abnormal_subdomain", "nb_subdomains", "prefix_suffix", "random_domain", "shortening_service", "path_extension", "nb_redirection"
--

3.1.2.2 Feature Engineering

Pada tahap *Feature Engineering* dilakukan proses *parsing* URL untuk mengekstraksi komponen-komponen penting dari setiap URL yang dapat dimanfaatkan sebagai fitur tambahan dalam proses analisis. URL memiliki struktur standar yang terdiri atas beberapa bagian, seperti *protocol*, *domain*, *path*, *query parameter*, dan *fragment*. Melalui proses *parsing*, setiap URL dipisahkan ke dalam komponen-komponen tersebut sehingga informasi yang terkandung di dalamnya dapat dianalisis secara lebih mendetail. Informasi hasil *parsing* ini selanjutnya dapat digunakan untuk membentuk fitur-fitur turunan, seperti panjang domain, jumlah subdomain, penggunaan protokol HTTPS, keberadaan alamat IP dalam

URL, serta karakteristik lainnya yang relevan dalam membedakan URL *phishing* dan URL non-*phishing*.

3.1.2.3 Data Transformation

Data transformation merupakan tahap penting dalam *preprocessing* data yang bertujuan untuk mengubah format dan struktur data mentah menjadi bentuk yang sesuai dengan kebutuhan algoritma *machine learning*. Pada penelitian ini, proses *data transformation* difokuskan pada transformasi label kelas.

Dataset yang diperoleh dari Kaggle memiliki kolom status sebagai label target yang merepresentasikan klasifikasi URL. Nilai pada kolom ini masih berupa data kategorikal dalam bentuk teks, yaitu “*phishing*” untuk URL berbahaya dan “*legitimate*” untuk URL yang aman. Sementara itu, sebagian besar algoritma *machine learning* memerlukan label dalam bentuk numerik agar dapat melakukan perhitungan dan pembelajaran pola secara matematis. Oleh karena itu, dilakukan transformasi label kelas dengan mengonversi nilai “*phishing*” menjadi nilai numerik 1 dan “*legitimate*” menjadi nilai numerik 0. Transformasi ini memungkinkan model untuk melakukan klasifikasi biner, di mana nilai 1 merepresentasikan kelas positif (*phishing*) sebagai fokus utama deteksi, sedangkan nilai 0 merepresentasikan kelas negatif (*legitimate*) yang menunjukkan URL aman.

3.1.2.3 Data Cleaning

Setelah tahap *data transformation*, dilakukan proses *data cleaning* untuk menjamin kualitas dan integritas dataset sebelum masuk ke tahap pemodelan. Proses pembersihan data dalam penelitian ini difokuskan pada aspek utama, yaitu penanganan data duplikat.

Pada *data cleaning* yang dilakukan adalah penanganan data duplikat. Proses ini dilakukan dengan mengidentifikasi dan menghapus baris data yang memiliki entri identik, khususnya pada fitur URL. Penghapusan baris dengan *link* yang sama bertujuan untuk menghindari redundansi informasi dan mencegah terjadinya bias

pada saat model melakukan proses *training*, sehingga setiap sampel data yang masuk ke tahap pemodelan bersifat unik.

3.2 Analisis *Machine Learning*

Subbab ini membahas analisis penerapan metode *machine learning* yang akan digunakan, yaitu *Random Forest* dan *XGBoost*. Analisis ini mencakup masing-masing model dalam mengolah data URL hasil *preprocessing*, serta perannya dalam mengklasifikasikan URL sebagai *phishing* atau *non-phishing*.

3.2.1 Analisis *Random Forest*

Pada penelitian ini, algoritma *Random Forest* dioptimalkan menggunakan beberapa *hyperparameter* utama. Penjelasan mengenai konsep dan karakteristik *hyperparameter Random Forest* telah diuraikan pada sub bab 2.5 *Random Forest*. Oleh karena itu, pada bagian ini pembahasan difokuskan pada pemilihan dan penggunaan *hyperparameter* dalam proses optimasi model. *Hyperparameter* yang dianalisis meliputi *num.trees*, *mtry*, *sample.fraction*, *replace (bootstrap)*, dan *respect.unordered.factors*, karena parameter-parameter tersebut mengontrol mekanisme utama *Random Forest* dalam membentuk *ensemble decision tree*.

Pemilihan *hyperparameter* ini dinilai telah mencukupi untuk mengendalikan kompleksitas model, meningkatkan diversitas antar *tree*, serta menjaga kemampuan generalisasi model dalam mendeteksi *website phishing*. *Hyperparameter* yang dipilih dalam penelitian ini merupakan *hyperparameter* inti *Random Forest* yang secara langsung memengaruhi mekanisme pembentukan *ensemble* dan paling relevan dengan karakteristik data *phishing* berbasis fitur hasil *rule-based filtering*. Ringkasan analisis pemilihan *hyperparameter* tersebut disajikan pada Tabel 3. 5 berikut.

Tabel 3. 5 Analisis Pemilihan *Hyperparameter Random Forest*

No	<i>Hyperparameter</i>	Alasan Pemilihan	Nilai
1.	<i>num.trees</i>	<i>Hyperparameter</i> ini mengatur jumlah <i>Decison Tree</i> yang	Nilai diuji pada beberapa tingkat dengan

No	Hyperparameter	Alasan Pemilihan	Nilai
		dibangun dalam model <i>Random Forest</i>. <i>Hyperparameter</i> ini digunakan karena semakin banyak jumlah <i>tree</i>, semakin stabil hasil prediksi karena varians model berkurang. Namun, jumlah <i>tree</i> yang terlalu besar akan meningkatkan waktu komputasi. Oleh karena itu, <i>num.trees</i> dianalisis untuk mendapatkan keseimbangan antara performa model dan efisiensi komputasi.	transformasi <i>trans_2pow_round</i>. Jumlah <i>tree</i> dipilih cukup besar untuk memastikan stabilitas model. Batas : <i>Lower</i> = 0; <i>upper</i> = 11. Rentang: <i>num. trees</i> $\in [1, \infty)$
2.	<i>mtry</i>	<i>Hyperparameter</i> <i>mtry</i> menentukan jumlah fitur yang dipilih secara acak pada setiap pemisahan node. Parameter ini berpengaruh terhadap tingkat keberagaman antar <i>tree</i>. Nilai <i>mtry</i> yang lebih kecil menghasilkan <i>tree</i> yang lebih beragam dan mengurangi korelasi antar <i>tree</i>, sehingga meningkatkan kinerja model <i>ensemble</i>, khususnya pada data <i>phishing</i> dengan banyak fitur berbasis aturan.	Nilai <i>mtry</i> ditentukan dengan nilai awal mengikuti <i>heuristik</i> umum $\sqrt{n}$. Nilai terbaik diperoleh melalui evaluasi performa model. Batas: <i>Lower</i> = 1 <i>upper</i> = <i>nFeatures</i> Rentang : <i>mtry</i> $\in [1, n]$.

No	Hyperparameter	Alasan Pemilihan	Nilai
3.	<i>sample.fraction</i>	<i>Hyperparameter</i> ini dipilih karena mampu mengatur proporsi data yang digunakan untuk melatih setiap <i>tree</i> . Penggunaan <i>sample.fraction</i> yang lebih kecil dari satu meningkatkan variasi data antar <i>tree</i> dan membantu mengurangi <i>overfitting</i> . Hal ini penting untuk meningkatkan kemampuan model dalam mengenali pola <i>phishing</i> yang beragam.	Nilai optimal dipilih berdasarkan hasil evaluasi performa dan waktu komputasi. Batas: <i>Lower</i> = 0.1; <i>upper</i> = 1 Rentang: <i>sample.fraction</i> $\in (0, 1]$.
4.	<i>replace</i> (<i>bootstrap</i>)	Parameter <i>replace</i> menentukan apakah pengambilan sampel data dilakukan dengan pengembalian (<i>bootstrap</i>). <i>Bootstrap</i> sampling meningkatkan variasi antar <i>tree</i> dan mengurangi korelasi antar <i>tree</i> , sehingga membantu menurunkan varians model. Parameter ini dianalisis untuk memastikan mekanisme bagging bekerja secara optimal.	Nilai yang digunakan adalah TRUE atau FALSE dan dievaluasi bersamaan dengan <i>sample.fraction</i> untuk memperoleh kombinasi sampling yang paling stabil. Rentang: <i>replace</i> $\in \{\text{TRUE}, \text{FALSE}\}$. Batas: <i>lower</i> = 1 (FALSE); <i>upper</i> = 2 (TRUE).
5.	<i>respect.unordered.factors</i>	<i>Hyperparameter</i> ini mengatur cara penanganan fitur kategorikal tanpa urutan alami. Parameter ini sangat relevan karena dataset <i>phishing</i>	Parameter <i>respect.unordered.factors</i> dapat dianggap sebagai nilai biner, di mana TRUE menunjukkan

No	Hyperparameter	Alasan Pemilihan	Nilai
		mengandung banyak fitur kategorikal berbasis aturan. Opsi order dipertimbangkan untuk menyeimbangkan kualitas model dan efisiensi komputasi dibandingkan partition yang lebih mahal secara waktu.	bahwa urutan dipertimbangkan, dan FALSE mengabaikan urutan. Pemilihan nilai ini bergantung pada jumlah kelas dan efisiensi komputasi. Batas: $lower = 1$ (<i>ignore</i>); $upper = 2$ (<i>order</i>). Rentang : $respect.unordered.factors \in \{ignore, order, partition\}$.

3.2.2 Analisis *XGBoost*

Pada penelitian ini, algoritma *XGBoost* digunakan sebagai salah satu metode utama dalam mendeteksi *website phishing* karena kemampuannya dalam memodelkan hubungan nonlinier dan menangani data dengan kompleksitas tinggi. Penjelasan konseptual mengenai mekanisme kerja dan karakteristik *hyperparameter XGBoost* telah diuraikan pada Subbab 2.6 *XGBoost*. Oleh karena itu, pada bagian ini pembahasan difokuskan pada pemilihan *hyperparameter* yang digunakan dalam proses pelatihan dan optimasi model, serta keterkaitannya dengan karakteristik data URL *phishing* yang dianalisis. *Hyperparameter* yang dipilih merupakan *hyperparameter* inti *XGBoost* yang secara langsung memengaruhi proses *boosting*, kompleksitas model, serta kemampuan generalisasi, sehingga relevan untuk digunakan dalam konteks penelitian ini. Ringkasan *hyperparameter XGBoost* yang digunakan dalam penelitian ini beserta alasan pemilihannya disajikan pada **Error! Reference source not found.** berikut.

Tabel 3. 6 Analisis Pemilihan Hyperparameter XGBoost

No	Hyperparameter	Alasan Pemilihan	Nilai
1.	<i>nrounds</i>	Dipilih karena <i>nrounds</i> mengontrol jumlah tree dalam proses <i>boosting</i> . Parameter ini berpengaruh langsung terhadap kompleksitas dan performa model. Pengaturannya memungkinkan model belajar secara bertahap tanpa meningkatkan risiko <i>overfitting</i> secara berlebihan, terutama ketika dikombinasikan dengan nilai <i>eta</i> yang kecil.	Rentang nilai dengan transformasi <i>trans_2pow_round</i> Bounds: <i>lower</i> = 0; <i>upper</i> = 11 Range : $\in [1, \infty[$. <i>Only integer values are valid.</i>
2.	<i>eta (learning rate)</i>	Dipilih untuk mengontrol kecepatan pembelajaran model. Nilai <i>eta</i> yang kecil membantu mengurangi pengaruh setiap <i>tree</i> tunggal, sehingga model menjadi lebih stabil dan memiliki kemampuan generalisasi yang lebih baik pada data <i>phishing</i> yang kompleks dan berisik.	Nilai kecil pada skala logaritmik (<i>trans_2pow</i>) Range <i>eta</i> $\in [0, 1]$. Bounds: <i>Lower</i> = -10; <i>upper</i> = 0
3.	<i>lambda (L2 regularization)</i>	Dipilih untuk mengendalikan kompleksitas model dan mencegah <i>overfitting</i> . Parameter ini berfungsi memperhalus bobot model sehingga hasil prediksi lebih stabil, terutama pada dataset dengan jumlah fitur yang cukup banyak.	Skala logaritmik (<i>trans_2pow</i>) Range <i>lambda</i> $\in [0, \infty[$. Bounds: <i>lower</i> = -10; <i>upper</i> = 10.
4.	<i>alpha (L1 regularization)</i>	Dipilih sebagai pelengkap <i>lambda</i> dalam mekanisme	Skala logaritmik (<i>trans_2pow</i>)

No	Hyperparameter	Alasan Pemilihan	Nilai
		regularisasi. <i>Alpha</i> membantu mengurangi pengaruh fitur yang kurang relevan, sehingga model lebih fokus pada fitur penting dalam mendeteksi pola <i>phishing</i> .	Range : $\alpha \in [0, \infty[$. Bounds: $lower = -10$; $upper = 10$.
5.	<i>subsample</i>	Dipilih untuk meningkatkan diversitas antar <i>tree</i> dengan melatih setiap <i>tree</i> hanya pada sebagian data. Pendekatan ini membantu mengurangi <i>overfitting</i> dan meningkatkan kemampuan generalisasi model tanpa menurunkan performa secara signifikan.	Range: $subsample \in]0, 1]$. Bounds: $lower = 0.1$; $upper = 1$.
6.	<i>colsample_bytree</i>	Dipilih karena mengontrol jumlah fitur yang digunakan pada setiap <i>tree</i> . Parameter ini meningkatkan keberagaman antar <i>tree</i> dan mengurangi ketergantungan model pada fitur tertentu, mirip dengan peran <i>mtry</i> pada <i>Random Forest</i> .	Range: $colsample_bytree \in]0, 1]$. Bounds: $(1/nFeatures - 1.0)$
7.	<i>gamma</i>	Dipilih untuk membatasi pemisahan node yang tidak memberikan peningkatan <i>loss</i> yang signifikan. Parameter ini membuat model lebih konservatif dan membantu mengurangi pembentukan <i>tree</i> yang terlalu kompleks.	Range: $\gamma \in [0, \infty[$. Skala logaritmik $(trans_2pow)$ Bounds: $lower = -10$; $upper = 10$.

No	Hyperparameter	Alasan Pemilihan	Nilai
8.	<i>max_depth</i>	Dipilih untuk mengontrol kedalaman <i>tree</i> dan mencegah model menjadi terlalu kompleks. Pembatasan kedalaman membantu menjaga keseimbangan antara kemampuan belajar pola dan risiko <i>overfitting</i> .	<i>trans_id</i> Rentang nilai terbatas (misalnya 1–15)
9.	<i>min_child_weight</i>	Dipilih untuk membatasi pembentukan <i>node</i> dengan jumlah informasi yang terlalu kecil. Parameter ini membantu mengurangi <i>split</i> yang tidak stabil dan meningkatkan ketahanan model terhadap <i>noise</i> pada data <i>phishing</i> .	Skala logaritmik (trans_2pow) Range : min_child_weight $\in [0, \infty[$. Bounds: <i>lower</i> = 0; <i>upper</i> = 7.

3.3 Analisis *Stacking* menggunakan *Logistic Regression*

Setelah menganalisis kinerja model dasar *Random Forest* dan *XGBoost*, langkah selanjutnya adalah menggabungkan prediksi dari kedua algoritma tersebut menggunakan pendekatan *ensemble learning* yaitu *Stacking* (*Stacked Generalization*). Penggabungan ini dilakukan untuk mengatasi keterbatasan yang dimiliki oleh masing-masing model dasar dan menghasilkan keputusan yang lebih seimbang. Berikut Tabel 3. 7 terdapat beberapa metode penggabungan model yang umum digunakan dalam *ensemble learning*.

Tabel 3. 7 Metode Umum *Ensamble Learning*

No	Metode	Cara Kerja	Kekurangan
1.	<i>Voting (Hard & Soft Voting)</i>	Memilih hasil prediksi terbanyak (<i>Hard</i>) atau	Menganggap setiap model punya bobot yang sama atau ditentukan manual. Tidak bisa

No	Metode	Cara Kerja	Kekurangan
		rata-rata probabilitas (<i>soft</i>).	mendeteksi jika mayoritas model salah arah.
2.	<i>Averaging</i>	Menghitung rata-rata angka dari semua prediksi model.	Tidak fleksibel terhadap pola non-linear. Jika ada satu model yang hasilnya sangat buruk (<i>outlier</i>), rata-ratanya akan ikut rusak.
3.	<i>Bagging (Bootstrap Aggregating)</i>	Melatih banyak model sejenis secara paralel.	Biasanya hanya menggunakan satu jenis algoritma. Jika algoritma tersebut punya kelemahan bawaan, semua model akan punya kelemahan yang sama.
4.	<i>Boosting</i>	Model dilatih berurutan, lalu model baru memperbaiki kesalahan model sebelumnya.	Sangat agresif mengejar kesalahan, sehingga jika data kotor, model akan "menghafal" sampah data tersebut (<i>overfitting</i>).
5.	<i>Stacking (Stacked Generalization)</i>	Menggunakan model tambahan (<i>Meta-Model</i>) untuk belajar menggabungkan prediksi.	Butuh waktu training lebih lama dan memori lebih besar karena melatih banyak lapisan model.

Stacking (Stacked Generalization) dipilih dalam penelitian ini karena metode ini menawarkan apa yang tidak dimiliki oleh teknik *ensemble* lain seperti *Voting* atau *Averaging*. Jika *Voting* hanya mengandalkan suara terbanyak, *Stacking* mampu mempelajari hubungan kompleks antar model. Alasan terkuatnya adalah kemampuan untuk menggabungkan algoritma yang berbeda yaitu *Random Forest* dan *XGBoost*. *Stacking* bertindak sebagai penengah yang secara otomatis memberikan bobot lebih tinggi kepada model yang paling akurat dan mengurangi pengaruh model yang sering salah, sehingga hasil akhirnya jauh lebih stabil dan akurat dalam menghadapi data baru.

Proses *Stacking* berjalan dalam dua tahap utama. Tahap pertama disebut *Level 0*, di mana beberapa model dasar (*base-learners*) dilatih menggunakan dataset asli. Setelah terlatih, model-model ini akan memprediksi data. Hasil prediksi dari masing-masing model tersebut tidak langsung dianggap sebagai jawaban akhir, melainkan dikumpulkan dan disusun menjadi sebuah dataset baru yang disebut *meta features*. Dataset baru inilah yang kemudian diteruskan ke tahap kedua, yaitu *Level 1*, di mana sebuah model pemutus yang disebut *meta learner* akan dilatih menggunakan input dari prediksi-prediksi *Level 0* tadi untuk menghasilkan keputusan akhir yang paling optimal.

Meta learner adalah "otak" di balik arsitektur *Stacking* yang bertugas sebagai pengambil keputusan final. *Meta learner* tidak lagi melihat data mentah (seperti fitur asli), melainkan belajar dari pola prediksi yang dihasilkan oleh *base-learners*. Tugas utamanya adalah mengenali kapan suatu model dasar bisa dipercaya dan kapan tidak. Misalnya, jika Model A selalu salah saat memprediksi data kategori tertentu, *meta learner* akan belajar untuk lebih mempercayai Model B pada kondisi tersebut. Secara spesifik dalam penelitian ini, *Logistic Regression* digunakan sebagai *meta learner* karena sederhana namun efektif dalam menggabungkan prediksi dari beberapa *base learner* dan merupakan *meta learner* yang umum digunakan karena performanya yang konsisten dan stabil dalam metode *stacking*.

3.4 Analisis *Rule based Filtering*

Pada sub-bab ini dilakukan analisis penerapan *rule-based filtering* sebagai tahap awal dalam mendeteksi indikasi *phishing* pada sebuah URL. Pendekatan ini digunakan untuk mengevaluasi karakteristik struktural URL yang umum ditemukan pada situs *phishing*, sehingga sistem dapat memberikan penilaian awal secara cepat tanpa memerlukan proses komputasi yang kompleks. *Rule-based filtering* memanfaatkan seperangkat aturan berbasis fitur URL untuk mengidentifikasi pola mencurigakan sebelum dilakukan proses analisis lanjutan.

Penelitian ini menggunakan total 89 fitur dalam proses deteksi *phishing*. Namun, tidak seluruh fitur tersebut digunakan sebagai dasar pembentukan aturan (*rule-based filtering*). Hal ini disebabkan oleh perbedaan karakteristik antar fitur, di mana tidak semua fitur memiliki pola kemunculan yang jelas dan konsisten untuk diformulasikan dalam bentuk aturan statis. Beberapa fitur menunjukkan indikasi *phishing* yang dapat dikenali secara langsung dari keberadaan karakter, struktur URL, atau atribut domain tertentu. Sebaliknya, terdapat fitur-fitur lain yang bersifat statistik, berbasis rasio, atau mencerminkan kompleksitas struktur URL, sehingga memerlukan proses pembelajaran pola dari data untuk dapat dimanfaatkan secara optimal.

Oleh karena itu, pemilihan fitur untuk *rule-based filtering* dilakukan melalui analisis karakteristik data, tinjauan terhadap penelitian terdahulu, serta evaluasi terhadap kejelasan pola dan kekuatan indikasi masing-masing fitur terhadap perilaku *phishing*. Fitur-fitur yang dijadikan dasar pembentukan aturan merupakan fitur yang dapat diidentifikasi secara langsung dari struktur URL, penggunaan token mencurigakan, serta informasi domain yang bersifat eksplisit. Sementara itu, fitur-fitur lainnya tetap digunakan pada tahap pembelajaran *machine learning* karena kontribusinya lebih efektif ketika dipelajari melalui hubungan non-linear antar fitur.

Setiap aturan merepresentasikan satu kondisi tertentu yang sering muncul pada URL *phishing*, seperti penggunaan karakter khusus, struktur domain yang kompleks, serta pola penyamaran URL. Tabel 3.6 menampilkan daftar fitur yang digunakan, kategori tingkat kepentingannya, serta aturan deteksi yang diterapkan pada masing-masing fitur.

Tabel 3. 8 Rule base Filtering

No	Rule	Kategori	Deskripsi Rule
1.	suspecious_tld	Sangat Penting	TLD yang ditandai sebagai mencurigakan seperti: TLD Negara:

No	Rule	Kategori	Deskripsi Rule
			• .pw (Palau) - sering digunakan untuk <i>phishing</i> • .top - populer untuk spam dan malware • .xyz - banyak digunakan untuk situs mencurigakan • .cc (Cocos Islands) - sering untuk konten ilegal • .ws (Samoa) - digunakan untuk skema penipuan • .biz - reputasi buruk karena spam TLD Generik Baru (New gTLD): • .club - sering untuk situs perjudian ilegal • .online - tinggi penyalahgunaan phishing • .site - banyak malware distribution • .live - digunakan untuk streaming ilegal • .work - sering untuk scam pekerjaan • .icu - tinggi aktivitas phishing • .info - spam dan konten <i>low-quality</i> TLD dengan Reputasi Buruk: • .cn (China) - banyak situs spam (meski benign juga banyak) • .ru (Russia) - sering dikaitkan dengan cybercrime

No	Rule	Kategori	Deskripsi Rule
			• .zip - baru tapi sangat berisiko karena mirip ekstensi file • .loan - jelas untuk pinjaman mencurigakan • .download - distribusi malware • .click - clickbait dan malware
2.	nb_at	Sangat Penting	≥ 1 karakter "@" ditandai sebagai mencurigakan, jika "@" sebelum <i>hostname</i> ditandai sebagai <i>phishing</i> .
3.	ip	Sangat Penting	• IP = 1, jika hostname URL mengandung IP Address. • IP = 0, jika <i>hostname</i> URL menggunakan nama domain dan tidak mengandung IP Address.
4.	nb_underscore	Sangat Penting	> 3 karakter garis bawah (_) ditandai sebagai mencurigakan, jika pada domain ditandai sebagai sangat mencurigakan.
5.	ratio_digits_url	Penting	Persentase angka dalam <i>hostname</i> . Jika > 0.3 (lebih dari 30% URL adalah angka).
6.	nb_subdomains	Penting	> 3 <i>subdomain</i> ditandai sebagai mencurigakan.
7.	nb_percent	Penting	> 5 karakter persen (%) ditandai sebagai mencurigakan.
8.	nb_tilde	Penting	≥ 1 karakter tilde "~" ditandai sebagai mencurigakan.
9.	nb_semicolumn	Penting	≥ 1 karakter titik koma (;) ditandai sebagai mencurigakan.
10.	nb_star	Penting	≥ 1 karakter <i>star</i> (*) ditandai sebagai mencurigakan.
11.	nb_comma	Penting	≥ 1 karakter koma (,) ditandai sebagai mencurigakan.

No	Rule	Kategori	Deskripsi Rule
12.	random_domain	Penting	Domain acak/tidak bermakna menunjukkan pembuatan otomatis (DGA-like).
13.	length_hostname	Cukup Penting	> 30 panjang karakter <i>hostname</i> ditandai sebagai mencurigakan.
14.	nb_dollar	Cukup Penting	≥ 1 karakter dolar "\$" ditandai sebagai mencurigakan.
15.	nb_qm	Cukup Penting	> 2 karakter tanda tanya "?" ditandai sebagai mencurigakan.
16.	nb_colon	Cukup Penting	> 1 karakter titik dua (:) (lebih dari protokol)
17.	nb_eq	Cukup Penting	> 8 karakter sama dengan (=) ditandai sebagai mencurigakan.
18.	nb_dots	Cukup Penting	> 4 karakter titik (.) ditandai sebagai mencurigakan.
19.	nb_slash	Cukup Penting	> 7 karakter garis miring (/) ditandai sebagai mencurigakan.
20.	nb_and	Cukup Penting	> 3 karakter and "&" ditandai sebagai mencurigakan.
21.	nb_hyphens	Cukup Penting	> 3 karakter minus (-) ditandai sebagai mencurigakan.
22.	http_in_path	Cukup Penting	Jika kata http muncul pada path, menandakan upaya penyamaran URL
23.	https_token	Cukup Penting	Token https di domain/path digunakan untuk memberi kesan aman palsu.
24.	port	Cukup Penting	Penggunaan port selain <i>default</i> (Standard Ports: 21, 22, 23, 80, 443, 445, 1433, 1521, 3306, and 3389). ditandai mencurigakan.
25.	shortening_service	Cukup Penting	Jika URL menggunakan layanan pemendek (bit.ly, tinyurl), ditandai mencurigakan

Aturan-aturan pada Tabel 3. 8 berfungsi sebagai aturan dasar yang mengevaluasi keberadaan karakteristik mencurigakan pada sebuah URL secara individual. Setiap aturan menghasilkan keluaran berupa kondisi terpenuhi atau tidak terpenuhi, tanpa langsung menentukan keputusan akhir.

Hasil evaluasi dari seluruh aturan tersebut kemudian dikombinasikan melalui mekanisme *rule-based filtering* untuk menentukan tingkat risiko *phishing* secara keseluruhan. Dalam mekanisme ini, kategori kepentingan fitur (sangat penting, penting, dan cukup penting) digunakan sebagai dasar untuk menilai seberapa besar kontribusi masing-masing aturan terhadap risiko *phishing*.

Proses pengambilan keputusan dilakukan melalui dua tahapan. Tahap pertama menggunakan aturan keras (*hard rule*), yaitu keputusan langsung yang diambil apabila ditemukan kombinasi fitur dengan tingkat kepentingan sangat penting yang menunjukkan pola *phishing* yang sangat kuat. Apabila kondisi ini tidak terpenuhi, sistem melanjutkan ke tahap kedua menggunakan aturan berbasis ambang batas (*threshold-based rule*), di mana skor risiko dihitung dari akumulasi bobot fitur yang terdeteksi dan dibandingkan dengan nilai ambang batas (*threshold*) untuk menentukan kategori URL.

3.4.1 Rule 1: High-Confidence Rule

Aturan keras digunakan untuk mendeteksi pola *phishing* dengan tingkat keyakinan sangat tinggi.

Ketentuan Rule 1:

Apabila ditemukan dua atau lebih (≥ 2) fitur dengan tingkat kepentingan sangat penting, maka URL langsung dikategorikan sebagai *phishing*, tanpa melalui proses perhitungan *threshold* lebih lanjut.

Rule 1 menetapkan bahwa URL langsung diklasifikasikan sebagai *phishing* apabila terdeteksi dua atau lebih fitur dengan tingkat kepentingan sangat penting. Penentuan kombinasi minimal dua fitur ini bertujuan untuk menghindari kesalahan

klasifikasi akibat satu fitur tunggal yang masih mungkin muncul pada URL sah dalam kondisi tertentu.

Kombinasi beberapa fitur sangat penting secara bersamaan menunjukkan pola yang sangat khas pada URL *phishing* dan jarang ditemukan pada website yang sah. Oleh karena itu, apabila dua atau lebih fitur sangat penting terdeteksi, URL dianggap memiliki tingkat risiko phishing yang sangat tinggi sehingga dapat langsung diklasifikasikan tanpa melalui perhitungan skor risiko berbasis *threshold*.

3.4.2 Rule 2 : *Threshold-Based Rule*

Apabila *Rule 1* tidak terpenuhi, maka sistem akan menerapkan perhitungan skor risiko berdasarkan akumulasi poin dari fitur-fitur yang terdeteksi. Pada Tabel 3. 9 diberikan bobot sebagai berikut:

Tabel 3. 9 Kategori Fitur

Kategori Fitur	Bobot
Sangat penting	3
Penting	2
Cukup penting	1

Skor risiko dihitung dengan menerapkan aturan umum *threshold-based risk scoring*, di mana setiap fitur memberikan kontribusi bobot tertentu terhadap nilai risiko. Rumus perhitungan skor risiko dinyatakan sebagai berikut:

$$Risk\ Score = (3 \times sangat\ penting) + (2 \times penting) + (1 \times cukup\ penting)$$

Berdasarkan skor risiko yang diperoleh, URL diklasifikasikan ke dalam beberapa kategori tingkat risiko menggunakan pendekatan *threshold-based classification*. Penentuan kategori dilakukan dengan membandingkan nilai skor risiko terhadap ambang batas (*threshold*) yang telah ditetapkan. Berikut adalah Tabel 3. 10 Nilai Skor Risiko

Tabel 3. 10 Nilai Skor Risiko pada *Rule-based*

<i>Risk Score</i>	Kategori
≥ 7	<i>Phishing</i>
4-6	<i>Suspicious</i>
1-3	<i>Caution</i>
0	<i>Benign</i>

URL yang dikategorikan sebagai *phishing* merupakan URL yang memenuhi aturan keras atau memiliki skor risiko tinggi (≥ 7). Pada kondisi ini, URL tidak dilanjutkan ke tahap *machine learning*, karena kombinasi fitur yang terdeteksi menunjukkan karakteristik *phishing* yang sangat kuat. Dengan demikian, keputusan phishing dapat diambil secara langsung untuk menghindari potensi *false negative* serta meningkatkan efisiensi proses deteksi.

Kategori *suspicious* diberikan kepada URL yang memiliki skor risiko sedang (4–6). URL dalam kategori ini menunjukkan adanya beberapa indikator *phishing*, namun belum cukup kuat untuk diklasifikasikan secara langsung sebagai *phishing*. Oleh karena itu, URL dengan status *suspicious* diteruskan ke tahap *machine learning* untuk dianalisis lebih lanjut menggunakan model *Random Forest* dan *XGBoost*.

Kategori *Caution* mencerminkan URL dengan skor risiko rendah (1–3). Pada kategori ini, hanya terdapat sedikit indikator yang berpotensi mengarah ke *phishing*. Meskipun demikian, URL dengan kategori *Caution* tetap diteruskan ke tahap *machine learning* sebagai langkah pemeriksaan tambahan, untuk memastikan bahwa tidak terdapat pola *phishing* tersembunyi yang belum terdeteksi oleh aturan sederhana.

Selanjutnya, URL yang dikategorikan sebagai *Benign* merupakan URL yang tidak menunjukkan adanya indikator *phishing* berdasarkan *rule-based filtering* (skor risiko 0). Meskipun demikian, URL pada kategori ini tetap diteruskan ke tahap *machine learning* sebagai langkah verifikasi tambahan. Hal ini dilakukan karena *rule-based filtering* memiliki keterbatasan dalam mendeteksi pola phishing yang

tidak dapat diidentifikasi hanya dari ciri-ciri sederhana pada URL, sehingga dapat dikenali lebih baik oleh model *machine learning*. Dengan tetap melakukan pemeriksaan menggunakan model *Random Forest* dan *XGBoost*, sistem dapat meminimalkan risiko *false negative* serta meningkatkan ketepatan hasil deteksi secara keseluruhan.

3.5 Analisis Eksperimen

Subbab ini membahas analisis eksperimen dilakukan untuk mengevaluasi kinerja berbagai pendekatan yang digunakan dalam sistem deteksi URL *phishing*, yaitu model *Random Forest*, *XGBoost*, penggabungan keduanya menggunakan metode *stacking*, serta integrasi *rule-based filtering* dengan model *machine learning*.

3.5.1 Eksperimen *Random Forest*

Pada eksperimen pertama ini, menggunakan algoritma *Random Forest* sebagai salah satu model klasifikasi tunggal yang akan diuji dalam mendeteksi URL *phishing*. *Random Forest* memanfaatkan kumpulan *decision tree* yang dibangun dari subset data dan fitur yang berbeda, sehingga mampu mengurangi *overfitting* dan mampu bekerja dengan baik pada data baru yang belum pernah dilihat sebelumnya. Karakteristik ini menjadikan *Random Forest* relevan untuk menangani pola *non-linear* pada fitur URL, seperti panjang URL, struktur hostname, dan keberadaan karakter khusus.

Analisis eksperimen terhadap *Random Forest* direncanakan untuk mengamati sejauh mana model ini mampu mengklasifikasikan URL *phishing* dan *non-phishing* secara tepat. Selain itu, evaluasi juga akan difokuskan pada kemampuan model dalam mendeteksi URL *phishing* secara menyeluruh, yang tercermin dari metrik evaluasi. Hasil pengujian *Random Forest* nantinya akan digunakan sebagai acuan *baseline* dalam membandingkan performa dengan pendekatan lainnya.

3.5.2 Eksperimen *XGBoost*

Pada eksperimen kedua, menggunakan algoritma *XGBoost* sebagai salah satu model klasifikasi tunggal yang dirancang sebagai model pembanding yang akan

diuji untuk meningkatkan performa deteksi URL *phishing*. Algoritma ini menerapkan metode *boosting*, di mana model dibangun secara bertahap dengan berfokus pada perbaikan kesalahan dari model sebelumnya. Pendekatan tersebut memungkinkan *XGBoost* mempelajari pola-pola kompleks yang sulit ditangkap oleh satu model klasifikasi tunggal.

Analisis eksperimen *XGBoost* direncanakan untuk mengevaluasi pengaruh mekanisme *boosting* terhadap peningkatan performa klasifikasi pada metrik evaluasi. Selain itu, aspek kompleksitas komputasi dan kebutuhan pengaturan parameter juga akan diperhatikan dalam analisis, guna menilai keseimbangan antara peningkatan performa dan efisiensi sistem.

3.5.3 Eksperimen *Stacking*

Dalam eksperimen ketiga ini, menggunakan metode *Stacking* (*Stacked Generalization*) digunakan sebagai pendekatan *ensemble* untuk meningkatkan performa model dalam melakukan prediksi. Metode *stacking* dipilih karena mampu menggabungkan keunggulan beberapa model *machine learning* dengan cara yang lebih adaptif dibandingkan metode *ensemble* sederhana seperti *voting* atau *averaging*. Pada metode *voting*, setiap model memiliki bobot yang sama, sehingga perbedaan tingkat kepercayaan dan pola kesalahan antar model tidak diperhitungkan secara eksplisit.

Algoritma *Random Forest* dan *XGBoost* dipilih sebagai *base learners* karena keduanya merupakan algoritma berbasis *tree*, tetapi memiliki mekanisme pembelajaran yang berbeda. *Random Forest* menggunakan metode *bagging* dengan membangun banyak *tree* yang belajar dari sampel data secara acak, sedangkan *XGBoost* menggunakan *boosting* yang membangun *tree* secara bertahap dengan memfokuskan diri pada kesalahan model sebelumnya. Perbedaan prinsip pembelajaran ini menghasilkan dua model dengan pola prediksi yang tidak identik. Sebaliknya, *XGBoost* merupakan algoritma *boosting* yang membangun model secara bertahap dengan fokus memperbaiki kesalahan model sebelumnya. Pendekatan ini memungkinkan *XGBoost* untuk menangkap pola-pola kompleks dan

hubungan *non-linear* antar fitur, meskipun memiliki sensitivitas yang lebih tinggi terhadap *overfitting*. Dengan menggabungkan kedua model sebagai *base learners*, sistem mampu memanfaatkan keunggulan *Random Forest* dalam stabilitas dan keunggulan *XGBoost* dalam ketelitian pembelajaran.

Stacking mengambil sejumlah algoritma *learning* $\{A_1, \dots, A_N\}$ dan menjalankannya pada dataset T yang sedang dipertimbangkan (yaitu, data tingkat dasar) untuk menghasilkan serangkaian model $\{h_1, \dots, h_N\}$. Kemudian, dataset baru T dibuat dengan mengganti deskripsi setiap *instance* dalam dataset tingkat dasar dengan prediksi dari setiap model tingkat dasar untuk *instance* tersebut.

Meta-learner berperan sebagai pengambil keputusan akhir dalam arsitektur *stacking*. Dalam penelitian ini, *meta-learner* dilatih menggunakan *output* probabilistik dari *Random Forest* dan *XGBoost* sebagai fitur masukan. Dengan pendekatan ini, *meta-learner* tidak hanya menggabungkan prediksi, tetapi juga mempelajari kondisi di mana salah satu model lebih dapat dipercaya dibandingkan model lainnya.

Peran *meta-learner* sangat penting dalam mengatasi konflik prediksi antara *Random Forest* dan *XGBoost*, terutama pada kasus URL *phishing* yang bersifat ambigu. *Meta-learner* memungkinkan sistem untuk menyeimbangkan bias dan varians dari kedua *base learners*, sehingga menghasilkan keputusan yang lebih stabil dan akurat. Dengan demikian, penggunaan *stacking ensemble* tidak hanya meningkatkan performa prediksi, tetapi juga memperkuat kemampuan sistem dalam beradaptasi terhadap variasi pola serangan *phishing* yang terus berkembang.

3.5.4 Eksperimen *Rule based Filtering* dengan *Stacking*

Pada eksperimen keempat mengintegrasikan *rule-based filtering* sebagai lapisan penyaring awal dengan model *stacking ensemble* yang menggabungkan *Random Forest* dan *XGBoost*. Pendekatan *hybrid* ini dirancang untuk memanfaatkan keunggulan dari metode berbasis aturan yang relevan, serta kekuatan *machine learning* yang mampu menangkap pola kompleks. Integrasi ini bertujuan untuk

meningkatkan akurasi deteksi, mengurangi kesalahan klasifikasi, dan meningkatkan efisiensi komputasi sistem secara keseluruhan.

Rule-based filtering berperan sebagai mekanisme deteksi cepat yang mengevaluasi URL berdasarkan karakteristik struktural yang telah diidentifikasi sebagai indikator kuat *phishing*. Aturan-aturan ini didasarkan pada analisis fitur yang telah dilakukan pada Subbab sebelumnya, mencakup karakteristik seperti penggunaan TLD mencurigakan, keberadaan karakter khusus tertentu, penggunaan alamat IP sebagai domain, dan pola-pola struktural URL yang tidak wajar.

Proses *filtering* dimulai dengan ekstraksi fitur dari URL yang akan dievaluasi. Fitur-fitur ini kemudian diperiksa terhadap serangkaian aturan yang telah didefinisikan, dengan setiap aturan memiliki tingkat kepentingan yang berbeda sebagaimana dianalisis pada Tabel 3. 8. Aturan dikategorikan menjadi tiga tingkat kepentingan yaitu sangat penting, penting, dan cukup penting, yang mencerminkan seberapa kuat indikasi aturan tersebut terhadap klasifikasi *phishing*.

Aturan dengan kategori "sangat penting" mencakup fitur-fitur yang memiliki korelasi tinggi dengan URL *phishing* dan jarang ditemukan pada URL *benign*. Contohnya adalah penggunaan TLD mencurigakan seperti *.xyz*, *.top*, *.pw*, atau *.icu* yang sering digunakan untuk aktivitas *phishing* karena mudah didaftarkan dan memiliki pengawasan minimal. Keberadaan karakter '@' dalam URL, terutama jika muncul sebelum *hostname*, juga merupakan indikator kuat karena dapat menyebabkan browser mengabaikan bagian awal URL. Penggunaan alamat IP sebagai domain menggantikan nama domain teks juga termasuk kategori sangat penting karena jarang digunakan oleh *website* sah.

Aturan dengan kategori "penting" mencakup fitur-fitur yang berkontribusi signifikan dalam membedakan *phishing* dan situs *benign*, namun tidak cukup untuk langsung menetapkan klasifikasi secara definitif. Contohnya adalah jumlah *subdomain* yang berlebihan (>3), penggunaan karakter persen (%) yang tinggi, atau keberadaan karakter-karakter khusus seperti tilde (~) dan *semicolon* (;) yang tidak

umum dalam URL normal. Fitur-fitur ini perlu dikombinasikan dengan indikator lain untuk menghasilkan keputusan yang lebih akurat.

Aturan dengan kategori "cukup penting" merupakan fitur pendukung yang memiliki pengaruh relatif kecil namun dapat meningkatkan akurasi ketika digabungkan dengan fitur lain. Contohnya adalah panjang *hostname* yang berlebihan, jumlah karakter tertentu seperti tanda tanya (?), sama dengan (=), atau garis miring (/) yang melebihi batas normal. Meskipun fitur-fitur ini sendiri tidak cukup kuat sebagai indikator *phishing*, kombinasinya dapat memberikan informasi tambahan yang berharga.

Evaluasi *rule-based filtering* dilakukan menggunakan sistem skoring kumulatif berdasarkan tingkat kepentingan aturan yang terpenuhi. Berdasarkan skor risiko yang diperoleh, URL diklasifikasikan ke dalam empat kategori, yaitu *Phishing*, *Suspicious*, *Caution*, dan *Benign*, sesuai dengan ketentuan yang ditunjukkan pada tabel klasifikasi skor risiko. URL dengan skor risiko tinggi atau yang memenuhi satu atau lebih aturan sangat penting dapat langsung diklasifikasikan sebagai *Phishing* tanpa melibatkan model *machine learning*. URL dengan skor risiko menengah dikategorikan sebagai *Suspicious* dan diteruskan ke tahap analisis menggunakan model *machine learning*. Sementara itu, URL dengan skor risiko rendah diklasifikasikan sebagai *Caution* atau *Benign* dan tetap diverifikasi oleh model *machine learning* untuk memastikan keakuratan klasifikasi.

Integrasi *rule-based filtering* dengan *stacking ensemble* bertujuan untuk memanfaatkan keunggulan kedua pendekatan secara optimal. *Rule-based filtering* memberikan keputusan cepat untuk kasus-kasus *phishing* yang jelas, sehingga dapat mengurangi beban komputasi sistem. Di sisi lain, *stacking ensemble* digunakan untuk menganalisis URL dengan tingkat ambiguitas yang lebih tinggi, khususnya pada kategori *Suspicious*, *Caution*, dan *Benign*.

Pada tahap *stacking ensemble*, *Random Forest* dan *XGBoost* berperan sebagai *base learners* yang menganalisis fitur-fitur URL secara komprehensif. Kedua model menghasilkan prediksi probabilitas yang kemudian dikombinasikan oleh *meta-*

learner untuk menghasilkan keputusan klasifikasi akhir. Pendekatan ini memungkinkan sistem untuk menangkap pola-pola kompleks yang tidak dapat diidentifikasi oleh aturan sederhana.

Keunggulan pendekatan hibrid ini terletak pada kemampuannya menggabungkan kecepatan dan interpretabilitas *rule-based filtering* dengan adaptabilitas serta kemampuan pembelajaran dari *machine learning*. Selain itu, pendekatan ini juga berkontribusi dalam menekan *false positive* dan *false negative*. *Rule-based filtering* dapat dengan cepat mengidentifikasi *phishing* klasik, sementara *stacking ensemble* mampu mendeteksi teknik *phishing* yang lebih canggih dan terus berkembang.

Analisis kinerja sistem hibrid dilakukan dengan membandingkan hasil evaluasi terhadap tiga pendekatan sebelumnya, yaitu model *Random Forest* tunggal, *XGBoost* tunggal, dan *stacking ensemble* dengan *rule-based filtering*. Evaluasi mencakup metrik *accuracy*, *precision*, *recall*, *F1-score*, dan *AUC*, serta efisiensi komputasi yang diukur melalui waktu pemrosesan rata-rata per URL.

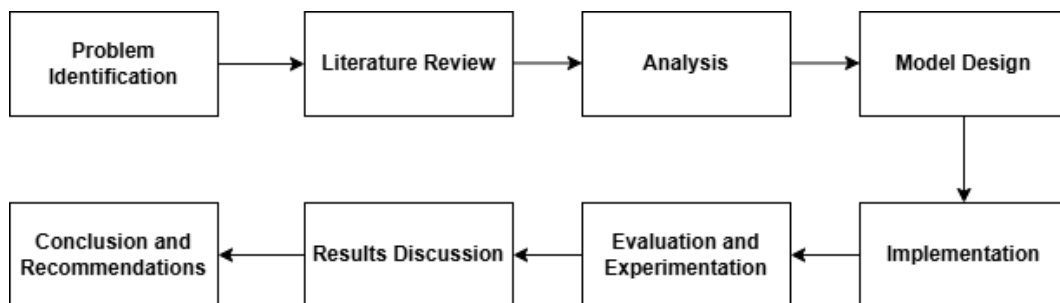
BAB IV

DESAIN

Pada bab ini menjelaskan desain sistem yang akan diimplementasikan dan metode penelitian yang akan diterapkan dalam penelitian ini. Desain sistem yang dimaksud meliputi beberapa aspek utama, meliputi penerapan metode *rule-based*, algoritma *Random Forest*, *XGBoost*, serta pendekatan *Stacking Ensemble* sebagai metode penggabungan.

4.1 Metode Penelitian

Metode Penelitian ini menggunakan serangkaian langkah - langkah, pengumpulan data serta instrumen penelitian yang akan digunakan untuk mencapai hasil. Langkah - langkah penelitian yang dilakukan dapat dilihat pada Gambar 4. 1 Tahapan Penelitian berikut.



Gambar 4. 1 Tahapan Penelitian

Berikut adalah penjelasan tahapan penelitian Tugas Akhir yang terdapat pada gambar diatas.

1. *Problem Identification*

Tahap pembentukan masalah merupakan hal utama yang ditentukan selama penelitian awal. Rumusan masalah adalah pernyataan khusus mengenai ruang lingkup masalah yang sedang dipelajari serta upaya untuk menyatakan pertanyaan mana yang akan dibahas.

2. *Literature Review*

Tahap studi literatur dilakukan untuk memperoleh pemahaman teoritis dan empiris terkait topik penelitian. Tahapan ini bertujuan untuk memperkuat

dasar teori, meninjau penelitian sebelumnya, dan menentukan pendekatan metodologis yang paling sesuai dengan tujuan penelitian.

3. *Analysis*

Pada tahap ini dilakukan analisis data dan metode yang digunakan. Metode pengumpulan informasi adalah dengan mencari dataset terkait topik penelitian ini di kaggle.com yang menyediakan data URL *phishing* dan *benign*. Data yang dikumpulkan meliputi berbagai atribut yang relevan seperti struktur URL, panjang domain, jumlah subdomain, dan umur domain.

4. *Model Design*

Pada tahap perancangan akan dilakukan proses perancangan arsitektur dan sistem yang akan dibangun berdasarkan hasil analisis data dan algoritma yang digunakan. Perancangan dirancang untuk dijadikan sebagai acuan dalam implementasi dan pengembangan sistem.

5. *Implementation*

Pada tahap ini, penulis akan menerapkan teknik- teknik yang telah dianalisis dan dirancang sebelumnya. Penulis akan melakukan implementasi yang melibatkan pendekatan *machine learning* terhadap desain yang di analisis untuk mendapatkan desain yang diharapkan. Eksperimen dilakukan untuk membandingkan kinerja model ensemble yang menggabungkan *Rule-Based Filtering*, *Random Forest*, dan *XGBoost* dengan model-model individual yang dijalankan secara terpisah.

6. *Evaluation and Experimentation*

Pada tahap ini, dilakukan evaluasi kinerja keberhasilan proyek dan hasil eksperimen untuk menilai performa model deteksi *phishing* yang dikembangkan. Pengukuran dilakukan menggunakan metrik akurasi, *precision*, *recall*, *F1-score*, dan *AUC* untuk melihat kemampuan model dalam membedakan URL *phishing* dan *benign*. Evaluasi juga membandingkan model tunggal dan model hybrid dengan *rule-based filtering* guna mengetahui efektivitas kombinasi metode dalam menurunkan kesalahan klasifikasi.

7. *Results Discussion*

Pada tahap ini berisi pembahasan dari hasil yang didapatkan dari evaluasi dan eksperimen terhadap model. Hasil dari eksperimen digunakan untuk menentukan konfigurasi model terbaik berdasarkan keseimbangan akurasi dan efisiensi deteksi.

8. *Conclusion and Recommendations*

Tahap ini dimuat untuk mengambil keputusan dari hasil evaluasi dan memberikan saran untuk pengembangan selanjutnya. Tahapan akhir penelitian ini adalah menyusun kesimpulan untuk menjawab efektivitas kombinasi *rule-based filtering* dengan algoritma *Random Forest* dan *XGBoost* dalam mendeteksi *phishing*.

4.2 Rancangan Sistem

Pada subbab ini dijelaskan rancangan sistem yang digunakan dalam penelitian untuk mendeteksi URL *phishing* dengan pendekatan *hybrid*. Sistem ini dirancang untuk mengintegrasikan komponen *rule-based Filtering* dengan model *machine learning*.

4.2.1 Desain Umum Sistem

Arsitektur sistem dirancang secara bertahap dan terstruktur untuk memastikan bahwa data yang digunakan memiliki kualitas yang baik serta proses deteksi dapat dilakukan secara akurat dan efisien. Secara umum, arsitektur ini mengombinasikan dua pendekatan utama, yaitu pendekatan *rule-based filtering* dan *machine learning* berbasis *ensemble*, yang saling melengkapi dalam mendeteksi URL *phishing*. Pendekatan *rule-based* berfungsi sebagai *filtering* awal yang cepat, sedangkan pendekatan *machine learning* digunakan untuk melakukan analisis yang lebih mendalam terhadap URL yang tidak dapat diputuskan secara tegas oleh aturan.

Tahap awal dalam arsitektur sistem adalah *Data Preparation*. Pada tahap ini, dataset URL dikumpulkan dari sumber yang relevan dan terpercaya sebagai bahan utama dalam proses pembelajaran model. Selanjutnya dilakukan analisis atribut dataset untuk memahami karakteristik setiap fitur, baik fitur berbasis struktur URL,

domain, maupun konten. Analisis ini bertujuan untuk memastikan bahwa atribut yang digunakan memiliki relevansi terhadap pola phishing serta mendukung proses ekstraksi fitur pada tahap berikutnya. Dengan adanya tahap *data preparation*, sistem memiliki dasar data yang terstruktur dan siap diproses lebih lanjut.

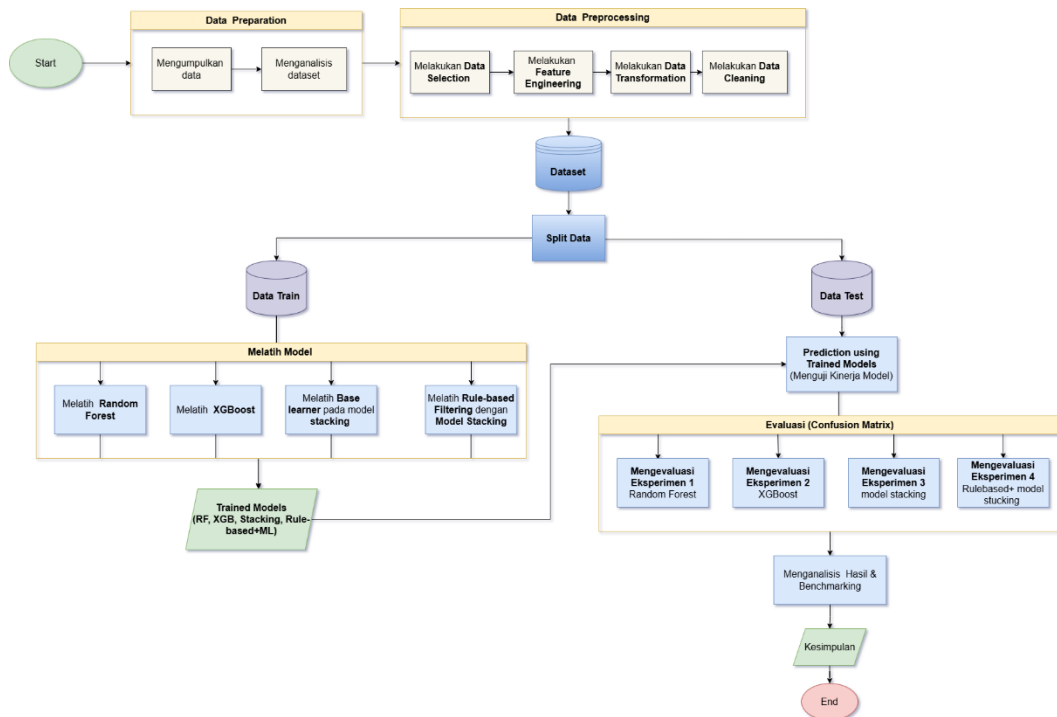
Setelah data dipersiapkan, sistem memasuki tahap *Data Preprocessing*. Tahap ini mencakup beberapa proses penting, yaitu *data selection*, *data cleaning*, *data integration*. *Data selection* dilakukan untuk memilih fitur-fitur yang paling relevan dalam mendeteksi phishing, sehingga dapat mengurangi kompleksitas model dan meningkatkan performa. *Data cleaning* bertujuan untuk menghapus data duplikat, data tidak lengkap, atau nilai yang tidak valid agar tidak mengganggu proses pembelajaran. Selanjutnya, *data integration* dilakukan apabila dataset berasal dari lebih dari satu sumber, sehingga seluruh data dapat digabungkan ke dalam satu format yang konsisten.

Dataset yang telah melalui tahap *preprocessing* kemudian dibagi pada tahap *Data Split* menjadi *training data* dan *testing data*. Data latih digunakan untuk membangun dan melatih model *machine learning*, sedangkan data uji digunakan untuk mengevaluasi performa model. Pada proses pelatihan, data latih dimasukkan ke dalam base learner yang terdiri dari algoritma *Random Forest* dan *XGBoost*. *Random Forest* memanfaatkan teknik *bagging* untuk meningkatkan stabilitas dan mengurangi *overfitting*, sedangkan *XGBoost* menggunakan teknik *boosting* yang berfokus pada perbaikan kesalahan prediksi secara bertahap. Hasil prediksi dari kedua *base learner* ini kemudian digabungkan menggunakan *meta learner* dengan pendekatan *stacking*, di mana algoritma *Logistic Regression* digunakan sebagai *meta-model* untuk menghasilkan keputusan akhir yang lebih optimal.

Pada tahap eksperimen atau pengujian, URL baru yang akan diuji terlebih dahulu diproses melalui *rule-based filtering*. Aturan-aturan ini dirancang berdasarkan karakteristik umum *phishing*, seperti panjang URL yang tidak wajar, penggunaan simbol mencurigakan, atau pola domain tertentu. Jika URL memenuhi aturan *phishing* secara jelas, maka sistem dapat langsung mengklasifikasikannya tanpa

harus melalui model *machine learning*. Sebaliknya, jika URL tidak memenuhi aturan secara tegas, URL tersebut akan diteruskan ke model *machine learning* untuk dianalisis lebih lanjut menggunakan *Random Forest*, *XGBoost*, dan *meta learner*.

Tahap akhir dari arsitektur ini adalah *hybridisasi*, yaitu penggabungan hasil dari *rule-based filtering* dan model *machine learning*. Pendekatan *hybrid* ini bertujuan untuk meningkatkan ketepatan deteksi, mengurangi kesalahan klasifikasi, serta menekan kemungkinan *false positive* dan *false negative*. *Output* akhir dari sistem berupa label biner, yaitu *phishing* atau *benign* (0-1), yang menjadi keputusan akhir dari proses deteksi. Berikut adalah Gambar 4. 2 Desain Umum Sistem.



Gambar 4. 2 Desain Umum Sistem

Diagram alur sistem menggunakan notasi *flowchart* standar untuk merepresentasikan proses deteksi phishing URL secara sistematis dan konsisten. Simbol oval (*terminator*) digunakan untuk menandai awal dan akhir proses, persegi panjang (*process*) merepresentasikan seluruh aktivitas pengolahan data dan komputasi seperti *data preparation*, *data preprocessing*, *data split*, serta pelatihan dan pengujian model *Random Forest*, *XGBoost*, dan *stacking*, sedangkan jajar

genjang (*input/output*) digunakan khusus untuk menunjukkan data yang masuk ke sistem misalnya input URL dan keluaran sistem berupa hasil klasifikasi. Simbol tabung (*data storage*) digunakan untuk menggambarkan dataset, *data training*, dan *data testing* sebagai media penyimpanan data, sementara belah ketupat (*decision*) merepresentasikan pengambilan keputusan logis pada tahap evaluasi *rule-based*, seperti penentuan apakah URL memenuhi aturan *phishing* atau *benign*. Selain itu, simbol dokumen (*document*) digunakan untuk merepresentasikan aturan *rule-based phishing* sebagai artefak tertulis yang didefinisikan sebelumnya dan digunakan oleh sistem, bukan sebagai proses komputasi, sehingga penggunaannya dibedakan secara konseptual dari proses penerapan *rule-based filtering*.

4.3 Desain Analisis Dataset

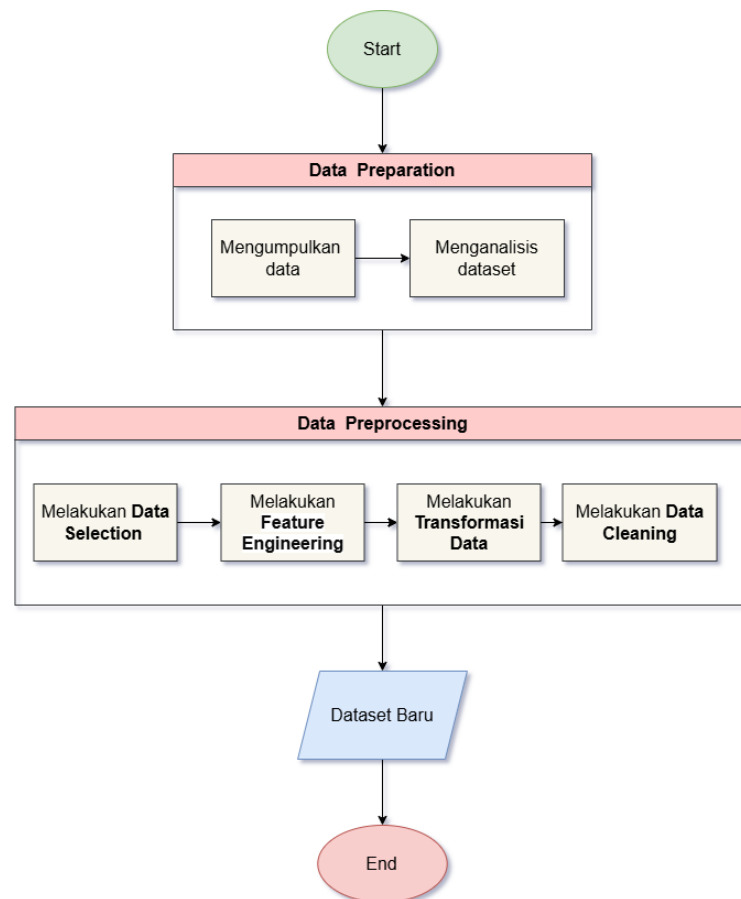
Desain Analisis Dataset merupakan kerangka kerja sistematis yang digunakan untuk mengelola dan menyiapkan data URL agar dapat digunakan secara optimal pada tahap pemodelan sistem deteksi *phishing*. Perancangan analisis dataset bertujuan untuk memastikan bahwa data yang digunakan telah melalui tahapan pengolahan yang sesuai, memiliki kualitas yang baik, serta relevan dengan tujuan penelitian.

Proses analisis dataset diawali dengan tahap *Data Preparation*, yang mencakup kegiatan pengumpulan dataset URL *phishing* dan *legitimate* dari platform Kaggle, serta analisis awal terhadap struktur dataset. Pada tahap ini dilakukan identifikasi jumlah data, jumlah atribut, serta karakteristik fitur yang tersedia guna memperoleh pemahaman menyeluruh terhadap dataset sebelum dilakukan pengolahan lebih lanjut.

Selanjutnya, proses dilanjutkan dengan tahap *Data Preprocessing* sebagai upaya penyiapan data agar berada dalam format yang seragam dan siap digunakan dalam proses pemodelan. Tahap *Data Preprocessing* terdiri dari beberapa sub-tahapan, yaitu *Data Selection*, *Feature Engineering*, *Data Transformation*, dan *Data Cleaning*. *Data Selection* dilakukan untuk memilih fitur-fitur yang relevan dengan kebutuhan deteksi *phishing*. *Feature Engineering* dilakukan untuk menyiapkan dan

memanfaatkan fitur-fitur URL sehingga mampu merepresentasikan pola phishing secara lebih efektif. *Data Transformation* digunakan untuk menyesuaikan format data, termasuk konversi label kelas ke dalam bentuk numerik. Sementara itu, *Data Cleaning* dilakukan untuk menghapus data duplikat serta fitur atau nilai yang tidak informatif sehingga dapat meningkatkan kualitas dataset.

Keseluruhan rangkaian tahapan analisis dataset menghasilkan dataset akhir yang bersih, terstruktur, dan konsisten, sehingga layak dan valid untuk digunakan pada tahap pemodelan *machine learning*. Desain alur Analisis Dataset pada penelitian ini ditampilkan pada Gambar 4. 3, yang menggambarkan hubungan antar tahapan pengolahan data secara menyeluruh.



Gambar 4. 3 Desain Analisis Dataset

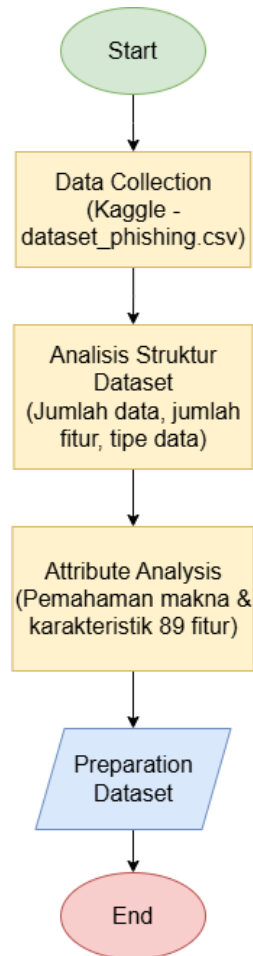
4.3.1 Desain *Data Preparation*

Tahap *Data Preparation* pada penelitian ini merupakan proses persiapan data sebelum digunakan pada tahap pemodelan dan evaluasi sistem deteksi *phishing*. Pada tahap ini, fokus utama kegiatan meliputi pengumpulan dataset URL, analisis atribut yang terdapat dalam dataset, serta penyesuaian struktur data agar siap digunakan dalam proses pengolahan dan pemodelan selanjutnya.

Proses *Data Preparation* dimulai dengan pengambilan dataset mentah URL *phishing* dan non-*phishing* dari *platform* Kaggle. Dataset yang digunakan memiliki total 11.431 data URL dengan 89 atribut fitur, yang mencerminkan berbagai karakteristik URL seperti struktur domain, penggunaan simbol khusus, serta elemen pendukung lainnya. Dataset tersebut telah dilengkapi dengan label klasifikasi yang membedakan antara URL *phishing* dan URL *legitimate*.

Setelah proses pengumpulan data, dilakukan analisis terhadap atribut–atribut yang terdapat pada dataset untuk memahami karakteristik masing-masing fitur, termasuk struktur kolom, tipe data, dan relevansinya terhadap tujuan penelitian. Analisis ini bertujuan untuk memastikan bahwa setiap atribut dipahami dengan baik sebelum dilakukan proses pengolahan data lebih lanjut, seperti *preprocessing*, seleksi fitur, dan penerapan metode *rule-based filtering* serta *machine learning*.

Desain alur *Data Preparation* pada penelitian ini ditampilkan pada Gambar 4. 4, yang menggambarkan tahapan pengambilan data, analisis atribut, dan kesiapan dataset sebelum memasuki tahap pemodelan sistem deteksi *phishing*.



Gambar 4. 4 Desain *Data Preparation*

4.3.2 Desain *Data Preprocessing*

Desain *Data Preprocessing* pada penelitian ini digunakan untuk menyiapkan data URL agar berada dalam kondisi optimal sebelum diproses oleh sistem deteksi phishing. Proses diawali dengan tahap *Load Dataset*, di mana dataset yang berisi URL *phishing* dan *non-phishing* dimuat dari sumbernya untuk kemudian diproses lebih lanjut.

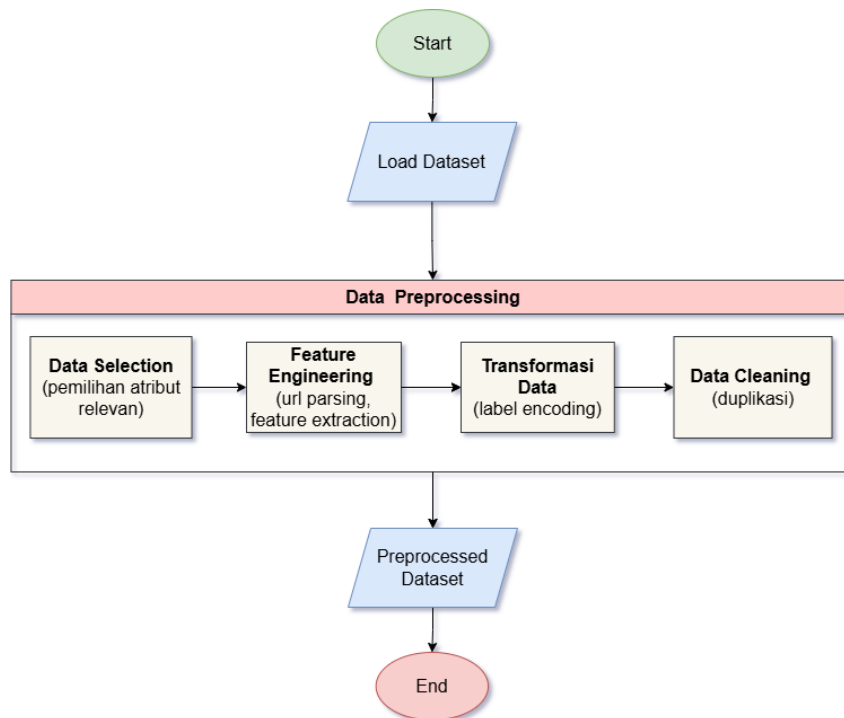
Tahap selanjutnya adalah *Data Selection* yaitu pemilihan atribut relevan, dilakukan proses seleksi terhadap atribut-atribut yang relevan dan memiliki kontribusi signifikan terhadap proses deteksi *phishing*. Pada tahap ini, hanya fitur-fitur yang berkaitan dengan karakteristik URL *phishing* yang dipilih, seperti panjang URL, struktur domain, keberadaan karakter khusus, penggunaan protokol HTTPS, dan pola URL yang mencurigakan.

Proses berlanjut ke tahap *Feature Engineering (url parsing, feature extraction)*, di mana dilakukan ekstraksi fitur-fitur dari URL mentah menjadi representasi numerik atau kategorikal yang dapat diproses oleh algoritma *machine learning*. Pada tahap ini, URL diparsing untuk mengekstrak komponen-komponennya seperti *domain*, *subdomain*, *path*, dan *query parameters*. Selanjutnya dilakukan perhitungan fitur-fitur seperti jumlah karakter, rasio digit terhadap karakter, jumlah *subdomain*, keberadaan *IP address* dalam URL, dan fitur-fitur statistik lainnya yang dapat menjadi indikator URL *phishing*.

Tahap berikutnya adalah *Transformasi Data (label encoding)*, yaitu proses konversi data kategorikal menjadi format numerik yang dapat diterima oleh algoritma *machine learning*. Pada tahap ini, label kelas yaitu *phishing* atau *legitimate* diencode menjadi nilai numerik, misalnya 0 untuk *benign* dan 1 untuk *phishing*.

Setelah itu dilakukan *Data Cleaning* (duplikasi), yaitu proses penghapusan data duplikat untuk menghindari bias dalam pembelajaran model atau penghapusan baris.

Setelah seluruh tahapan *preprocessing* selesai, dihasilkan *Preprocessed Dataset* yang telah bersih, terstruktur, dan siap untuk digunakan dalam proses *training* dan *testing* model deteksi *phishing*. Berikut ditampilkan Gambar 4. 5 yang menggambarkan desain proses *data preprocessing*.



Gambar 4. 5 Desain *Data Preprocessing*

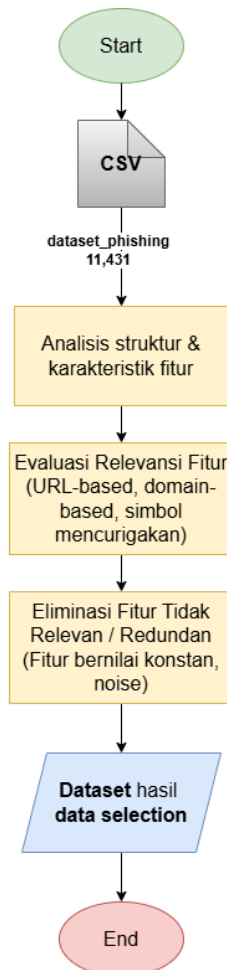
4.3.2.1 Desain *Data Selection*

Desain *Data Selection* pada penelitian ini bertujuan untuk menentukan atribut-atribut yang relevan dalam mendukung proses deteksi website phishing. Tahapan ini dirancang untuk memastikan bahwa hanya fitur-fitur yang memiliki keterkaitan langsung dengan karakteristik phishing yang digunakan pada tahap pengolahan data selanjutnya, sehingga proses analisis menjadi lebih efisien dan terfokus.

Data Selection dilakukan setelah dataset berhasil dimuat dan sebelum tahap pembersihan data, dengan mempertimbangkan struktur serta karakteristik awal dataset. Pada tahap ini, dilakukan identifikasi terhadap seluruh atribut yang tersedia, kemudian dipilih fitur-fitur yang mampu merepresentasikan pola *phishing*, seperti karakteristik URL, struktur domain, serta penggunaan simbol-simbol yang bersifat mencurigakan. Atribut yang tidak memberikan kontribusi signifikan terhadap proses klasifikasi atau bersifat redundan dieliminasi dari dataset.

Hasil dari tahapan *Data Selection* berupa dataset dengan jumlah fitur yang lebih ringkas dan informatif, tanpa mengurangi kemampuan representasi karakteristik

phishing. Dataset hasil seleksi fitur ini selanjutnya digunakan sebagai masukan pada proses *rule-based filtering* serta pada tahap pelatihan model *machine learning*, yaitu *Random Forest* dan *XGBoost*. Desain alur *Data Selection* pada penelitian ini ditampilkan pada Gambar 4. 6.



Gambar 4. 6 Desain *Data Selection*

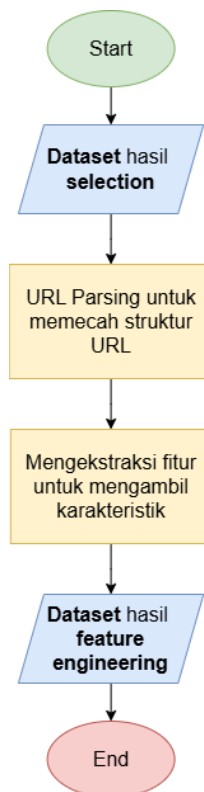
4.3.2.2 Desain *Feature Engineering*

Desain *Feature Engineering* pada penelitian ini bertujuan untuk membentuk dan merepresentasikan karakteristik URL ke dalam bentuk fitur numerik yang dapat digunakan oleh algoritma machine learning. Tahapan ini dilakukan setelah proses *Data Selection*, sehingga hanya atribut terpilih yang diproses lebih lanjut pada tahap ini.

Proses *Feature Engineering* diawali dengan penggunaan dataset hasil Data Selection sebagai masukan utama. Selanjutnya, dilakukan proses *URL parsing* untuk memecah struktur URL ke dalam beberapa komponen utama, seperti protokol, hostname, path, dan parameter. Proses ini bertujuan untuk memudahkan identifikasi pola-pola yang umum ditemukan pada *URL phishing*.

Setelah struktur URL dipisahkan, dilakukan ekstraksi fitur untuk memperoleh karakteristik yang merepresentasikan perilaku *phishing*, seperti panjang URL, jumlah subdomain, penggunaan alamat IP, serta kemunculan simbol-simbol mencurigakan. Fitur-fitur tersebut diekstraksi dalam bentuk numerik agar dapat diproses oleh algoritma *Random Forest* dan *XGBoost*.

Hasil dari tahapan *Feature Engineering* adalah dataset hasil *feature engineering* yang berisi fitur-fitur numerik terstruktur dan informatif, sehingga siap digunakan pada tahap *Data Transformation* dan pemodelan sistem deteksi *phishing*. Desain alur *Feature Engineering* pada penelitian ini ditampilkan pada Gambar 4. 7.



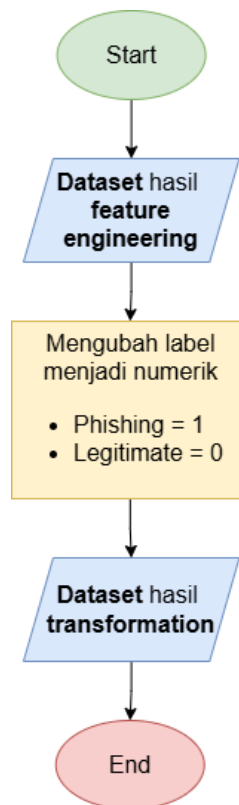
Gambar 4. 7 Desain *Feature Engineering*

4.3.2.3 Desain *Transformation Data*

Desain *Data Transformation* pada penelitian ini bertujuan untuk mengubah format data agar sesuai dengan kebutuhan algoritma *machine learning* yang digunakan. Tahapan ini difokuskan pada penyesuaian representasi data tanpa mengubah informasi dasar yang terkandung di dalam dataset.

Proses *Data Transformation* diawali dengan penggunaan dataset hasil *Data Selection* sebagai masukan. Pada tahap ini dilakukan konversi label kelas dari bentuk kategorikal menjadi bentuk numerik, sehingga dapat diproses oleh algoritma klasifikasi. Label *phishing* dikonversi menjadi nilai 1, sedangkan label *legitimate* dikonversi menjadi nilai 0.

Hasil dari tahapan *Data Transformation* adalah dataset hasil *transformation* yang telah memiliki label numerik dan format data yang sesuai untuk digunakan pada tahap *Data Cleaning* dan proses pemodelan selanjutnya. Desain alur *Data Transformation* pada penelitian ini ditampilkan pada Gambar 4. 8.

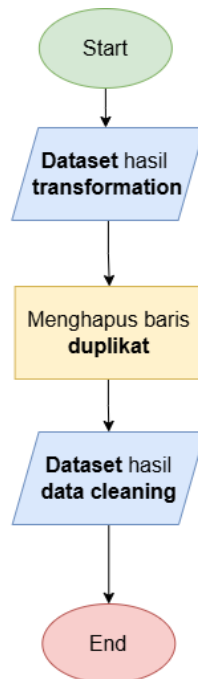


Gambar 4. 8 Desain *Transformation Data*

4.3.2.4 Desain *Data Cleaning*

Pada penelitian ini, proses *data cleaning* dilakukan setelah tahap *transformation data*. Proses *data cleaning* pada penelitian ini hanya difokuskan pada penanganan data duplikat yang terdapat pada dataset.

Desain *data cleaning* ini dilakukan untuk mengidentifikasi dan mengatasi data duplikat yang dapat menyebabkan pengulangan informasi dalam dataset. Dengan menangani data duplikat, dataset menjadi lebih rapi dalam kondisi data yang sebenarnya. Proses ini memastikan bahwa setiap data yang digunakan bersifat unik, sehingga dapat membantu meningkatkan keakuratan proses pelatihan dan evaluasi model klasifikasi. Berikut adalah Gambar 4. 9 dari desain *data cleaning*.



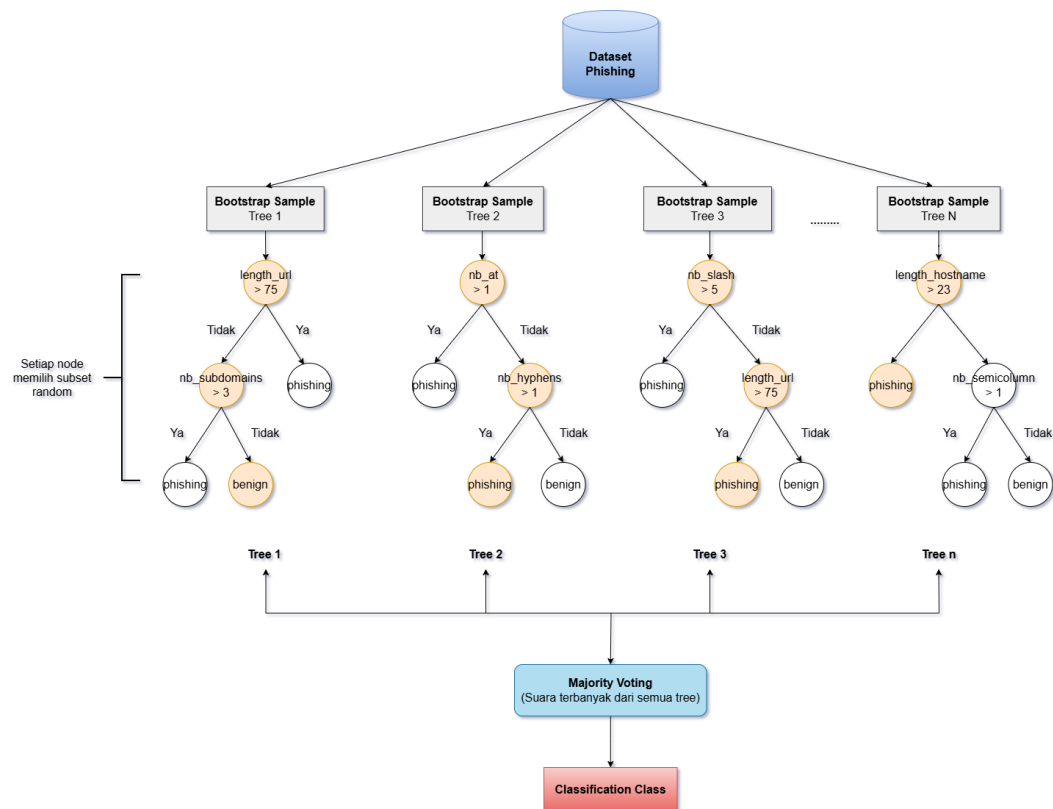
Gambar 4. 9 Desain *Data Cleaning*

4.4 Rancangan *Arsitektur Machine Learning*

Subbab ini merupakan desain dan struktur dari dua model *machine learning* yang akan digunakan, yaitu *Random Forest* dan *XGBoost*. Rancangan arsitektur menggambarkan bagaimana model bekerja, mulai dari menerima data *input*, memproses data, hingga menghasilkan prediksi.

4.4.1 Rancangan Arsitektur *Random Forest*

Desain arsitektur *Random Forest* untuk klasifikasi phishing URL ditunjukkan pada Gambar 4. 10. Arsitektur ini menggambarkan bagaimana model *Random Forest* memanfaatkan teknik *ensemble learning* dengan membangun banyak *decision tree*. Proses klasifikasi diawali dengan penggunaan dataset *phishing* yang telah melalui tahap *preparation* dan *preprocessing*, sehingga setiap URL direpresentasikan dalam bentuk fitur-fitur *numerik* yang relevan untuk proses pembelajaran model.



Gambar 4. 10 Rancangan Arsitektur *Random Forest*

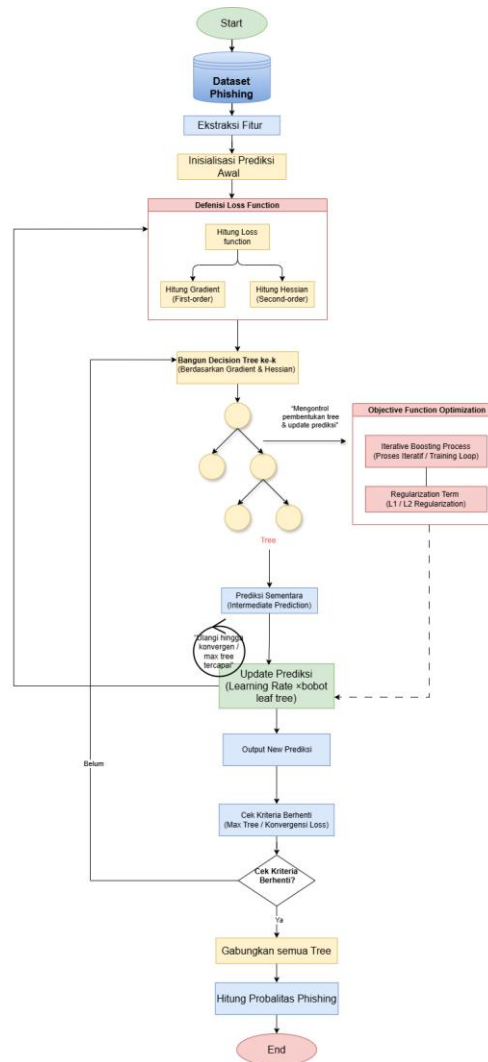
1. Dataset yang telah diproses kemudian dibagi menjadi beberapa subset data menggunakan teknik *bootstrap sampling*, yaitu pengambilan sampel secara acak dengan pengembalian dari dataset asli. Setiap subset *bootstrap* digunakan untuk membangun satu decision tree yang bersifat *independen*. Dengan mekanisme ini, terbentuk sejumlah *tree* seperti *Tree 1*, *Tree 2*, *Tree 3*, hingga *Tree N*, sesuai dengan jumlah estimator yang ditentukan pada model *Random Forest*.

Setelah tahap *bootstrap sampling*, setiap *decision tree* dibangun secara paralel dengan menerapkan pemilihan subset fitur secara acak pada setiap node. Fitur-fitur yang digunakan dalam proses pemisahan meliputi *length_url*, *nb_at*, *nb_slash*, *length_hostname*, *nb_subdomains*, *nb_hyphens*, *nb_semicolumn*, serta fitur-fitur lain yang telah melalui proses *feature selection*. Pada setiap node internal, algoritma *Random Forest* akan memilih fitur terbaik dari subset fitur acak tersebut berdasarkan kriteria *Gini* atau *entropy* untuk memisahkan data. Proses pemisahan ini berlangsung sehingga mencapai kondisi berhenti, seperti batas kedalaman maksimum *tree* atau jumlah minimum sampel pada *leaf node*, yang kemudian menghasilkan keputusan klasifikasi berupa *phishing* atau *benign*.

Setelah seluruh *decision tree* menghasilkan prediksi masing-masing, hasil tersebut digabungkan menggunakan mekanisme *majority voting*. Pada tahap ini, setiap *tree* memberikan satu suara terhadap kelas tertentu, dan kelas dengan jumlah suara terbanyak dipilih sebagai hasil klasifikasi akhir. Pendekatan *ensemble* ini membuat *Random Forest* memiliki tingkat akurasi dan kestabilan yang lebih baik, karena mampu mengurangi *overfitting* serta meningkatkan kemampuan generalisasi model terhadap data baru yang belum pernah dilihat sebelumnya.

4.4.2 Rancangan Arsitektur XGBoost

Desain arsitektur *XGBoost* untuk klasifikasi phishing URL ditunjukkan pada Gambar 4. 11. Arsitektur ini menggambarkan bagaimana model *XGBoost* memanfaatkan teknik *gradient boosting* dengan membangun *decision tree* secara sekuensial untuk meningkatkan akurasi prediksi. Proses klasifikasi diawali dengan penggunaan dataset *phishing* yang telah melalui tahap *preparation* dan *preprocessing*, sehingga setiap URL direpresentasikan dalam bentuk fitur-fitur *numerik* yang relevan untuk proses pembelajaran model.



Gambar 4. 11 Rancangan Arsitektur *XGBoost*

Proses deteksi phishing diawali dengan tahap *Load Dataset*, di mana kumpulan data yang berisi sampel URL atau aktivitas jaringan yang telah memiliki label (*phishing* atau *benign*) dimuat ke dalam sistem. Setelah data tersedia, dilakukan tahap ekstraksi fitur untuk mengubah informasi mentah menjadi variabel numerik yang dapat diproses oleh mesin. Langkah selanjutnya adalah melakukan inisialisasi prediksi awal, di mana model menetapkan nilai dasar atau nilai prediksi yang sama mewakili label yang paling dominan untuk seluruh data. Tahapan ini menjadi titik awal yang digunakan dalam perhitungan selanjutnya.

Setelah inisialisasi, sistem masuk ke tahap inti algoritma *XGBoost*, yaitu penghitungan *Gradient* dan Residual. Pada tahap ini, *Gradient* menunjukkan arah

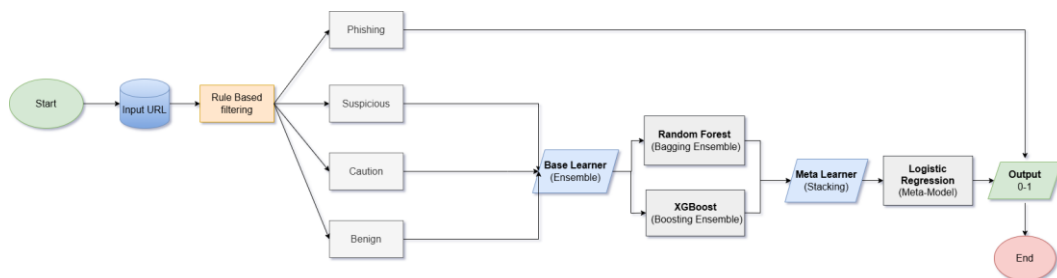
dan ukuran perubahan yang diperlukan untuk mengurangi kesalahan tersebut. Residual mengukur selisih antara nilai aktual dengan prediksi saat ini untuk menemukan tingkat kesalahan yang perlu diperbaiki. Kesalahan tersebut kemudian digunakan sebagai landasan untuk tahap *Build Tree* atau pembangunan pohon keputusan. Pohon ini dibangun secara sekuensial atau berurutan untuk mengoreksi kesalahan (residual). Pada setiap langkah, pohon baru ditambahkan untuk meminimalkan kesalahan residual. Selama proses ini, diterapkan pula *L1/L2 Regularization* untuk menjaga kompleksitas pohon agar model tidak mengalami *overfitting* dan tetap mampu menggeneralisasi data baru dengan baik.

Setiap hasil dari pembangunan pohon baru tidak langsung ditambahkan secara utuh, melainkan dikalikan terlebih dahulu dengan *Learning Rate* pada tahap *update* prediksi. Hal ini dilakukan agar proses belajar model berjalan secara bertahap dan lebih stabil. Melalui proses iteratif, langkah-langkah penghitungan residual dan pembangunan pohon diulang hingga mencapai jumlah pohon yang ditentukan atau tingkat kesalahan yang minimum. Pada tahap akhir, sistem akan menggabungkan semua *tree* yang telah terbentuk menjadi satu model prediksi yang komprehensif. Hasil penggabungan ini kemudian dikonversi melalui tahap hitung probabilitas *phishing* untuk menghasilkan nilai akhir yang menentukan apakah suatu data termasuk kategori *phishing* atau *benign*.

4.5 Rancangan Arsitektur *Rule based Filtering*

Sistem deteksi URL *phishing* pada penelitian ini dirancang menggunakan pendekatan hybrid yang mengombinasikan *rule-based filtering* dan *machine learning*. Proses dimulai dari input URL yang kemudian diekstraksi fitur-fiturnya untuk merepresentasikan karakteristik struktural dan pola URL dalam bentuk *numerik*. Fitur-fitur tersebut selanjutnya dievaluasi pada tahap *rule-based filtering* dengan menerapkan aturan logis tertentu yang merepresentasikan indikator umum *phishing*. Berdasarkan hasil evaluasi ini, URL dapat langsung diklasifikasikan sebagai *phishing* atau *benign* apabila memenuhi kriteria yang jelas, sedangkan URL yang tidak dapat ditentukan secara pasti akan dikategorikan sebagai *suspicious*. URL yang termasuk dalam kategori *suspicious* dan *benign* kemudian diproses lebih

lanjut menggunakan pendekatan *machine learning* berbasis *ensemble*. Pada tahap ini digunakan dua *base learner*, yaitu *Random Forest* dengan pendekatan *bagging* dan *XGBoost* dengan pendekatan *boosting*, untuk menangkap pola kompleks yang tidak dapat diidentifikasi oleh aturan sederhana. Hasil prediksi dari kedua model tersebut selanjutnya digabungkan melalui metode *stacking* dengan *Logistic Regression* sebagai *meta-learner* untuk menghasilkan keputusan akhir berupa klasifikasi URL *phishing* atau *benign*. Dengan arsitektur ini, *rule-based filtering* berperan sebagai penyaring awal yang efektif dalam mengurangi beban komputasi sekaligus meningkatkan efisiensi dan akurasi sistem deteksi URL *phishing*. Berikut adalah Gambar 4. 12 dari Rancangan Arsitektur *Rule based Filtering*.



Gambar 4. 12 Rancangan Arsitektur *Rule based Filtering*

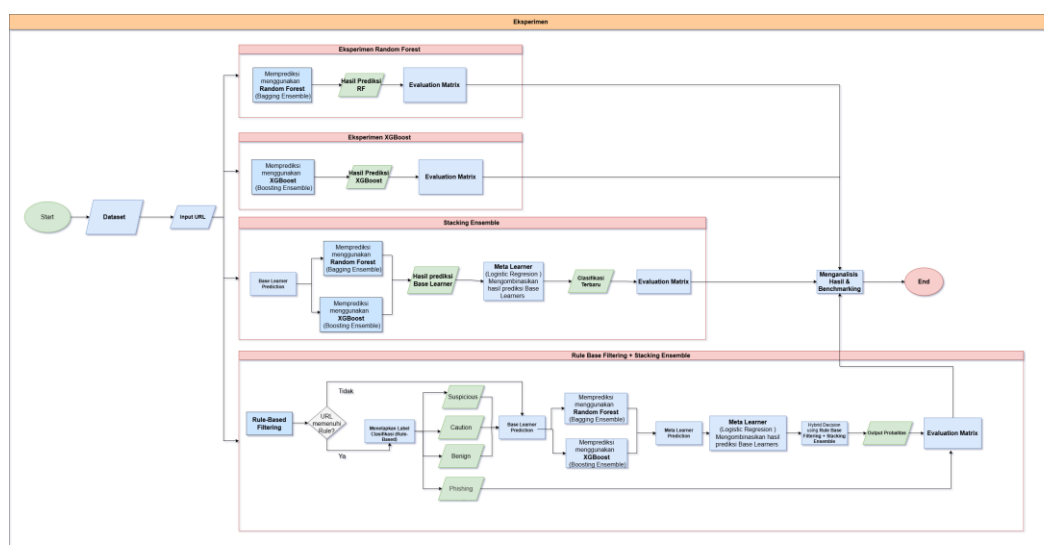
4.6 Rancangan Arsitektur Eksperimen

Rancangan arsitektur eksperimen pada Gambar 4. 13 disusun untuk mengevaluasi kinerja berbagai pendekatan dalam mendeteksi URL phishing. Proses eksperimen diawali dengan penggunaan dataset yang telah melalui tahapan data preparation, data preprocessing, dan feature extraction, sehingga data berada dalam kondisi siap digunakan pada seluruh skenario pengujian. Tahapan awal ini bertujuan untuk memastikan bahwa setiap skenario eksperimen menggunakan data dengan struktur dan karakteristik yang sama, sehingga hasil evaluasi yang diperoleh dapat dibandingkan secara adil dan objektif.

Eksperimen dilakukan melalui empat skenario pengujian. Skenario pertama menggunakan model Random Forest sebagai base learner dengan pendekatan bagging ensemble. Skenario kedua menggunakan model XGBoost sebagai base learner dengan pendekatan *boosting ensemble*. Skenario ketiga menerapkan metode

stacking ensemble, di mana Random Forest dan XGBoost berperan sebagai base learner, sedangkan Logistic Regression digunakan sebagai meta-learner untuk mengombinasikan hasil prediksi dari kedua model tersebut. Skenario keempat mengombinasikan rule-based filtering dengan metode stacking ensemble, di mana URL yang memenuhi aturan tertentu akan diklasifikasikan secara langsung oleh sistem berbasis aturan, sedangkan URL yang tidak memenuhi aturan tersebut diproses lebih lanjut menggunakan metode stacking ensemble.

Pada seluruh skenario eksperimen, setiap URL akan menghasilkan satu label prediksi akhir sesuai dengan pendekatan yang digunakan pada masing-masing skenario. Khusus pada skenario keempat, label prediksi akhir diperoleh dari dua sumber, yaitu hasil klasifikasi *rule-based filtering* untuk URL yang memenuhi aturan dan hasil prediksi *stacking ensemble* untuk URL yang tidak memenuhi aturan. Seluruh label prediksi akhir tersebut kemudian dibandingkan dengan label asli pada dataset untuk membentuk *confusion matrix*. Berdasarkan confusion matrix tersebut, dilakukan perhitungan metrik evaluasi berupa *accuracy*, *precision*, *recall*, dan *F1-score* menggunakan rumus yang telah dijelaskan pada Subbab 2.8. Hasil perhitungan metrik evaluasi ini selanjutnya digunakan untuk menganalisis serta membandingkan efektivitas masing-masing pendekatan dalam mendeteksi URL phishing.



Gambar 4.13 Desain Eksperimen

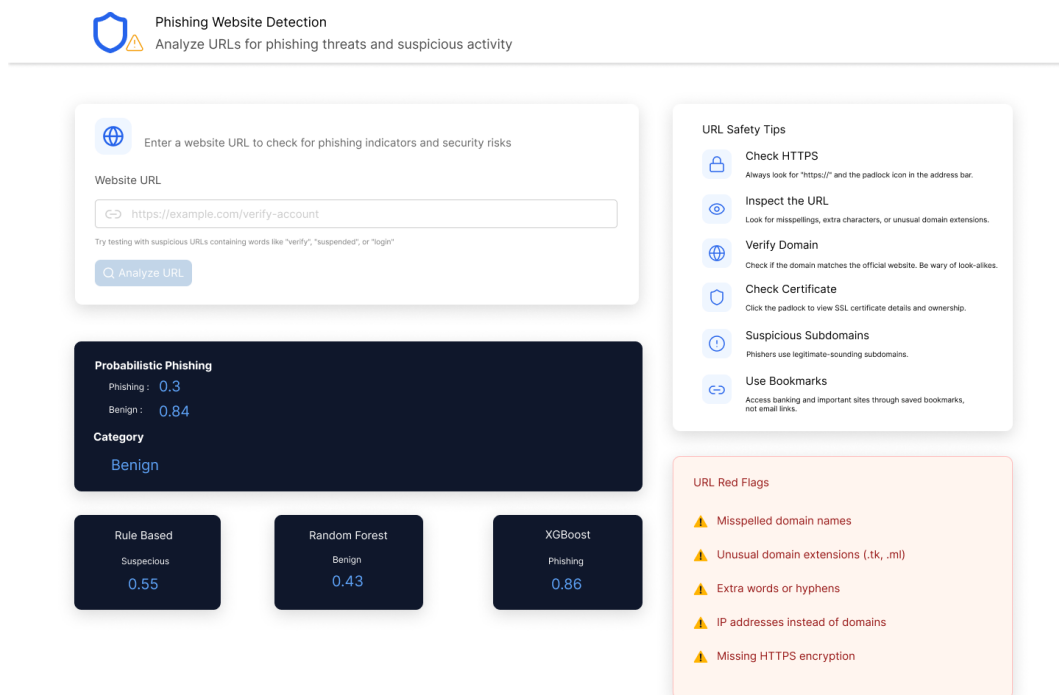
4.7 Mockup Desain Sistem

Mockup desain sistem pada Gambar 4. 14 merupakan rancangan antarmuka pengguna (*user interface*) dari sistem deteksi *phishing* berbasis URL yang dikembangkan pada penelitian ini. *Mockup* ini dibuat untuk memberikan gambaran visual mengenai alur interaksi pengguna dengan sistem, mulai dari proses memasukkan URL hingga menampilkan hasil analisis dan klasifikasi *phishing* secara informatif.

Pada rancangan *mockup*, pengguna dapat memasukkan URL yang ingin dianalisis melalui kolom input yang disediakan. Setelah proses analisis dilakukan, sistem menampilkan hasil klasifikasi URL dalam bentuk label prediksi, seperti *legitimate*, *suspicious*, atau *phishing*. Selain hasil klasifikasi utama, sistem juga menampilkan informasi pendukung berupa skor risiko *phishing*, kategori tingkat risiko, serta ringkasan fitur atau aturan rule-based yang terdeteksi pada URL tersebut.

Informasi statistik yang ditampilkan pada sistem diperoleh dari hasil pemrosesan internal, yaitu berdasarkan distribusi hasil klasifikasi URL dan skor risiko yang dihasilkan oleh rule-based filtering maupun model machine learning. Statistik ini bertujuan untuk memberikan gambaran tambahan kepada pengguna mengenai tingkat risiko URL yang dianalisis, tanpa bergantung pada sumber eksternal atau data *real-time*.

Mockup dirancang dengan mempertimbangkan aspek *usability* dan keterbacaan informasi, sehingga hasil analisis dapat dipahami dengan mudah oleh pengguna. Dengan adanya desain *mockup* ini, diharapkan sistem yang dibangun tidak hanya berfungsi secara teknis, tetapi juga mampu menyajikan hasil deteksi *phishing* secara jelas bagi pengguna.



Gambar 4. 14 Mockup Desain Sistem

Penghitungan probabilitas pada sistem deteksi website phishing dilakukan berdasarkan keluaran dari tiga pendekatan, yaitu *Rule-Based Filtering*, *Random Forest*, dan *XGBoost*. Setiap metode menghasilkan nilai probabilitas dalam rentang 0 hingga 1, di mana nilai yang semakin mendekati 1 menunjukkan risiko *phishing* yang semakin tinggi, sedangkan nilai yang mendekati 0 menunjukkan URL cenderung benign.

BAB V

IMPLEMENTASI

Pada bab ini dijelaskan bentuk fisik dari hasil analisis yang telah dijelaskan pada bab analisis dan bab desain. Hal-hal yang akan dijelaskan pada bab ini adalah lingkungan implementasi, batasan implementasi, implementasi eksperimen dan implementasi evaluasi.

5.1 Lingkungan Implementasi

Lingkungan implementasi terdiri dari perangkat keras (*hardware*) dan perangkat lunak (*software*) yang digunakan dalam proses implementasi. Berikut ini sub bab spesifikasi dari perangkat keras dan perangkat lunak yang digunakan dalam proses implementasi penelitian ini.

5.1.1 Perangkat Keras (*Hardware*)

Perangkat keras yang digunakan untuk melakukan proses implementasi memiliki spesifikasi pada Tabel 5. 1 sebagai berikut.

Tabel 5. 1 Perangkat keras (Hardware)

No	Hardware	Specifications
1.	Processor	Intel core i5
2.	RAM (Random Access Memory)	8 GB

5.1.2 Perangkat Lunak (*Software*)

Berikut ini spesifikasi dari *software* yang diperlukan dalam melakukan penelitian ini dapat dilihat pada Tabel 5. 2 berikut.

Tabel 5. 2 Perangkat Lunak (Software)

No	Software	Specifications
1.	Operating System	Windows 11
2.	Programming Language	Python
3.	Browser	Google Chrome
4.	Text Editor	Visual Studio Code

5.2 Batasan Implementasi

Adapun batasan yang ditentukan dalam implementasi penelitian ini adalah:

- a. Implementasi yang dilakukan menggunakan bahasa pemrograman Python.
- b. Implementasi setiap tahapan penelitian dilakukan di Visual Studio Code.

5.3 Implementasi Analisis Dataset

Pada implementasi ini, dataset Kaggle berformat CSV (*Comma-Separated Values*) yang telah diperoleh dari repositori publik dimuat lalu dianalisis menggunakan bahasa pemrograman Python melalui *Visual Studio Code*.

5.3.1 Implementasi *Data Preparation*

Implementasi *data preparation* dilakukan untuk mempersiapkan dataset agar siap digunakan dalam proses pemodelan melalui beberapa langkah seperti pembacaan data, pemeriksaan kualitas data, pengecekan duplikat, dan analisis distribusi label.

5.3.1.1 Pembacaan Dataset

Implementasi awal implementasi dimulai dengan membaca dataset agar dapat dikenali sehingga memudahkan proses eksplorasi, pemeriksaan kualitas data, dan persiapan sebelum memasuki tahap pemodelan yang lebih kompleks.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from urllib.parse import urlparse
from collections import Counter

file_dataset = r"D:\TUGAS AKHIR\phishing
detector\dataset\dataset_phishing.csv"

df = pd.read_csv(file_dataset)
```

Kode 5. 1 Membaca dataset

Pada Kode 5. 1 dilakukan *import library*. Selanjutnya, *path* file dataset didefinisikan dan fungsi *pd.read_csv()* digunakan untuk membaca file CSV.

5.3.1.2 Jumlah baris dan kolom

Implementasi analisis dataset bertujuan untuk memahami struktur data serta mengidentifikasi jumlah baris dan kolom yang terdapat dalam dataset.

```
print(f"Dataset memiliki {df.shape[0]} baris dan {df.shape[1]}  
kolom.\n")
```

Kode 5. 2 Menampilkan jumlah baris dan kolom

Pada Kode 5. 2 menganalisis tipe data pada setiap kolom dalam dataset untuk mengetahui struktur data yang digunakan. Fungsi *df.shape* digunakan untuk menampilkan dimensi dataset yaitu jumlah baris dan kolom.

5.3.1.3 Pengecekan *Missing Values*

Implementasi *missing value* digunakan untuk menangani data yang kosong atau tidak terisi pada dataset agar data dapat digunakan dengan baik dalam proses analisis atau pemodelan.

```
missing = df.isnull().sum()  
if missing.sum() == 0:  
    print("Tidak ada data yang kosong di seluruh 89 kolom.")  
else:  
    print("Kolom dengan data kosong:")  
    print(missing[missing > 0])
```

Kode 5. 3 Implementasi Pengecekan Missing Values

Pada Kode 5. 3 fungsi *df.isnull().sum()* digunakan untuk menghitung jumlah nilai yang hilang pada setiap kolom. Jika tidak ditemukan *missing values*, maka akan menampilkan pesan konfirmasi. Sebaliknya, jika terdapat *missing values*, maka kolom-kolom yang mengandung nilai kosong akan ditampilkan beserta jumlahnya.

5.3.1.4 Pengecekan Duplikat

Pengecekan data duplikat dilakukan untuk memastikan tidak terdapat URL yang muncul lebih dari satu kali dalam dataset karena dapat mempengaruhi proses *training* model.

```
nama_kolom = 'url'
df_duplikat_url = df[df[nama_kolom].duplicated(keep=False)]

if len(df_duplikat_url) > 0:
    print(df_duplikat_url[nama_kolom].value_counts())

    display(df_duplikat_url.head(5))
else:
    print("\nSelamat! Tidak ada baris duplikat pada kolom URL.")
```

Kode 5. 4 Implementasi Pengecekan Data Duplikat

Pada Kode 5. 4 dilakukan pengecekan data duplikat pada kolom URL menggunakan *fungsi duplicated()* dengan parameter *keep=False* yang berarti semua baris duplikat akan ditandai. Jika ditemukan data duplikat, maka akan menampilkan daftar URL yang duplikat beserta jumlah kemunculannya dan menampilkan pratinjau data tersebut.

5.3.1.5 Distribusi Label Status

Distribusi label status dianalisis untuk melihat keseimbangan data antara kelas *phishing* dan *legitimate* dalam dataset sebelum proses pemodelan.

```
target_col = 'status'
if target_col in df.columns:
    print(df[target_col].value_counts())
    plt.show()
else:
    print(f"\nKolom '{target_col}' tidak ditemukan. Cek kembali nama kolom target.")
```

Kode 5. 5 Implementasi Analisis Distribusi Kelas Target

Pada Kode 5. 5 dilakukan analisis terhadap kolom target yang bernama status. Fungsi `value_counts()` digunakan untuk menghitung jumlah sampel pada setiap kelas. Hasil analisis divisualisasikan menggunakan `countplot` dari untuk memberikan gambaran visual yang lebih jelas. Visualisasi ini membantu dalam mengidentifikasi apakah terdapat *class imbalance* yang perlu ditangani.

5.3.1.6 Korelasi Antar Fitur

Implementasi korelasi antar fitur dilakukan untuk mengetahui hubungan antar variabel numerik dalam dataset. Korelasi ini membantu dalam memahami keterkaitan antar fitur serta mengidentifikasi fitur yang memiliki hubungan kuat dengan fitur lainnya.

```
numeric_df = df.select_dtypes(include=['number'])  
plt.title('Korelasi Antar Fitur')
```

Kode 5. 6 Implementasi korelasi antar fitur

Pada Kode 5. 6 dataset difilter untuk mengambil kolom yang memiliki tipe data numerik menggunakan fungsi `select_dtypes(include=['number'])`, karena perhitungan korelasi hanya dapat dilakukan pada data numerik.

5.3.2 Implementasi Data Preprocessing

Implementasi *preprocessing* merupakan proses transformasi data mentah menjadi data yang siap digunakan untuk pelatihan model *machine learning*. Tahap ini mencakup beberapa sub-tahapan yaitu *data selection*, *feature engineering*, *data transformation*, dan *data cleaning*.

5.3.2.1 Data Selection

Implementasi *data selection* bertujuan untuk memilih fitur yang relevan, sehingga beberapa kolom dihapus berdasarkan analisis korelasi dan relevansi fitur karena tidak memberikan kontribusi signifikan. Berikut implementasi kode pada tahap *data selection*.

```
kolom_yang_dihapus = [
```

```

        'nb_or',
        'ratio_nullHyperlinks',
        'ratio_intRedirection',
        'ratio_intErrors',
        'submit_email',
        'sfh'
    ]

    df_selected = df.drop(columns=kolom_yang_dihapus,
                          errors='ignore')

    output_path = r'D:\TUGAS AKHIR\phishing
    detector\processed\data_selection.csv'
    df_selected.to_csv(output_path, index=False)

```

Kode 5. 7 Implementasi data selection

Pada Kode 5. 7 dilakukan penghapusan 6 kolom yang dianggap tidak relevan atau memiliki korelasi rendah dengan target klasifikasi, yaitu *nb_or*, *ratio_nullHyperlinks*, *ratio_intRedirection*, *ratio_intErrors*, *submit_email*, dan *sfh*, karena tidak memiliki nilai pembeda yang berguna bagi model dalam mengenali pola antara kelas *phishing* dan *non-phishing*. Fungsi *drop()* dengan parameter *errors='ignore'* digunakan untuk menghapus kolom tersebut, dan hasil seleksi kemudian disimpan dalam file CSV baru untuk tahap selanjutnya.

5.3.2.2 Feature Engineering

Implementasi *feature engineering* bertujuan untuk menghasilkan fitur baru dari data mentah agar membantu model mengenali pola URL *phishing* dan *non-phishing*. Pada implementasi ini, fitur diekstraksi dari URL menggunakan teknik *parsing* untuk memecah URL menjadi komponen seperti protokol, domain, subdomain, dan *path*. Berikut implementasi kode pada tahap *feature engineering*.

```

# A. FITUR URL & PANJANG
new_row['length_url'] = url
new_row['length_hostname'] = domain_part

# B. FITUR SIMBOL

```

```

        cols_symbol = {
            'nb_dots': '.', 'nb_hyphens': '-', 'nb_at': '@',
            'nb_qm': '?',
            'nb_and': '&', 'nb_eq': '=', 'nb_underscore': '_',
            'nb_percent': '%',
            'nb_slash': '/', 'nb_star': '*', 'nb_colon': ':',
            'nb_comma': ',',
            'nb_semicolumn': ';', 'nb_dollar': '$', 'nb_space':
            '%20',
            'nb_www': 'www', 'nb_com': 'com', 'nb_dslash': "//",
            'nb_tilde': "~"
        }
        for col, sym in cols_symbol.items():
            if col in new_row: new_row[col] = get_symbol_str(url,
sym)

# C. FITUR BOOLEAN / TOKEN
new_row['ip'] = "IP Address" if
re.match(r'\d+\.\d+\.\d+\.\d+', domain_part) else "Domain Name"
new_row['http_in_path'] = "http (path)" if "http" in
path_part else 0
new_row['https_token'] = "HTTPS" if parsed.scheme == 'https'
else "HTTP"
new_row['punycode'] = "Punycode" if "xn--" in domain_part
else "Standard"
new_row['port'] = f"Port {parsed.port}" if parsed.port else
"Default"
new_row['tld_in_path'] = f".{suffix} (path)" if suffix in
path_part and suffix else 0
new_row['tld_in_subdomain'] = f".{suffix} (sub)" if suffix
in subdomain and suffix else 0
new_row['abnormal_subdomain'] = "Abnormal" if
re.search(r'(http|www)', subdomain) else "Normal"
new_row['prefix_suffix'] = "Hyphen-Domain" if '-' in
domain_part else "Clean"
new_row['random_domain'] = "Random Hex" if re.search(r'[0-
9a-f]{5,}', domain_part) else 0

```

```

        new_row['shortening_service'] = "Shortener" if
re.search(r'bit\.ly|goo\.gl|shorte\.st|tinyurl', domain_part)
else 0

        if re.search(r'\.(txt|exe|jar|js|php|html)$', path_part):
            new_row['path_extension'] =
re.search(r'\.(txt|exe|jar|js|php|html)$', path_part).group(0)
        else:
            new_row['path_extension'] = 0

# D. FITUR KATA
        new_row['shortest_word_host'] = get_word_str(domain_part,
'shortest')
        new_row['longest_word_host'] = get_word_str(domain_part,
'longest')
        new_row['shortest_word_path'] = get_word_str(path_part,
'shortest')
        new_row['longest_word_path'] = get_word_str(path_part,
'longest')
        new_row['shortest_words_raw'] = get_word_str(url,
'shortest')
        new_row['longest_words_raw'] = get_word_str(url, 'longest')

# E. FITUR KHUSUS
        new_row['length_words_raw'] = get_words_from_url(url)
        new_row['char_repeat'] = get_most_repeated_char(url)

        new_row['nb_redirection'] = "Direct Link" if
row['nb_redirection'] == 0 else "Redirects Found"
        new_row['nb_external_redirection'] = "Internal/None" if
row['nb_external_redirection'] == 0 else "External Jump"
        new_row['ratio_extRedirection'] =
f"{int(row['ratio_extRedirection']*100)}%"

        # Menambahkan pengecekan jika kolom ratio_intErrors ada di
row sebelum diubah
        if 'ratio_intErrors' in row:
            new_row['ratio_intErrors'] =
f"{int(row['ratio_intErrors']*100)}%"

```

```

        new_row['nb_hyperlinks'] = "Contains Links" if
row['nb_hyperlinks'] > 0 else "No Links"
        new_row['ratio_intHyperlinks'] =
f"{int(row['ratio_intHyperlinks']*100)}%"
        new_row['ratio_extHyperlinks'] =
f"{int(row['ratio_extHyperlinks']*100)}%"
        new_row['nb_extCSS'] = "External CSS" if row['nb_extCSS'] >
0 else "Internal/None"

        new_row['domain_registration_length'] =
f"{int(row['domain_registration_length'])} days"
        new_row['domain_age'] = f"{int(row['domain_age'])} days"

        # F. AVERAGE & RATIOS
        for col in ['avg_words_raw', 'avg_word_host',
'avg_word_path']:
            if col in new_row: new_row[col] =
f"{float(row[col]):.1f} avg"

        for col in ['ratio_digits_url', 'ratio_digits_host',
'links_in_tags', 'ratio_intMedia', 'ratio_extMedia',
'safe_anchor']:
            if col in new_row: new_row[col] =
f"{float(row[col])*100:.0f}%"

        # G. HINTS & BRANDS
        hints = [h for h in PHISH_HINTS if h in url.lower()]
        new_row['phish_hints'] = "|".join(hints) if hints else 0

        brands_dom = [b for b in BRANDS if b in full_domain]
        new_row['domain_in_brand'] = "|".join(brands_dom) if
brands_dom else 0
        brands_sub = [b for b in BRANDS if b in subdomain]
        new_row['brand_in_subdomain'] = "|".join(brands_sub) if
brands_sub else 0
        brands_path = [b for b in BRANDS if b in path_part]
        new_row['brand_in_path'] = "|".join(brands_path) if
brands_path else 0

```

```

        new_row['suspicious_tld'] = f".{suffix}" if suffix in
SUSPICIOUS_TLDS else 0

    # H. SISA FITUR
    if 'nb_subdomains' in new_row: new_row['nb_subdomains'] =
f"{int(row['nb_subdomains'])} subdomains"

    bool_cols = ['login_form', 'external_favicon', 'iframe',
'popup_window', 'onmouseover',
                'right_click', 'empty_title', 'domain_in_title',
'domain_with_copyright',
                'whois_registered_domain', 'dns_record',
'google_index']
    for col in bool_cols:
        if col in new_row: new_row[col] = "Found" if row[col] ==
1 else 0

    if 'web_traffic' in new_row: new_row['web_traffic'] =
f"{row['web_traffic']}"
    if 'page_rank' in new_row: new_row['page_rank'] =
f"{row['page_rank']}"
    if 'statistical_report' in new_row:
new_row['statistical_report'] = "Reported" if
row['statistical_report'] == 1 else "Clean"

    return new_row

```

Pada Kode 5.X, proses ekstraksi fitur dilakukan secara terstruktur yang terdiri dari delapan bagian, yaitu:

- A (Fitur URL & Panjang) mengekstraksi panjang URL dan *hostname* menggunakan variabel *url* dan *domain_part*.
- B (Fitur Simbol) menghitung kemunculan 19 simbol khusus dalam URL menggunakan fungsi *get_symbol_str(url, sym)* yang dipanggil melalui *dictionary cols_symbol*.

- C (Fitur *Boolean*/Token) mendeteksi karakteristik mencurigakan pada URL seperti alamat IP dan *punycode* menggunakan fungsi *re.match()* dan *re.search()*, sedangkan pengecekan token seperti *http* pada *path* dilakukan menggunakan operator *in*.
- D (Fitur Kata) mengekstraksi kata terpendek dan terpanjang dari *hostname* dan *path* menggunakan fungsi *get_word_str()*.
- E (Fitur Khusus) menghitung jumlah kata menggunakan fungsi *get_words_from_url()* dan mendeteksi karakter yang paling sering muncul menggunakan fungsi *get_most_repeated_char()*.
- F (*Average & Ratios*) mengonversi nilai rata-rata dan rasio ke dalam format persentase menggunakan *f-string*.
- G (*Hints & Brands*) mendeteksi kata kunci *phishing* dan merek terkenal pada URL dengan mencocokkan terhadap daftar *PHISH_HINTS*, *BRANDS*, dan *SUSPICIOUS_TLDS*.
- H (Sisa Fitur) memproses fitur *boolean* elemen HTML dan reputasi domain secara iteratif menggunakan daftar *bool_cols*.

Dataset yang telah melalui tahap *parsing* kemudian disimpan dalam file CSV baru untuk tahap *transformation* pada label.

5.3.2.3 Data Transformation

Tahap *data transformation* bertujuan mengubah data ke dalam format yang sesuai untuk proses pemodelan. Pada implementasi ini, transformasi yang dilakukan adalah mengubah label kelas dari format kategorikal (*phishing* dan *legitimate*) menjadi format numerik (1 dan 0).

```
if df_final['status'].dtype == 'object':
    df_final['status'] = df_final['status'].map({'phishing': 1,
    'legitimate': 0})
```

Kode 5. 8 Implementasi *data transformation*

Pada Kode 5.4, dilakukan transformasi label target menggunakan fungsi *map()* yang memetakan nilai *phishing* menjadi 1 dan *legitimate* menjadi 0. Transformasi ini diperlukan karena algoritma *machine learning* seperti *Random Forest* dan *XGBoost* memerlukan input numerik. Setelah proses *transformation* selesai, dataset kemudian disimpan dalam file CSV untuk tahap *cleaning*.

5.3.2.4 Data Cleaning

Implementasi *data cleaning* merupakan tahap akhir dari *preprocessing* ini yang bertujuan untuk membersihkan data dari duplikasi. Dalam penelitian ini, pembersihan data difokuskan pada penghapusan URL duplikat. Berikut adalah implementasi kode untuk *data cleaning*.

```
if jumlah_duplikat > 0:
    df_final = df_final.drop_duplicates(subset=['url'],
    keep='first').reset_index(drop=True)
    print("--- Duplikat berhasil dihapus ---")
else:
    print("--- Tidak ada duplikat yang perlu dihapus ---")
```

Kode 5. 9 Implementasi korelasi antar fitur

Pada Kode 5.5, dilakukan penghapusan data duplikat berdasarkan kolom URL menggunakan fungsi *drop_duplicates()* dengan parameter *subset=['url']* untuk menentukan kolom yang dijadikan acuan pengecekan duplikasi. Parameter *keep='first'* digunakan untuk mempertahankan kemunculan pertama dan menghapus duplikasi selanjutnya. Fungsi *reset_index(drop=True)* digunakan untuk mengatur ulang indeks setelah penghapusan. Dataset yang telah dibersihkan kemudian disimpan dalam file CSV baru untuk tahap pemodelan.

5.4 Implementasi Model

5.4.1 Eksperimen *Random Forest*

5.4.2 Eksperimen *XGBoost*

5.4.3 Eksperimen *Stacking*

5.4.4 Eksperimen *Rule based Filtering* dengan *Stacking*

5.5 Implementasi Eksperimen

5.6 Implementasi Evaluasi

DAFTAR PUSTAKA

- [1] L. A. Febrika Ardy, I. Istiqomah, A. E. Ezer, and S. N. Neyman, “Phishing di Era Media Sosial: Identifikasi dan Pencegahan Ancaman di Platform Sosial,” *J. Internet Softw. Eng.*, vol. 1, no. 4, p. 11, 2024, doi: 10.47134/pjise.v1i4.2753.
- [2] E. Mouncey and S. Ciobotaru, “Phishing scams on social media: An evaluation of cyber awareness education on impact and effectiveness,” *J. Econ. Criminol.*, vol. 7, no. December 2024, p. 100125, 2025, doi: 10.1016/j.jeconc.2025.100125.
- [3] R. Zieni, L. Massari, and M. C. Calzarossa, “Phishing or Not Phishing? A Survey on the Detection of Phishing Websites,” *IEEE Access*, vol. 11, no. January, pp. 18499–18519, 2023, doi: 10.1109/ACCESS.2023.3247135.
- [4] F. P. E. Putra, U. Ubaidi, A. Zulfikri, G. Arifin, and R. M. Ilhamsyah, “Analysis of Phishing Attack Trends, Impacts and Prevention Methods: Literature Study,” *Brill. Res. Artif. Intell.*, vol. 4, no. 1, pp. 413–421, 2024, doi: 10.47709/brilliance.v4i1.4357.
- [5] A. January-march, “1 Quarter,” no. July, pp. 1–13, 2025.
- [6] A. Khan, D. M. Ahmed, and A. Fathima, “Enhanced Phishing Detection Using Machine Learning Algorithms: A Comparative Study of Random Forest, SVM, and Logistic Regression Models,” *SSRN Electron. J.*, 2025, doi: 10.2139/ssrn.5191566.
- [7] V. A. Windarni, A. F. Nugraha, S. T. A. Ramadhani, D. A. Istiqomah, F. M. Puri, and A. Setiawan, “Deteksi Website Phishing Menggunakan Teknik Filter Pada Model Machine Learning,” *Inf. Syst. J.*, vol. 6, no. 01, pp. 69–80, 2023, doi: 10.24076/infosjournal.2023v6i01.1268.
- [8] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab, “Machine Learning for Anomaly Detection: A Systematic Review,” *IEEE Access*, vol. 9, pp. 78658–78700, 2021, doi: 10.1109/ACCESS.2021.3083060.
- [9] A. Almomani *et al.*, “Phishing Website Detection With Semantic Features Based on Machine Learning Classifiers: A Comparative Study,” *International Journal on Semantic Web and Information Systems*, vol. 18,

- no. 1. 2022. doi: 10.4018/IJSWIS.297032.
- [10] J. Tanimu and S. Shiaeles, "Phishing Detection Using Machine Learning Algorithm," *Proc. 2022 IEEE Int. Conf. Cyber Secur. Resilience, CSR 2022*, pp. 317–322, 2022, doi: 10.1109/CSR54599.2022.9850316.
 - [11] V. Adeyemi Onih, "Phishing Detection Using Machine Learning: A Model Development and Integration," *Int. J. Sci. Manag. Res.*, vol. 07, no. 04, pp. 27–63, 2024, doi: 10.37502/ijsmr.2024.7403.
 - [12] M. S. I. Ovi, M. H. Rahman, and M. A. Hossain, "PhishGuard: A Multi-Layered Ensemble Model for Optimal Phishing Website Detection," *2024 6th Int. Conf. Sustain. Technol. Ind. 5.0, STI 2024*, 2024, doi: 10.1109/STI64222.2024.10951075.
 - [13] Sohaib Latif and Saher Pervaiz, "Detecting Phishing Attacks in Cybersecurity Using Machine Learning With Data Preprocessing and Feature Engineering," *Kashf J. Multidiscip. Res.*, vol. 2, no. 03, pp. 89–99, 2025, doi: 10.71146/kjmr335.
 - [14] M. Dharani, S. Badkul, K. Gharat, A. Vidhate, and D. Bhosale, "Detection of Phishing Websites Using Ensemble Machine Learning Approach," vol. 03012, pp. 1–5, 2021.
 - [15] M. A. Tamal, M. K. Islam, T. Bhuiyan, A. Sattar, and N. U. Prince, "Unveiling suspicious phishing attacks: enhancing detection with an optimal feature vectorization algorithm and supervised machine learning," *Front. Comput. Sci.*, vol. 6, no. January, 2024, doi: 10.3389/fcomp.2024.1428013.
 - [16] R. Kumar, R. Kumar, R. K. Sahu, R. Patra, and A. Ghosh, "Detection of Phishing Websites Using Machine Learning," *Smart Innov. Syst. Technol.*, vol. 313, pp. 317–330, 2023, doi: 10.1007/978-981-19-8669-7_29.
 - [17] I. D. Mienye and Y. Sun, "A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects," *IEEE Access*, vol. 10, no. September, pp. 99129–99149, 2022, doi: 10.1109/ACCESS.2022.3207287.
 - [18] C. Giraud-Carrier, *Combining Base-Learners into Ensembles*. Springer International Publishing, 2022. doi: 10.1007/978-3-030-67024-5_9.
 - [19] M. Al-Sarem *et al.*, "An optimized stacking ensemble model for phishing websites detection," *Electron.*, vol. 10, no. 11, pp. 1–18, 2021, doi:

10.3390/electronics10111285.

- [20] S. Shafieian and M. Zulkernine, “Multi-layer stacking ensemble learners for low footprint network intrusion detection,” *Complex Intell. Syst.*, vol. 9, no. 4, pp. 3787–3799, 2023, doi: 10.1007/s40747-022-00809-3.
- [21] Y. Mourtaji, M. Bouhorma, D. Alghazzawi, G. Aldabbagh, and A. Alghamdi, “Hybrid Rule-Based Solution for Phishing URL Detection Using Convolutional Neural Network,” *Wirel. Commun. Mob. Comput.*, vol. 2021, 2021, doi: 10.1155/2021/8241104.
- [22] C. Zhao, X. Yuan, J. Long, L. Jin, and B. Guan, “Chinese stock market Preprint not peer reviewed,” pp. 1–20, 2025.
- [23] A. Safi and S. Singh, “A systematic literature review on phishing website detection techniques,” *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 35, no. 2, pp. 590–611, 2023, doi: 10.1016/j.jksuci.2023.01.004.
- [24] E. S. Aung, C. T. Zhan, and H. Yamana, “A Survey of URL-based Phishing Detection,” pp. 1–8, 2019, [Online]. Available: <http://quadrodefertas.com.br/www1>.
- [25] R. Basnet and A. H. Sung, “Rule-Based Phishing Attack Detection Rule-Based Phishing Attack Detection,” no. October, 2016.
- [26] N. S. Zaini *et al.*, “Phishing detection system using machine learning classifiers,” *Indones. J. Electr. Eng. Comput. Sci.*, vol. 17, no. 3, pp. 1165–1171, 2020, doi: 10.11591/ijeecs.v17.i3.pp1165-1171.
- [27] M. Elsadig *et al.*, “Intelligent Deep Machine Learning Cyber Phishing URL Detection Based on BERT Features Extraction,” *Electron.*, vol. 11, no. 22, 2022, doi: 10.3390/electronics11223647.
- [28] Y. Wang, “Malicious URL Detection An Evaluation of Feature Extraction and Machine Learning Algorithm,” *Highlights Sci. Eng. Technol.*, vol. 23, pp. 117–123, 2022, doi: 10.54097/hset.v23i.3209.
- [29] D. Menaga, S. Vijay, V. Vignesh, and S. Ramalakshmi, “An Efficient Detection of Phishing Website using Machine Learning,” *Proc. 5th Int. Conf. Smart Electron. Commun. ICOSEC 2024*, vol. 11, no. 5, pp. 1189–1194, 2024, doi: 10.1109/ICOSEC61587.2024.10722540.
- [30] H. Ghuge, D. Ghuge, A. Rangneniwar, P. Samudra, and A. V Markad, “Real-

Time Detection of Malicious URLs Using Feature-Based Machine Learning Approaches,” vol. 13, no. 2, pp. 1–9, 2025.

- [31] R. Alazaidah *et al.*, “Website Phishing Detection Using Machine Learning Techniques,” *J. Stat. Appl. Probab.*, vol. 13, no. 1, pp. 119–129, 2024, doi: 10.18576/jsap/130108.
- [32] H. Gao *et al.*, “Machine learning in business and finance: a literature review and research opportunities,” *Financ. Innov.*, vol. 10, no. 1, 2024, doi: 10.1186/s40854-024-00629-z.
- [33] S. Alrefaai, G. Ozdemir, and A. Mohamed, “Detecting Phishing Websites Using Machine Learning,” *HORA 2022 - 4th Int. Congr. Human-Computer Interact. Optim. Robot. Appl. Proc.*, pp. 1–17, 2022, doi: 10.1109/HORA55278.2022.9799917.
- [34] A. U. Rehman, I. Imtiaz, S. Javaid, and M. Muslih, “Real-Time Phishing URL Detection Using Machine Learning †,” pp. 1–11, 2025.
- [35] J. Julio, M. Lamas, and R. W. Portillo, “Web architecture for URL-based phishing detection based on Random Forest , Classification Trees , and Support Vector Machine,” vol. 25, no. 69, pp. 107–121, 1988, doi: 10.4114/intartif.vol25iss69pp107-121.
- [36] F. Nthurima, A. Mutua, and W. S. Titus, “Detecting Phishing Emails Using Random Forest and AdaBoost Classifier Model,” vol. 6, no. 2, pp. 123–136, 2023.
- [37] P. M. Dinesh, M. Mukesh, B. Navaneethan, R. S. Sabeenian, M. E. Paramasivam, and A. Manjunathan, “Identification of Phishing Machine Learning Algorithm Attacks using,” vol. 04010, 2023.
- [38] D. Revaldo, “Implementation of Random Forest Classification and Support Vector Machine Algorithms for Phishing Link Detection,” vol. 8106, pp. 127–137, 2024.
- [39] R. Yang, K. Zheng, B. Wu, and C. Wu, “Phishing Website Detection Based on Deep Convolutional Neural Network and Random Forest Ensemble Learning,” pp. 1–18, 2021.
- [40] A. Fajar, S. Yazid, and I. Budi, “Enhancing Phishing Detection through Feature Importance Analysis and Explainable AI : A Comparative Study of

CatBoost , XGBoost , and EBM Models”.

- [41] L. Jovanovic, D. Jovanovic, and M. Antonijevic, “Improving Phishing Website Detection Using a Hybrid Two-level Framework for Feature Selection and XGBoost Tuning,” vol. 22, pp. 543–574, 2023, doi: 10.13052/jwe1540-9589.2237.
- [42] R. Alzubi, “Improving Web Security through Machine Learning : A Feature-Based Methodology for Detecting Phishing URLs,” vol. 15, no. 5, pp. 26845–26851, 2025.
- [43] R. B. Basnet, A. H. Sung, and Q. Liu, “Rule-Based Phishing Attack Detection”.
- [44] F. Salahdine, Z. El Mrabet, and N. Kaabouch, “Phishing Attacks Detection A Machine Learning-Based Approach,” 2021.
- [45] V. Sai and C. Telaprolu, “Analyzing URL Structure for Machine Learning : Feature Engineering and Classification Applications,” vol. 2024, pp. 1–5, 2024.
- [46] H. Dalianis, “Evaluation Metrics and Evaluation,” *Clinical Text Mining*. pp. 45–53, 2018. doi: 10.1007/978-3-319-78503-5_6.
- [47] Ž. Vujović, “Classification Model Evaluation Metrics,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 6, pp. 599–606, 2021, doi: 10.14569/IJACSA.2021.0120670.